# PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate `PNP-FORMAL-RECONSTRUCTION-STATUS-2026-10-13-281`

## The repository does not currently establish $P = NP$.

Public theorem emission is disabled. The current reconstruction boundary below is generated from the authoritative formal status, not a manually maintained description of an older builder prefix.

| | |
|---|---|
| CNF-SAT membership in NP | **true** |
| Complete all-input Cook-Levin builder | **true** |
| Concrete CNF-SAT NP-completeness | **true** |
| Complete polynomial serialized normalizer | **true** |
| Deterministic CNF-SAT membership in P | **false** |
| Eligible root theorem present | **false** |

Formal artefact coverage: 257 of 259 current scoped publication rows. This is not proof completion.

Risk-weighted proof completion estimate: 40 percent. Uncertainty range: 20 to 40 percent. Global proof gates closed: 0 of 5.

The estimate is not a probability that the route is correct or a time estimate. Exact theorem scope and remaining premises appear in the milestone ledger below.

| | |
|---|---|
| Compiled declarations | 68079 |
| Theorem-kind declarations | 37662 |
| Assumption-free theorem-kind declarations | 13972 |
| Project-specific axioms | 0 |
| Concrete publication gate passed | **false** |

Inventory SHA-256: `49cfcf6de14f560842766adef7fe9e54437ce57ac2a6c4ace650308164be890a`

Generated from the pinned compiled Lean environment, not from source-text parsing.

# Contents

**1 Complete source-derived serialized normalization** **2**

**2 Current authority and claim boundary** **2**

2.1 Verified correction to the unrestricted support rule . . . . . 2

2.2 Fixed windows do not certify global minimality . . . . . 2

**3 Concrete publication gate** **20**

3.1 Current concrete target and historical abstract bridge . . . . . 21

**4 Formal milestone ledger** **21**

**5 Compiled inventory summary** **211**

**6 Remaining formal blockers** **229**

**7 Historical report provenance** **229**

**8 Verification** **230**

# 1 Complete source-derived serialized normalization

The complete source-derived normalizer now has a concrete finite-machine implementation with polynomial encoded-input runtime and output bounds. Its exact serialized certificate language is equivalent to CNF-SAT on every source bitstring, including the checked malformed-input convention. All residual equations are retained. Satisfiable inputs admit short certificates, but certificate discovery is not provided.

A residual system packet is a problem to solve, not an unsatisfiability verdict. This construction does not prove the manuscript minimum-preserving NormalizeOrGain theorem, a general residual solver, unconditional ZeroSlack, exact polynomial PCCMin, or deterministic CNF-SAT membership in P. The fixed-weight proof estimate is unchanged. The publication ledger below pins the exact compiled interfaces and conservative scope.

# 2 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled `PNP` environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains $P = NP$, but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

## 2.1 Verified correction to the unrestricted support rule

The raw-port-complete compatible-support slack law in the historical manuscript has a kernel-checked counterexample. An eleven-gate whole circuit is semantically minimum, but its actual proper nine-gate cut has independent open minimum eight. Thus local slack is one while whole-circuit slack is zero. The smaller equivalent open replacement becomes cyclic when reconnected to the exterior.

This corrects the unrestricted reading of section 2.2, not the checked acyclic framed law or a properly restricted admissible-support law. Complete source-derived full-profile admissibility and first-loss routing remain open. No positive publication row or fixed proof-completion checkpoint is awarded for this finding. The historical manuscript is preserved unchanged.

## 2.2 Fixed windows do not certify global minimality

For every fixed circuit-independent support cap, the guarded-circuit family has a strictly smaller equivalent whole implementation while every proper extracted support is minimum among all equivalent independent-boundary NAND implementations. Disconnected selections, repeated records, sharing and constants are included. The actual complete bounded scan returns no accepted replacement; this follows symbolically, without executing exhaustive search.

Thus bounded local quietness does not imply global zero slack under any fixed cap. This does not decide $P$ versus $NP$ or exclude growing windows or global transformations. A fixed-cap candidate-count bound is not a complete polynomial-runtime theorem. Global coverage and full encoded-size and runtime bounds remain obligations of a repaired route.

This finding is distinct from the cyclic-reconnection counterexample above. No positive publication

row or fixed checkpoint is awarded, and the prior earned checkpoints do not assert fixed-window completeness. Historical manuscript artifacts remain unchanged.

The completed direct CNF verifier consumes the literal canonical pair of an encoded formula and assignment certificate. Lean proves universal acceptance equivalence, rejection equivalence, and absence of timeout at its explicit polynomial fuel bound, and constructs a proof-bearing `PolynomialTimeVerifier CNFSAT`. Thus `PNP.Concrete.CNFSAT` is in the concrete NP class. This is a verifier theorem, not a deterministic polynomial-time SAT algorithm.

The concrete Cook–Levin development proves that the generated CNF formula is semantically equivalent to ordinary raw verifier execution in both input modes. It now also bounds the actual canonical unary-indexed formula encoding by an explicit fixed-verifier polynomial evaluated only at external source-input length. That output-size theorem does not execute the formula builder as a raw finite machine, prove its construction runtime, or package a concrete polynomial reduction.

M230 now completes the all-input construction. For every concrete polynomial verifier and every ordinary raw bitstring, one finite machine executes the source-derived full loop and physical finalizer, and its ordinary output is exactly the original canonical formula. Total runtime and encoded output size are polynomial in the original input length. The machine-leaf `PolynomialTimeFunction`, its recursive `RawRefinement` output theorem, and the exact `PolynomialReduction` to `CNFSAT` are checked. No execution trace, route, finite family or correctness certificate is supplied. Seven public execution-side closures use only `propext` and `Quot.sound`; the reduction and its two endpoints additionally inherit the already permitted standard `Classical.choice` from formula semantics. The component descriptions below retain their original individual scope; their incomplete-builder limitations do not describe the completed M230 route.

M231 now packages concrete CNF-SAT NP-completeness. Every language in the concrete bounded-certificate NP class supplies its verifier through NP membership; the complete Cook–Levin construction gives its polynomial reduction to CNFSAT. Together with the existing concrete verifier, this proves the closed proposition `NPComplete CNFSAT`. The hardness theorem is `PNP.Concrete.CookLevin.cnfSAT_np_hard`; the closed endpoint is `PNP.Concrete.CookLevin.cnfSAT_np_complete`. Both closures use only the existing Lean standard axioms `Classical.choice`, `Quot.sound` and `propext`. No reduction or correctness certificate is supplied, and no deterministic CNFSAT-in-P result follows.

### Separate progress measurements

Formal artefact coverage: 257 of 259 current scoped publication rows earned. This is evidence-ledger coverage, not proof completion; its denominator can grow. The risk-weighted proof completion estimate is 40 percent, with an uncertainty range of 20 to 40 percent. Global gates closed: 0 of 5. These values are read from the canonical fixed-checkpoint progress ledger. Only named fixed-checkpoint transitions change the estimate; their individual reviews are recorded in the canonical ledger history. The estimate is not a probability of correctness or a time estimate. Deterministic SAT, unconditional ZeroSlack, full polynomial PCCMin and the eligible root theorem remain open.

The concrete CNF-to-NAND development traverses any decoded formula structurally and constructs an intrinsically topological NAND circuit without querying satisfiability. Lean proves exact valuation and satisfiability semantics, strict canonical decoder inversion, well-formed output bytes, the exact gate count, a quadratic serialized-output bound in external input length, and fail-closed equivalence on every bitstring. It also composes semantically with the concrete locked-NAND threshold builder. A subsequent fixed parser/carrier/controller work graph now implements exactly that pure compiler on every bitstring within one external-input polynomial. Its compiled machine never times out at the advertised bound, retains literal `RawRefinement`, packages `PolynomialReduction CNFSAT EncodedNANDSAT`, and explicitly composes with the strict locked-NAND reduction. The report-facing theorem `PNP.Main.locked_nand_threshold` now publishes that composition as a uniform all-bitstring reduction from `CNFSAT` to `EncodedLockedNANDThreshold`; its closure uses only `propext`

and `Quot.sound`. This reduction still does not decide CNF-SAT, put the locked target in P, discharge residual-band/ZeroSlack/PCCMin, or prove $P = NP$.

The new rectangular schedule allocates exact constraint, clause, token, and raw-bit opportunities without reading the already-materialized program, formula, token encoding, or encoded formula as schedule inputs. Filtering populated slots reproduces those canonical objects, and the raw-bit slot count is exactly the external encoded-size polynomial. This remains a pure specification: no theorem charges constant time per slot, implements a raw builder, or proves a construction-runtime bound.

The direct formula cursor decodes constraint, clause, token, and bit coordinates without first constructing those complete canonical lists. Its nested options distinguish out-of-range lookup from valid empty padding, and Lean proves exact prefix, full, one-step-short, terminal, and excess-fuel behavior. Filtering the exact full cursor output yields the canonical encoding. These are specification-level evaluation theorems, not a constant-time raw slot interpreter or raw builder.

The first literal builder stage is a fixed 19-rule work machine. Starting at the proved all-input framer endpoint, it restores every source bit and appends exactly one unary tally symbol per bit in fresh right-side workspace. Its exact work cost is $2n^2 + 4n + 2$, and six-for-one compilation gives the exact raw cost $12n^2 + 24n + 12$. Malformed scan symbols and one-step-short fuel time out. This is only length preparation: it emits no formula bits and does not compose or refine a complete builder.

The executable input prefix now places the total framer and that tally in disjoint state images, with one total nine-symbol launch table between them. From every ordinary raw bitstring it reaches the same preserved-input unary-tally endpoint in exactly the sum of the two proved local traces plus one launch, and compilation is bounded by $18n^2 + 63n + 93$ in external input length. First-match isolation, timeout for the unused `zeroOne` tally scan symbol, and one-step-short timeout are explicit. No formula bit, cursor coordinate, or construction-time refinement is produced.

The standalone token appender carries one of the four canonical CNF tokens in its finite control state, scans the represented source, exact tally, and existing token suffix, writes exactly the selected two-bit symbol, and rewinds to the source focus. Its generic theorem quantifies every input, canonical prior token list, request, and exterior-left region. The distinguished start requests `T`; Lean proves direct formula-bit slots zero and one are populated true bits and that `CNFToken.t.bits = problem.encodedFormula.take 2` for every concrete tableau problem. The compiled first-token bound is $24n + 48$. Malformed tally/output symbols and one-step-short fuel time out.

The composed first-token prefix renames the complete 116-rule input prefix and complete 59-rule appender into disjoint injective state images and places nine symbol-preserving rules first in the literal table. Only the appender's accept/reject images are global halts. Every raw bitstring follows the exact framing, tally, launch, and append trace to the preserved workspace containing `[CNFToken.t]`. Its compiled external bound is $18n^2 + 87n + 147$, and blank-equivalent raw tapes reach the same encoded endpoint. Ending at the prefix endpoint before launch, malformed prefix or appender phases, and one-step-short fuel all remain timeout. This emits only two fixed formula bits: it does not compute the remaining header, interpret a dynamic cursor, or emit the complete formula.

The complete-header composition continues from that endpoint with a structurally generated unary `NatPolynomial` evaluator, a 16-rule unary-root controller, two complete 59-rule appender copies, and five total nine-symbol bridges under pairwise-disjoint injective state renamings. Its literal table has exactly `363 + evaluator.ruleCount` rules. Every raw input emits exactly `FormulaWidth` copies of `T` followed by `F`; the emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 1))`, and an external `NatPolynomial` bounds the compiled run. This is the complete answer-independent width header only: dynamic cursor and formula-body emission, a complete builder refinement, and a concrete polynomial reduction remain absent.

The body-start composition continues from the complete header with a unary evaluator for the retained next-token coordinate, two total nine-symbol bridges, and one complete separator appender. Its literal table has exactly 440 plus the two evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F` and `Sep`; its bits equal the canonical encoded-formula

prefix through that separator. The token coordinate is two past the formula variable slot bound and the corresponding bit coordinate is twice it. This retained coordinate is data, not an executable dynamic cursor; all subsequent body emission remains absent.

The first-literal composition continues from that endpoint with a third unary evaluator, three total nine-symbol bridges, and complete fixed `T` and `F` appender copies. Its literal table has exactly 585 plus the width, body-start next-slot, and first-literal next-slot evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F`, `Sep`, `T`, and `F`. A constructive schedule proof pins the final pair to the positive sign and unary-zero terminator of the first at-least-one shape-clause literal: positive variable zero. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 4))`. The retained next token coordinate is four past the formula variable slot bound, but it remains data rather than an executable dynamic cursor; the rest of the formula body remains absent.

The first-clause composition continues from that endpoint with a fourth unary evaluator and a fixed 535-rule tail appender assembled from eight complete token-appender copies and seven total bridges. The global literal table has exactly 1138 plus the width, body-start next-slot, first-literal next-slot, and first-clause next-slot evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F`, `Sep`, `T`, `F`, `T`, `T`, `F`, `T`, `T`, `T`, `F`, and `Finish`. A constructive schedule proof identifies this as the complete positive at-least-one shape clause on variables zero, one, and two. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 12))`; the retained next token coordinate is twelve past the formula variable slot bound. It remains data rather than an executable dynamic cursor, and the remaining formula body is absent.

One literal token-cursor step continues from that first-clause endpoint. A total nine-symbol launch enters a fixed 45-rule cursor-advance table, giving a global table of exactly 1192 plus the four inherited unary-evaluator rule counts. Every raw input preserves the emitted clause while moving the retained unary coordinate from `FormulaVariableSlotBound + 12` to `FormulaVariableSlotBound + 13`. The direct decoder and token-level specification cursor prove that the consumed coordinate is the first in-range padding opportunity and therefore emits no token. Including launch, the suffix costs exactly `2*cursorWord.length + 8` work steps; the compiled run is bounded by `FirstClausePrefix.rawTimeBound + 48 + 12*cursorWord.length`. Malformed scratch, the unlaunched predecessor endpoint, and one-step-short total fuel remain timeout. This is one fixed padding transition, not a general cursor loop or arbitrary schedule decoder.

The remaining-padding composition continues from that one-step endpoint with two structurally generated unary evaluators and one fixed 25-rule countdown controller. Writing `D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6)`, its literal table has exactly 1244 plus the six inherited and generated evaluator rule counts. Every raw input traverses all `D` remaining first-clause padding coordinates without emitting a token and reaches `FormulaVariableSlotBound + 1 + FormulaTokensPerClause`. Direct lookup at that coordinate is proved to return `Sep`, and the recursive specification run agrees across every no-emission step and the following separator observation. The exact compiled run has a verifier-fixed external input-size polynomial bound; malformed countdown scratch/root states, the unlaunched endpoint, and one-step-short fuel remain timeout. This identifies the second-clause boundary but does not emit the second clause or implement a general formula cursor.

The second-clause-separator composition continues with a selected 59-rule `Sep` appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance. Its literal table has exactly 1366 plus the six inherited and generated evaluator rule counts. Every raw input emits the canonical prefix through the separator beginning clause two and advances the retained unary coordinate by one; direct lookup proves the following token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 13))`. The compiled run is bounded by the predecessor bound plus `246 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The combined 56-declaration audit has 15 empty closures, 11 using only `propext`, and 30 using only `propext` and `Quot.sound`. Malformed appender tally/output, malformed cursor scratch, both unlaunched endpoints, and one-step-short total fuel remain timeout. This is one fixed populated transition, not a general dynamic cursor or

remaining-body builder, and it does not emit the following `F`.

Two further fixed compositions emit the complete negative literals on variables zero and one in clause two. They use selected `F`, `T`, and `F` appender copies, fixed cursor advances, and total symbol-preserving bridges. Exact schedule proofs identify each sign, unary index, and literal terminator; the second composition retains the following `Finish`.

The complete-second-clause composition emits that retained `Finish` with one selected 59-rule appender and advances the cursor once. Its suffix has exactly 113 rules, while the global table has exactly 2098 plus the six inherited and generated evaluator rule counts. Every raw input emits bits equal to `encodedFormula.take (2 * (FormulaWidth + 19))`. Direct lookup proves that the executed coordinate is the clause terminator and the retained next coordinate is in-range padding. The new external raw bound is the predecessor bound plus `390 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The exact 57-line audit has 15 empty, 10 `propext`, and 32 `propext/Quot.sound` closures. This completes clause two but does not by itself traverse its padding or implement a general formula cursor.

The second-clause-padding composition evaluates `D = (V - 1) * (V + 6) + 5`. With `C` abbreviating `formulaTokensPerClause`, Lean proves `D = C - 7`, traverses exactly those remaining padding opportunities with the reused 25-rule countdown, and evaluates the target coordinate. Three total bridges and two fixed unary evaluators yield a global table with exactly 2150 plus the six inherited/generated evaluator counts and the two new evaluator counts. Every raw input reaches `V + 1 + 2 * formulaTokensPerClause`; every traversed coordinate is direct padding and the retained coordinate is the third clause's opening `Sep`. No token is emitted, so the bits remain `encodedFormula.take (2 * (FormulaWidth + 19))`. The exact 68-line audit has 26 empty, 9 `propext`, and 33 `propext/Quot.sound` closures. This reaches clause three only as a retained coordinate; it does not emit the separator or implement a general formula cursor.

The third-clause-separator composition reuses the selected 59-rule `Sep` appender and fixed 45-rule cursor advance behind one new total bridge. Its global table has exactly 2272 plus the eight inherited evaluator rule counts. Every raw input emits the separator beginning clause three, advances the retained coordinate to `V + 1 + 2 * formulaTokensPerClause + 1`, and proves the following direct token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 20))`. Its external bound adds `330 + 24*n + 12*FormulaWidth + 12*cursorWord.length` to the predecessor bound. The exact 56-line audit has 14 empty, 11 `propext`, and 31 `propext/Quot.sound` closures. This emits one fixed separator but does not emit the following `F`, complete clause three, or implement a general formula cursor.

The third-clause-first-literal composition reuses the audited 235-rule two-`F` appender/cursor suffix behind one total bridge. Its global table has exactly 2516 plus the eight inherited evaluator rule counts. Every raw input emits the complete negative literal on variable zero in clause three and retains `V + 1 + 2 * formulaTokensPerClause + 3`, the negative-sign coordinate on variable two. Constructive schedule proofs identify the third excluded pair as zero and two and prove that the emitted sign, unary-zero terminator, and retained sign are all `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 22))`. Its external bound adds `732 + 48*n + 24*FormulaWidth + 24*cursorWord.length` to the separator bound. The exact 87-line audit has 24 empty, 18 `propext`, and 45 `propext/Quot.sound` closures. This emits one fixed literal but does not emit the next negative sign, complete clause three, or implement a general formula cursor.

The third-clause-second-literal composition adds a fixed 479-rule `F T T F` appender/cursor suffix behind one total bridge. Its nested component tables have 113, 235, 357, and 479 rules, and its global table has exactly 3004 plus the eight inherited evaluator rule counts. Every raw input emits the complete negative literal on variable two and retains `V + 1 + 2 * formulaTokensPerClause + 7`, the following clause-terminator coordinate. Direct and specification schedule proofs establish `F`, `T`, `T`, `F`, and the following `Finish`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 26))`. Its external bound adds `1752 + 96*n + 48*FormulaWidth + 48*cursorWord.length` to the

first-literal bound. The exact 145-line audit has 46 empty, 32 `propext`, and 67 `propext/Quot.sound` closures. This emits the second fixed literal but does not emit the following `Finish`, complete clause three, or implement a general formula cursor.

The complete-third-clause composition emits that retained `Finish` with one selected 59-rule appender and advances the cursor once. Its suffix has exactly 113 rules, while the global table has exactly 3126 plus the eight inherited evaluator rule counts. Every raw input emits bits equal to `encodedFormula.take (2 * (FormulaWidth + 27))`. Direct lookup proves that the executed coordinate is the clause terminator and the retained next coordinate is in-range padding. The new external raw bound is the predecessor bound plus `498 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The exact 57-line audit has 14 empty, 10 `propext`, and 33 `propext/Quot.sound` closures. This completes clause three but does not by itself traverse its padding or implement a general formula cursor.

The third-clause-padding composition evaluates `D = (V - 1) * (V + 6) + 4`. With `C` abbreviating `formulaTokensPerClause`, Lean proves `D = C - 8`, traverses exactly those remaining padding opportunities with the reused 25-rule countdown, and evaluates the target coordinate. Three total bridges and two fixed unary evaluators yield a global table with exactly 3178 plus the eight inherited/generated evaluator counts and the two new evaluator counts. Every raw input reaches `V + 1 + 3 * formulaTokensPerClause`; every traversed coordinate is direct padding and the retained coordinate is the fourth clause's opening `Sep`. No token is emitted, so the bits remain `encodedFormula.take (2 * (FormulaWidth + 27))`. The exact 68-line audit has 26 empty, 9 `propext`, and 33 `propext/Quot.sound` closures. This reaches clause four only as a retained coordinate; it does not emit the separator or implement a general formula cursor.

The fourth-clause-separator composition reuses the selected 59-rule `Sep` appender and fixed 45-rule cursor advance behind one outer total nine-symbol bridge. Its 113-rule suffix yields a global table with exactly 3300 plus the ten inherited/generated unary-evaluator rule counts. Every raw input emits exactly the separator beginning clause four, advances the retained coordinate to `V + 1 + 3 * formulaTokensPerClause + 1`, and proves the following direct token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 28))`. Its external bound is the predecessor bound plus `426 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The exact 56-line audit covers all 48 new public declarations plus eight reused interfaces, with 14 empty, 11 `propext`, and 31 `propext/Quot.sound` closures. This emits one fixed separator but does not emit the following `F`, complete clause four, or implement a general formula cursor.

The fourth-clause-first-literal composition reuses the fixed 357-rule `F T F` appender/cursor suffix behind one outer total nine-symbol bridge. The global table has exactly 3666 plus the ten inherited/generated unary-evaluator rule counts. Every raw input emits the complete first negative literal on variable one, advances the retained coordinate to `V + 1 + 3 * formulaTokensPerClause + 4`, and proves the following direct token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 31))`. Its external bound is the predecessor bound plus `1422 + 72*n + 36*FormulaWidth + 36*cursorWord.length`. The exact 115-line audit covers 97 new declarations, 16 reused suffix interfaces, and two dead-state facts, with 33 empty, 25 `propext`, and 57 `propext/Quot.sound` closures. This emits one fixed literal but does not emit the second literal, complete clause four, or implement a general formula cursor.

The fourth-clause-second-literal composition reuses the fixed 479-rule `F T T F` appender/cursor suffix behind one outer total nine-symbol bridge. The global table has exactly 4154 plus the ten inherited/generated unary-evaluator rule counts. Every raw input emits the complete second negative literal on variable two, advances the retained coordinate to `V + 1 + 3 * formulaTokensPerClause + 8`, and proves the following direct token is `Finish`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 35))`. Its external bound is the predecessor bound plus `2232 + 96*n + 48*FormulaWidth + 48*BuilderFourthClauseSeparatorStep.cursorWord.length`. The exact 147-line audit covers 124 new declarations, 21 reused suffix interfaces, and two dead-state facts, with 46 empty, 32 `propext`, and 69 `propext/Quot.sound` closures. This emits one fixed literal but does

not emit the following `Finish`, complete clause four, or implement a general formula cursor.

The complete-fourth-clause composition reuses a selected 59-rule `Finish` appender and the fixed 45-rule cursor behind two total nine-symbol bridges. The selected suffix has exactly 113 rules and the global table has exactly 4276 plus the ten inherited/generated unary-evaluator rule counts. Every raw input emits the terminator that completes clause four, advances the retained coordinate to `V + 1 + 3 * formulaTokensPerClause + 9`, and proves the following direct token is padding. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 36))`. Its external bound is the predecessor bound plus `618 + 24*n + 12*FormulaWidth + 12*BuilderFourthClauseSeparatorStep.cursorWord.length`. The exact 57-line audit covers 55 new public declarations and two dead-state facts, with 14 empty, 10 `propext`, and 33 `propext/Quot.sound` closures. This completes clause four but does not traverse its padding or implement a general formula cursor.

At the current builder frontier, the second-constraint-first-literal terminator composition reuses the selected 59-rule `F` appender and fixed 45-rule cursor behind one outer total nine-symbol bridge. The global table has exactly 5164 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input emits exactly the terminating `F`, preserves `encodedFormula.take (2 * (FormulaWidth + 42))`, and retains `V + 1 + Q*C + 6`. A constructive schedule case split proves that the following direct token is `Finish` when the tape width is one and the positive `T` beginning the next literal at wider widths; neither token is emitted. The external bound is the predecessor bound plus `594 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The 56-declaration audit has 14 empty, 11 `propext`, and 31 `propext/Quot.sound` closures. This completes one literal, not the second constraint or the formula builder.

The successor-token composition now evaluates the represented tableau width and selects one of two literal transitions inside a fixed 93-rule branch table. At width one it appends `Finish`; at every wider width it appends `T`. The global table has exactly 5284 plus the eighteen inherited/generated unary-evaluator rule counts, preserves `encodedFormula.take (2 * (FormulaWidth + 43))`, and retains `V + 1 + Q*C + 7`. The next opportunity is proved to be padding at width one and unary `T` at wider widths, but is not emitted. The external bound is the predecessor bound plus `600 + 24*n + 12*FormulaWidth + 12*width + 12*widthRootPrefixLength + 6*widthWorkSteps + 6*targetWorkSteps`. The 82-declaration audit has 37 empty, 12 `propext`, and 33 `propext/Quot.sound` closures, with no project axiom or `Classical.choice`. This is one width-selected token transition, not the remaining second constraint or a complete formula builder.

The next four opportunity machines use the same reviewed 93-rule optional-appender table. At width one each schedule position is padding, so its direct skip bridge emits nothing. At wider widths the first appends the first unary `T` of the second literal and the second appends its second unary `T`; the third appends its third unary `T`, and the fourth appends its fourth unary `T`. The fourth machine has exactly 5764 literal rules plus the twenty-six inherited/generated unary-evaluator rule counts, preserves `encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 4))`, and retains `V + 1 + Q*C + 11`. The following position is proved to be padding at width one and the terminating `F` at wider widths, but is not consumed. Its external bound is the third opportunity bound plus `648 + 24*n + 12*FormulaWidth + 12*width + 12*widthRootPrefixLength + 6*widthWorkSteps + 6*targetWorkSteps`. The 82-declaration audit covers all 66 new declarations, fourteen reused optional-appender interfaces, and two strengthened schedule boundaries: 37 closures are empty, 12 use only `propext`, and 33 use only `propext` and `Quot.sound`.

The fifth opportunity machine uses a new 93-rule optional-terminator controller over the audited token-appender table. At width one it consumes padding and emits nothing; at wider widths it appends the second literal's terminating `F`. Its complete table has exactly 5884 literal rules plus twenty-eight inherited/generated unary-evaluator rule counts, preserves `encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 5))`, and retains `V + 1 + Q*C + 12`. The following position is padding at width one and the opening unary `T` of the following literal at wider widths, but is not consumed. Its external bound is the fourth opportu-

nity bound plus `660 + 24*n + 12*FormulaWidth + 12*width + 12*widthRootPrefixLength + 6*widthWorkSteps + 6*targetWorkSteps`. The 82-declaration audit covers 66 new outer declarations, fourteen new optional-terminator interfaces, and two strengthened schedule boundaries: 37 closures are empty, 12 use only `propext`, and 33 use only `propext` and `Quot.sound`.

The sixth opportunity machine reuses the reviewed 93-rule optional-appender controller. At width one it consumes padding and emits nothing; at wider widths it appends the following literal's opening positive `T`. Its complete table has exactly 6004 literal rules plus thirty inherited/generated unary-evaluator rule counts, preserves `encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 6))`, and retains `V + 1 + Q*C + 13`. The following position is padding at width one and the first unary-index `T` of the following literal at wider widths, but is not consumed. Its external bound is the fifth opportunity bound plus `672 + 24*n + 12*FormulaWidth + 12*width + 12*widthRootPrefixLength + 6*widthWorkSteps + 6*targetWorkSteps`. The 82-declaration audit covers 66 new declarations, fourteen reused optional-appender interfaces, and two strengthened schedule boundaries: 37 closures are empty, 12 use only `propext`, and 33 use only `propext` and `Quot.sound`.

The seventh opportunity machine reuses the reviewed 93-rule optional-appender controller. At width one it consumes padding and emits nothing; at wider widths it appends the first unary-index `T` of the following literal. Its complete table has exactly 6124 literal rules plus thirty-two inherited/generated unary-evaluator rule counts, preserves `encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 7))`, and retains `V + 1 + Q*C + 14`. The following position is padding at width one and the second unary-index `T` of the following literal at wider widths, but is not consumed. Its external bound is the sixth opportunity bound plus `684 + 24*n + 12*FormulaWidth + 12*width + 12*widthRootPrefixLength + 6*widthWorkSteps + 6*targetWorkSteps`. The 82-declaration audit covers 66 new declarations, fourteen reused optional-appender interfaces, and two strengthened schedule boundaries: 37 closures are empty, 12 use only `propext`, and 33 use only `propext` and `Quot.sound`.

Separately, the global locked-NAND construction now has both final semantic branches and the exact typed threshold. A strict version-zero grammar serializes source circuits, the complete full candidate, and the source-derived baseline. Direct normalization semantics, codec round trips, fail-closed malformed input, and the pure all-bitstring source/target equivalence are proved. The 48-declaration audit has four empty closures, 37 using only `propext`, and seven using only `propext` and `Quot.sound`. That module is a pure semantic transformation and does not by itself supply an executable parser or emitter.

The following strict-v0 source-parser milestone supplies a direct nine-symbol machine with 228 control states and 2,052 pairwise query-distinct literal rules. For every input bitstring, its exact execution accepts exactly the valid source encodings, preserves valid source bytes verbatim, and returns the empty word for invalid grammar or references. The compiled machine cannot time out within $6 \cdot 4096 \cdot (n+1)^3$ raw transitions. It is packaged as a polynomial-time language machine, a nonexpanding polynomial-time validation function, and the validator program's exact leaf raw-machine refinement.

The subsequent target-emitter milestone supplies a literal finite controller with an exact all-input polynomial work bound, a separate output-size bound, exact target bytes, and recursive raw-machine refinement for the strict parser/emitter composition. Malformed grammar and intrinsically invalid references remain fail closed at that strict boundary.

The present package binds that exact composed function and the already-proved encoded language equivalence as a concrete `PolynomialReduction` from `EncodedNANDSAT` to `EncodedLockedNANDThreshold`. It preserves exact function identity, output bytes, a `ReducesTo` witness, and recursive raw-machine refinement. Its 16-declaration audit has two empty closures, two using only `propext`, and twelve using only `propext` and `Quot.sound`. The report-facing `PNP.Main.locked_nand_threshold` theorem now composes the CNF-to-NAND and strict locked-

NAND reductions into an exact all-bitstring `ReducesTo CNFSAT EncodedLockedNANDThreshold` result. Its axiom transcript contains only `propext` and `Quot.sound`, not the legacy abstract string-handle assumption. The M186 compatibility boundary now makes the report-facing SAT and locked-NAND languages definitionally equal to these concrete endpoints, reuses the compiled CNF verifier and reduction, and removes the duplicate locked-NAND axiom and caller trust field. Its named endpoint also uses only `propext` and `Quot.sound`. The M187 compatibility boundary now defines the report-facing residual-band endpoint as the concrete encoded direct-wire minimum-threshold predicate, proves exact arbitrary-candidate semantics, and replaces the supplied locked-to-residual edge with an identity polynomial reduction. The former residual-band language axiom is absent; the exhaustive reference minimum is still only a semantic specification. The M188 boundary packages an explicit `PCCMinLoopCertificate`, checks a canonical identifier structurally, proves generated-package acceptance and mismatched-identifier rejection, and removes the two former PCCPack declarations from project-specific proof authority. Existence, exactness, and polynomial construction of the loop certificate remain open. The M189 boundary separately defines a total dependent oracle with only strict-gain, exact-minimum, and ZeroSlack outcomes and proves that transparent recursion on residual slack preserves semantics, terminates at an exact minimum, and uses no more gain branches than the starting slack. All eight focused declarations are assumption-free. The M190 boundary then composes a proof-bearing NormalizeOrGain stage with that oracle. Semantic normalization cannot increase gate count, so subsequent oracle gains lift to strict gains from the original implementation; exact-minimum and ZeroSlack endpoints transport back through normalization. All ten focused declarations are assumption-free. The total normalizer and oracle remain explicit arguments, and the exhaustive regression fixtures are not polynomial algorithms. The M191 boundary then reconstructs the oracle order itself: HResolve precedes BudgetResolve, arbitrary finite selector rows are scanned through every canonical rank, gains return immediately, and ZeroSlack requires a complete typed-blocker ledger. The fifteen focused declarations contain no project axiom or Classical choice; their executable membership proofs use only the permitted Lean-standard quotient and propositional-extensionality foundations. The resolver algorithms, selector rows, realizer, blocker semantics, and ZeroSlack closure remain explicit inputs. The M192 boundary then replaces the arbitrary selector rows and proof-bearing realizer with one complete checker-accepted data-only Packet table. It derives every rank row by filtering the exhaustive canonical handle list with the table-owned rank map, reconstructs strict gain only from a valid unit-charge blueprint, and retains checked HN, budget, or lower-seed blockers otherwise. The fifteen focused declarations use no project axiom or Classical choice. The grouped family, rank assignment, claims, resolver algorithms, blocker semantics, and complete-silence-to-ZeroSlack implication remain explicit supplied inputs; no encoded-size polynomial bound is proved. The M193 boundary then removes that last opaque silence-to-ZeroSlack callback: accepted claims plus rank-row silence are reflected into the executable selector-silence checker, and checked HB no-outcome closure proves every canonical handle nonfaithful. One explicit premise—that positive residual slack constructs a faithful canonical selector—now supplies the exact remaining contradiction edge and yields a conditional kernel-checked ZeroSlack result. All fifteen focused declarations avoid project axioms and Classical choice. The positive-slack bridge, terminal family and table, resolvers, normalizer, and encoded-size polynomial construction remain supplied, so this is not unconditional ZeroSlack or polynomial PCCMin. A deterministic target decider, concrete CNFSAT NP-hardness transport, ZeroSlack/PCCMin completion, and $P = NP$ remain unproved.

The residual-gain chain milestone reconstructs the iteration-count edge from legacy report Sections 16 and 17. For every finite disclosed chain whose adjacent implementations pass the strict equivalent-gain checker, Lean proves that the endpoint residual slack plus the chain length is at most the starting residual slack. The endpoint remains semantically equivalent and has the same exhaustive reference minimum. The already-proved locked-candidate bound then specializes every accepted chain to at most four gains. All 12 declarations in the universal module have empty axiom closure; the four locked declarations use only `propext` and `Quot.sound`. This does not find a route, certify stopping, prove ZeroSlack or exact minimization, or establish a polynomial checker or PCCMin

runtime.

The global strict-gain stopping milestone reconstructs the corresponding semantic endpoint condition from legacy report Section 16. For every finite direct-wire implementation, positive residual slack is equivalent to the existence of some strictly smaller semantically equivalent implementation. Zero slack and semantic minimality are each equivalent to global absence of such an implementation. A verified chain endpoint with separately proved global no-gain evidence therefore has zero slack and packages an exact minimum result equivalent to the starting implementation. All 12 public declarations have empty axiom closure. The positive witness comes from exhaustive reference minimization, so this is not a route generator or polynomial stopping algorithm; finite-list failure still cannot establish global absence, and the report's ZeroSlack certificate, PCCMin exactness/runtime, SAT-in-P conclusion, and (P=NP) theorem remain unproved.

The terminal full-carrier milestone reconstructs the direct-wire full-mode part of the terminal whole-carrier bridge from legacy report Section 8. A terminal realization agrees with the complete implementation on every input and output coordinate; terminalization preserves exact gate count. Lean states attainment and the universal lower bound independently, proves that the terminal minimum equals the exhaustive reference minimum (the direct-wire terminal `RW-MuBridge`), and proves that every cheaper whole-span realization gives strict residual descent. Positive slack is equivalent to existence of one and zero slack to its absence. All 22 public declarations have empty axiom closure. The quotient carrier and mode firewall, proper supports, saturation, BCEL/BN2–BN6, selector completeness, ZeroSlack/PCCMin, polynomial runtime, SAT-in-P, and $P = NP$ remain unproved.

The current Packet frontier assigns each exact grouped-footprint payload selector a canonical finite handle: its position in the explicit grouped BN6 family. Duplicate-free footprints make decoding injective, and every decoded footprint retains carrier containment, length at least two, and its original grouped cell-and-atom payload witness. Every payload selector has exactly one handle, with the pair, balanced-triple, and full-span alternatives unchanged. These input-list positions are not the manuscript's bit encoding or a polynomial selector universe. They do not prove manuscript-level selector faithfulness or compatibility, construct a selector realizer or route, derive the grouped family from a terminal candidate, or establish polynomial enumeration/runtime, ZeroSlack, PCCMin, SAT-in-P, or $P = NP$.

The next bounded frontier gives each such handle one canonical unary bitstring: position $i$ is $i$ one-bits followed by a zero. Its total decoder rejects missing delimiters, trailing bits, and positions beyond the explicit list. Lean proves exact round trip, injectivity, canonical successful decoding, exact code length, unique accepted codes, retained payload evidence, and every Packet branch. The length bound is by the supplied grouped-family list, not encoded circuit size. This does not establish polynomial enumeration/runtime, encode atom or payload data, prove manuscript-level faithfulness or compatibility, construct a realizer or route, derive the family, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, or prove $P = NP$.

The latest bounded frontier runs each accepted code through that total decoder and materializes its exact cell in the supplied grouped family together with the canonical first original positive payload atom of the nonempty cell. Lean proves that every result re-encodes to the exact input, retains source-family membership and footprint equality, and preserves every Packet branch; failure is exactly decoder failure. This is deterministic source-payload lookup, not the manuscript's gain-or-blocker selector realizer. It does not serialize atom or payload data, construct a replacement circuit or route, prove selector faithfulness or compatibility, derive the family, establish an encoded-size or polynomial-runtime bound, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, or prove $P = NP$.

The new checked frontier specializes those source payloads to direct-wire implementations and scans every original atom in the exact selected cell with the executable strict-equivalent-gain checker. An accepted code produces either an original payload atom carrying a genuine strict equivalent gain, and therefore strict residual descent, or proof that the selected cell contains no such candidate. Decoder

rejection remains exact and every Packet branch is preserved. The candidates and grouped family are explicit inputs. Cell-local no-gain is not an HN, budget, or lower-rank manuscript blocker, global minimality, or ZeroSlack. This does not construct replacement candidates, prove selector faithfulness or compatibility, derive the family, establish an encoded-size or polynomial-runtime bound, complete PkgC or global routes, put SAT in P, or prove $P = NP$.

The latest checked frontier enumerates every canonical input-relative selector handle in one supplied explicit grouped family and runs that complete source-cell candidate scan at every handle. The proof-bearing outcome is either a canonical handle and original source atom carrying a genuine strict equivalent gain, with an accepted canonical code and strict residual descent, or exact no-gain throughout that supplied selector universe. The original pair, balanced-triple, and full-span Packet conclusion is retained literally. This family-wide no-gain result is not manuscript selector silence, an HN, budget, or lower-rank blocker, global minimality, or ZeroSlack. The family and candidates remain explicit inputs. No selector faithfulness, compatibility, replacement construction, charge-surplus connection, encoded-size bound, polynomial runtime, complete PkgC or global route, SAT decider, or proof of $P = NP$ follows.

The current conditional frontier isolates the precise additional premise that would let the exhaustive family scan stop semantically. An explicit gain-coverage certificate must map every strict equivalent implementation to an original payload atom in one canonical selector source cell. Under exactly that premise, the checked scan returns either such a source-atom gain with strict residual descent or a proof-bearing semantic minimum and zero residual slack. Every Packet branch is still preserved. An empty-family positive-slack regression proves that scan silence alone cannot supply the certificate. This is not unconditional ZeroSlack: the certificate is not constructed from terminal data, and no selector faithfulness, compatibility, replacement or typed-blocker construction, HB/rank closure, encoded-size bound, polynomial runtime, global PkgC, PCCMin, SAT decider, or proof of $P = NP$ follows.

The finite arithmetic kernel of manuscript Section 14 `R-ChargeSurplus` is now checked over arbitrary support and replacement charge ledgers. Exact occurrence permutations preserve multiplicity, every paired replacement and support occurrence has equal weight, and an unmatched positive support charge forces strict replacement weight. If those totals are independently proved to equal the replacement and current NAND gate counts and semantic equivalence is proved separately, Lean derives a genuine strict equivalent gain and strict residual descent. Neither strict inequality is a premise. Duplicate support reuse and absence of an unmatched occurrence are negative regressions. The replacement circuit, ledgers, pairing, accounting, and semantics are not constructed from terminal data, so this is not the complete realizer, not unconditional ZeroSlack, and not a polynomial PCCMin, SAT decider, or proof of $P = NP$.

The checked unit-charge blueprint frontier now derives those finite ledgers mechanically for one explicit replacement format. A data-only blueprint contains a replacement implementation, an occurrence pairing, and unmatched current-gate occurrences. A constructive remove-first Boolean checker is proved exactly equivalent to natural-number list permutation without a `Classical.choice` dependency. It validates canonical unit-weight gate occurrence ledgers, a nonempty unmatched remainder, and semantic equivalence; acceptance constructs the generic charge realization, strict equivalent gain, and residual descent. The validator scans every original blueprint atom behind every canonical handle in one supplied explicit grouped family and preserves the complete Packet alternatives. The blueprints and family remain inputs, and its unresolved branch is only supplied-family validator silence, not `BotHN`, `BotBUD`, a lower-rank `BotSeed`, global no-gain, ZeroSlack, or a polynomial realizer. Blueprint construction from terminal data, faithfulness, compatibility, typed failure routing, HB closure, global PkgC, PCCMin, SAT in P, and $P = NP$ remain open.

The checked typed-realizer contract now validates the next public interface over arbitrary finite selector lists. A data row is either one unit-charge blueprint or one of three bot forms: HN, budget, or lower seed. For every row whose supplied faithfulness predicate is true, acceptance proves exactly a charge-derived strict equivalent gain, an active HN entry at no greater supplied rank, an active

budget entry at no greater supplied rank, or a faithful seed at strictly lower rank. The grouped-BN6 specialization covers every canonical input-relative handle. The family, finite rank assignment, faithfulness predicate, claims, and blocker-activity tables remain explicit inputs. The indices are not the manuscript rank tuple, and the checker does not establish blocker semantics, HB acyclicity, global selector silence, or polynomial generation/runtime. Thus this is not unconditional ZeroSlack, PCCMin, SAT in P, or $P = NP$.

The checked HB blocker-graph acyclicity contract now validates the next local Section 15 boundary. A closed data node is either HN or budget at one supplied finite rank, and an edge contains only a blocked node and its dependency. The checker exhaustively proves that the supplied finite indices embed strictly into the existing exact ten-coordinate residual rank and that every supplied edge descends that rank. Acceptance yields accessibility, a well-founded dependency relation, and absence of every nonempty directed cycle. It also upgrades valid lower-seed bots to exact-rank descent and composes with every faithful canonical handle in the checked typed-realizer table. The graph, edges, rank mapping, dependency completeness, and blocker semantics remain explicit inputs or open obligations. This is not the full HB negative closure, rank-complete selector silence, unconditional ZeroSlack, polynomial PCCMin, SAT in P, or $P = NP$.

The checked total-table HB dependency contract now removes the independent edge-list input from that finite boundary. Every HN and budget node at every supplied finite rank has one data-only dependency row, and the graph is materialized mechanically from all rows. Exact theorems identify graph edges with row membership in both directions. The checker exhaustively validates the finite-to-exact rank embedding and strict exact-rank descent for every listed dependency. Acceptance yields total row representation, accessibility, well-founded rank induction for arbitrary predicates with an explicit local step, and absence of every nonempty dependency cycle. HN and budget typed bots name covered rows, while valid lower seeds inherit exact-rank descent. The table, rank mapping, and local induction premise remain explicit inputs. This does not prove blocker semantics, semantic dependency completeness relative to terminal data, rank-complete selector silence, the full HB negative closure, unconditional ZeroSlack, polynomial PCCMin, SAT in P, or $P = NP$.

The checked HB active-dependency closure now supplies the local no-outcome step for those explicit tables. It projects HN/BUD activity directly from the typed-realizer environment and exhaustively requires every active node to name an active dependency in its own total row. The combined checker also requires the prior exact-rank table check. Well-founded induction therefore proves every supplied HN and budget activity bit false. Typed-realizer composition removes the HN/BUD bot branches and leaves only a verified strict gain or a faithful strictly lower-rank seed. The activity bits, dependency rows, rank mapping, selector data, and claims remain explicit inputs; blocker semantics and semantic dependency completeness are not derived from terminal data. Gain exclusion, lower-seed closure, rank-complete selector silence, the full HB negative closure, unconditional ZeroSlack, polynomial PCCMin, SAT in P, and $P = NP$ remain open.

The conditional selector-silence rank closure now combines checked HN/BUD inactivity with explicit global semantic gain exclusion. Any accepted faithful canonical selector would then require a faithful selector at strictly lower finite rank, so strong induction proves every handle in the supplied table nonfaithful. The coverage specialization derives global gain exclusion only from an explicit gain-coverage certificate plus exact source-cell no-gain. The family, rank and faithfulness tables, realizer claims, blocker sidecars, and global premise or certificate remain supplied. This does not establish selector faithfulness or compatibility, derive those inputs from terminal data, prove unconditional HB negative closure or ZeroSlack, provide polynomial bounds, put SAT in P, or prove $P = NP$.

The executable selector-silence induction removes that global semantic no-gain premise from the theorem interface. Its data-only checker scans every canonical handle and requires every realizer claim to be a typed bottom. Checked HB closure eliminates HN/BUD bottoms, while strong finite-rank induction eliminates faithful lower seeds. The grouped family, ranks, faithfulness, claims, activity, dependency rows, and rank map remain explicit data inputs. The theorem does not construct them from terminal data, establish selector faithfulness or compatibility, prove blocker semantics or

semantic dependency completeness, close the full unconditional HB negative closure or ZeroSlack, provide polynomial bounds, put SAT in P, or prove $P = NP$.

The Packet selector-faithfulness routing bridge checks ten data-only fields on the canonical positive source payload behind every handle, reports the first failed route, exhaustively scans the canonical handle list, and binds the computed result exactly to the supplied HB faithfulness table. Every positive BN6 Packet supplies a canonical handle. Route-clear acceptance therefore makes one handle faithful, while accepted executable selector silence makes every handle nonfaithful, yielding a kernel-checked contradiction. The grouped family, payload checks, finite rank tags, HB table, claims, blocker activity, and dependency rows remain explicit inputs. Positive slack, `SaturatePositive`, `BCELReady`, terminal-data construction, complete route silence, unconditional ZeroSlack, and polynomial PCCMin remain open.

The canonical Packet faithfulness-table constructor removes the independently supplied faithfulness function from that composition. For every arbitrary finite grouped BN6 family, it rebuilds the typed-realizer table with faithfulness equal to the canonical positive source-payload computation while preserving the rank map, realizer claims, and HN/BUD activity tables exactly. The exhaustive binding checker therefore accepts by construction, and the positive-Packet versus executable-selector-silence contradiction no longer accepts a binding premise. The payload checks, rank assignment, grouped family, claims, blocker semantics, activity, dependency rows, and their derivation from terminal data remain explicit or open. This is not full external selector compatibility, complete route silence, unconditional HB negative closure, ZeroSlack, or polynomial PCCMin.

The total Packet selector first-route outcome proves the converse side of this classifier. For every payload at an arbitrary finite rank, no route is returned exactly when the complete checker accepts, and rejection returns one concrete earliest typed route. Every positive Packet therefore supplies either a computed faithful handle or a first route. Accepted executable selector silence and HB active-dependency closure for the canonicalized table remove the faithful case without route-clear or independent binding premises. This names every checked payload failure, but it does not prove the external terminal semantics of any route, map routes into a decreasing complete global outcome system, or construct the payload fields and remaining tables from terminal data. Unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Exact Packet first-route semantics now identifies the meaning of that route at the supplied-payload boundary. Uniformly across all ten constructors, the executable route equality is equivalent to the exact proposition that every earlier field accepted and the named field failed. The failure is unique, and the positive-Packet/HB endpoint retains the exact field proof alongside its route without route-clear or binding premises. These are semantics only of the existing Boolean payload; the result does not derive fields from terminal data, prove their external manuscript semantics, map failures into a decreasing complete global outcome, or establish unconditional HB negative closure, ZeroSlack, or polynomial PCCMin.

Rank-reflected Packet descent now removes the free Boolean at the last route. The canonical payload preserves the first nine fields and computes `strictDescentClear` from the exact ten-coordinate `RankWF` comparison. Acceptance carries actual descent; a final descent failure carries actual nondecrease. The positive-Packet/HB endpoint therefore returns that proof or an earlier exact route without route-clear or descent-binding premises. The before/after ranks, first nine fields, grouped family, and HB data remain explicit, and the other nine routes are not yet mapped into a complete global outcome. Complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Canonical Packet rank-tag reflection now removes the payload's separate copy of the finite handle rank. The table-owned rank is copied into `rankTag`, while the exact residual-descent computation is retained. The canonical first-route classifier therefore cannot return `.rank`; a final `.descent` route still carries actual nondecrease. The finite rank map, before/after residual ranks, seven earlier Boolean fields, `exactRouteClear`, grouped family, and HB data remain explicit. The eight remaining

routes are not mapped into a complete global outcome. Complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Canonical Packet exact-route reflection now removes the payload's separate internal source-route bit. Every canonical handle already selects an original positive payload atom from its exact grouped cell and footprint, so the active payload sets `exactRouteClear` by construction while retaining the table-owned rank and computed residual descent. The canonical classifier can return neither `.exactRoute` nor `.rank`; a final `.descent` route still carries actual nondecrease. This internal route is not an external exact minimum. The seven semantic Boolean fields, grouped family, rank map, residual ranks, and HB data remain explicit. The seven remaining routes still lack complete external semantics and global routing, so complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Canonical Packet charge-route reflection now removes the payload's separate internal charge bit. Each canonical handle selects an original source atom whose strictly positive mass is already proved by the grouped-family structure, so the active payload sets `chargeChecked` by construction while retaining exact-route, rank-tag, and residual-descent reflection. The canonical classifier can return none of `.charge`, `.exactRoute`, or `.rank`; a final `.descent` route still carries actual nondecrease. Positive source mass is not the complete external charge-surplus, replacement, budget, or selector-compatibility semantics. The six remaining semantic Boolean fields, grouped family, rank map, residual ranks, and HB data remain explicit. Those six routes still lack complete external semantics and global routing, so complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Canonical Packet colour-route reflection now removes the payload's separate internal colour bit. Every canonical handle already has a grouped footprint proved to lie in the family carrier and to contain at least two atoms, so the active payload computes `colourChecked` from selector-relevant footprint size while retaining the carrier-sublist theorem, positive charge, internal source route, table-owned rank, and exact residual descent. The canonical classifier can return none of `.colour`, `.charge`, `.exactRoute`, or `.rank`; a final `.descent` route still carries actual nondecrease. This internal eligibility check is not full external manuscript colour equivalence. The five remaining semantic Boolean fields, grouped family, rank map, residual ranks, and HB data remain explicit. Those five routes still lack complete external semantics and global routing, so complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Canonical Packet typed-frontier route reflection now removes the payload's separate frontier bit. Each selected source payload carries explicit source and selector frontier signatures of one arbitrary decidable-equality type, so the active payload computes `frontierChecked` from signature equality while retaining canonical colour, positive charge, internal source route, table-owned rank, and exact residual descent. The `.frontier` route is equivalent to signature inequality; equal signatures exclude it. The classifier also continues to exclude `.colour`, `.charge`, `.exactRoute`, and `.rank`; a final `.descent` route carries actual nondecrease. The supplied signatures are not constructed from terminal data or bound to the manuscript BN5 frontier. Obligation, activation, direction, and budget remain explicit. Those four remaining routes still lack complete external semantics and global routing, so complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

BN5-bound Packet frontier and obligation route reflection now ties both checks to the typed terminal BN5 coordinate. The active payload compares the exact source and selector `frontier` and `obligation` projections while retaining canonical colour, positive charge, the internal source route, table-owned rank, and exact residual descent. A `.frontier` first route is exact frontier inequality. A `.obligation` first route carries prior frontier equality and exact obligation inequality. The classifier also continues to exclude `.colour`, `.charge`, `.exactRoute`, and `.rank`, while `.descent` carries actual nondecrease. The BN5 coordinates remain explicit rather than derived from terminal data. Activation, direction, and budget are the three remaining fields and routes; they still lack complete external semantics and global routing. Complete route silence, unconditional HB negative

closure, ZeroSlack, and polynomial PCCMin remain open.

BN4 activation-exact Packet route reflection now ties the next check to the activation atoms nested inside the typed source and selector BN5 coordinates. The active payload compares those atoms while retaining computed BN5 frontier and obligation checks, canonical colour, positive charge, the internal source route, table-owned rank, and exact residual descent. The existing BN4 theorem proves that activation-atom equality is equivalent to equality of the canonical activation predicates on every cut. An `.activation` first route carries prior frontier and obligation equality plus exact activation-atom inequality. The classifier continues to exclude `.colour`, `.charge`, `.exactRoute`, and `.rank`, while `.descent` carries actual nondecrease. The coordinates remain explicit rather than derived from terminal data. Direction and budget are the two remaining fields and routes; they still lack complete external semantics and global routing. Complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Typed Packet direction-route reflection now ties the next check to equality of explicit source and selector values in an arbitrary direction type. The active payload retains computed BN5 frontier, obligation, and BN4 activation checks, canonical colour, positive charge, the internal source route, table-owned rank, and exact residual descent. A `.direction` first route carries prior frontier, obligation, and activation equality plus exact typed-direction inequality. The classifier continues to exclude `.colour`, `.charge`, `.exactRoute`, and `.rank`, while `.descent` carries actual nondecrease. The direction values remain explicit rather than derived from terminal data and are not a complete construction of manuscript `Dir(u)`. Budget is the sole remaining supplied Boolean field and route; it still lacks complete external semantics and global routing. Complete route silence, unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Typed Packet budget-route reflection now ties the final supplied Packet check to equality of explicit source and selector values in an arbitrary budget type. The active payload retains computed BN5 frontier, obligation, BN4 activation, and typed direction checks, canonical colour, positive charge, the internal source route, table-owned rank, and exact residual descent. A `.budget` first route carries prior frontier, obligation, activation, and direction equality plus exact typed-budget inequality. The classifier continues to exclude `.colour`, `.charge`, `.exactRoute`, and `.rank`, while `.descent` carries actual nondecrease. The budget values remain explicit rather than derived from terminal data and are not a complete construction of manuscript `Bud(u)`, its budget envelope, BudgetResolve, or HB budget activity. Every local Packet field is now computed, but external route adequacy and global route silence remain open. Unconditional ZeroSlack and polynomial PCCMin remain open.

Checked Packet budget/HB activity binding now exhaustively requires every typed budget mismatch at a canonical handle to activate the HB budget node at the table-owned rank. The supplied checked well-founded HB no-outcome closure forces that activity false, so an accepted binding yields exact typed budget equality at every handle and excludes the `.budget` first route. The remaining outcome is frontier, obligation, activation, direction, or exact residual nondecrease. The binding, rank map, activity environment, dependency table, and closure premise remain explicit rather than constructed from terminal data; this is not BudgetResolve, blocker semantic completeness, full Packet adequacy, or remaining-route exclusion. Unconditional HB negative closure, ZeroSlack, and polynomial PCCMin remain open.

Checked Packet semantic/HN activity binding now exhaustively requires any failure of simultaneous frontier, obligation, activation, and direction agreement at a canonical handle to activate the HN node at the table-owned rank. The supplied checked well-founded HB no-outcome closure forces that activity false, so all four exact typed fields agree and their first routes are excluded. Together with the separately checked budget/HB binding, positive Packet existence, and executable selector silence, the sole route is `.descent` and carries exact residual nondecrease. The semantic/HN binding, typed fields, rank map, activity environment, dependency table, and closure premise remain explicit rather than constructed from terminal data. This does not prove HN blocker semantics, semantic completeness, a decreasing transition, a no-lower contradiction, unconditional ZeroSlack, or polynomial PCCMin.

Checked Packet descent no-lower binding now scans every canonical handle and accepts exactly when no fully computed Packet first route is `.descent`. Positive Packet existence together with the checked semantic/HN and budget/HB bindings, executable selector silence, and checked well-founded HB no-outcome closure forces one such residual-nondecrease route. The exhaustive local no-lower checker therefore returns false, and accepting that same row is contradictory. This closes one local Packet ledger row over supplied data; it does not construct terminal inputs or the complete no-lower ledger, cover HResolve, BudgetResolve, normalization, named routes, saturation, or replay, establish unconditional ZeroSlack, or prove polynomial PCCMin.

Checked Packet no-lower ledger composition now recomputes the five exact semantic/HN binding, budget/HB binding, selector-silence, HB no-outcome closure, and Packet descent/no-lower rows in one Boolean boundary. Its reflection theorem characterizes acceptance without a caller-supplied success flag, and an accepted ledger excludes a positive Packet conclusion over every supplied arbitrary finite grouped family. This closes only the Packet branch, not the complete no-lower ledger or ZeroSlack. Terminal inputs and bindings remain supplied, while HResolve, BudgetResolve, normalization, other named obstructions, saturation, replay, and polynomial PCCMin remain open.

Checked finite HResolve coverage now computes one exact, gain, blocked, or unresolved route in fixed priority order for every member of an arbitrary supplied finite candidate family. The generated ledger has one computed row per supplied occurrence, while the NoHereditary sidecar checker independently recomputes duplicate-free enumeration and requires every candidate to be blocked after both constructive routes fail. Its reflection theorem characterizes exact acceptance, and accepted sidecar data excludes exact and gain routes for every listed candidate. The family and decidable predicates remain supplied; this does not construct the governed hereditary universe from terminal data, prove HN grammar, BWL exactness, H-disjointness, exact-minimum, strict-gain, or blocker dependency semantics, discharge full HResolve or BudgetResolve, complete the no-lower ledger, establish unconditional ZeroSlack, or prove polynomial PCCMin.

The terminal-derived HResolve support resolver now removes the supplied family and supplied exact/gain predicate boundary for canonical terminal supports. For every finite direct-wire candidate it constructs the complete support-seed family from the terminal primitive-record universe, proves duplicate-free coverage, saturates through the candidate-derived dependency system, and extracts the corresponding open support. Actual reference residual slack then computes the only two routes: zero gives semantic minimality, while positive slack gives a strict semantically equivalent gain witness. Thus every governed seed has a proof-bearing exact or gain route. Subset enumeration and reference minimization are exhaustive and may be exponential. This does not formalize HN grammar, BWL, ParseOrExit, H-disjointness, blocker dependencies or NoHereditary, polynomial HResolve, BudgetResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The terminal budget-envelope resolver now places a computed finite resource boundary over that canonical support universe. Given supplied natural gate and saturated-record caps, it saturates each seed in the candidate-derived system, requires a nonempty gate and interface support, and recomputes both bounds. A feasible support carries either semantic-minimum or strict-equivalent-gain evidence; exhaustive search failure excludes every canonical seed and supplies the strong `NoBudget` branch. The caps remain supplied, and subset search, saturation, and reference minimization may be exponential. This is not the manuscript HN/BUD grammar, BWL or budget-envelope dynamic program, blocker dependency semantics, polynomial BudgetResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The terminal budget no-lower ledger now evaluates the same envelope for every canonical support. Each row is computed as exact, strict gain, or `NoBudget`; the exhaustive Boolean accepts exactly when every budget-feasible governed support is a semantic minimum. Acceptance therefore excludes every feasible strict-equivalent-gain witness. The caps remain supplied, and subset enumeration, saturation, and reference minimization may be exponential. This closes only a finite terminal-derived budget branch, not the manuscript HN/BUD grammar, polynomial BudgetResolve, Packet or complete

no-lower composition, unconditional ZeroSlack, or polynomial PCCMin.

The terminal Packet-budget no-lower composition now places that canonical budget-support ledger and the checked five-row Packet ledger behind one Boolean over the same direct-wire candidate. Exact reflection exposes the conjunction of both component propositions. Acceptance makes every budget-feasible governed support a semantic minimum, excludes every such strict-equivalent-gain witness, and excludes a positive Packet over the supplied grouped family and tables. The caps, Packet family, typed payloads, ranks, realizer claims, activity environment, and dependency rows remain supplied. This is a finite two-branch composition, not the complete no-lower ledger, a derivation of Packet data from terminal data, unconditional ZeroSlack, or polynomial PCCMin.

The terminal HResolve maximal H-disjoint-family milestone separately formalizes the finite family-assembly edge over arbitrary supplied hereditary footprints. It checks support, frontier, origin, kernel, obligation, prefix-tail, charge, and interface noninterference, then constructs a governed duplicate-free maximal pairwise H-disjoint family. Every rejected governed candidate carries a selected blocker and its exact first interference route. The footprints remain supplied. This does not prove HN grammar, BWL, ParseOrExit, leaf tightness, the full NoHereditary sidecar, blocker-to-HB rank semantics, full or polynomial HResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The terminal HN BWL certified-path-minimum milestone formalizes exact four-coordinate lexicographic selection over a nonempty finite supplied path family. Each path carries a hereditary shape, a nonempty block decomposition, semantic-equivalence evidence, and frontier fidelity, while cost is derived from its realized gate count. The selected path is listed and lower-bounds every governed alternative under an explicit family-completeness premise. The family remains supplied. This is not accepted-grammar generation or completeness, LN confluence, ParseOrExit, leaf tightness, the full BWL theorem, polynomial HResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The terminal HResolve certified-path-family milestone composes the maximal eight-domain H-disjoint selector with the finite certified-path minimum. Each supplied candidate owns its expected frontier, footprint, nonempty finite path family, governed predicate, completeness proof, and path-to-footprint coherence. Every selected candidate exposes an exact four-coordinate minimum with semantic, frontier, block, shape, and footprint evidence. Every rejected candidate exposes a selected blocker and exact first interference route. The proof-bearing candidates remain supplied. This is not terminal candidate or path derivation, accepted-grammar completeness, LN confluence, ParseOrExit, leaf tightness, the H0-H4 sidecar, full or polynomial HResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The proof-bearing HResolve ZeroSlack sidecar milestone upgrades the report-facing certificate boundary. In place of three HResolve string handles, Lean consumes an arbitrary finite supplied governed family, the exact equation returned by the executable NoHereditary checker, and semantic proofs for its exact-minimum and strict-equivalent-gain predicates. Acceptance yields duplicate-free total blocked coverage and excludes both constructive routes for every listed candidate. The family, implementation map, predicates, decidability witnesses, and blocker semantics remain supplied. This is not terminal candidate derivation, H0–H4 blocker semantics, full or polynomial HResolve, the complete no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The proof-bearing Budget ZeroSlack sidecar milestone upgrades the matching report-facing certificate boundary without rerunning or restating the budget resolver. In place of three Budget string handles, Lean stores the exact equation showing that exhaustive terminal-envelope search returned no feasible canonical support. Reflection excludes the recomputed gate/record predicate for every governed seed, while existing exact-slack and positive-slack theorems give semantic minimum and strict-equivalent-gain meanings to the two constructive route forms. The caps remain supplied and support enumeration, saturation, and reference minimization may be exponential. This is not the BUD grammar or B0–B4 sidecar, full or polynomial BudgetResolve, the complete no-lower ledger,

unconditional ZeroSlack, or polynomial PCCMin.

The proof-bearing Selector/HB ZeroSlack sidecar milestone upgrades the joint report-facing boundary without rerunning or restating the underlying executable checks. In place of eight selector-silence and HB-closure strings, Lean stores the exact all-row selector-silence and ranked no-outcome closure equations. Reflection and strong finite-rank induction prove every canonical selector nonfaithful, every claim an exact typed bottom, every supplied HN/BUD activity bit false, and the dependency relation well founded. The grouped family, realizer and dependency tables, environment, claims, activity bits, and rank map remain supplied. This is not terminal-data construction, selector faithfulness or compatibility, blocker semantics, semantic dependency completeness, the BCEL contradiction, unconditional ZeroSlack, or polynomial PCCMin.

The proof-bearing Packet/budget no-lower ZeroSlack sidecar milestone upgrades the matching report-facing field without rerunning or restating either underlying finite checker. In place of a no-lower string, Lean stores the exact same-candidate Packet/budget composition equation. Reflection proves every governed budget-feasible support semantically minimum, excludes every such strict equivalent gain, and excludes a positive Packet conclusion for the supplied family. The caps, candidate-derived model, grouped family, payloads, ranks, claims, activity, dependencies, and rank maps remain supplied. This is the existing finite two-branch boundary, not the complete manuscript no-lower ledger, unconditional ZeroSlack, or polynomial PCCMin.

The proof-bearing BCEL/Packet no-lower ZeroSlack sidecar milestone upgrades the remaining BCEL contradiction bundle from five strings to one dependent boundary tied to that exact M180 certificate. Its stored decidable equation checks that the same supplied grouped family has at least two anchors. BCEL constant activation would let the existing arbitrary-finite BN6 theorem construct a positive Packet, contradicting the accepted same-family no-lower exclusion. The family and all M180 inputs remain supplied. This does not derive BCELReady or constant activation from positive residual slack, complete the no-lower ledger, establish unconditional ZeroSlack, or prove polynomial PCCMin.

The same-family Selector/HB, Packet, and BCEL ZeroSlack coherence milestone removes a detached certificate seam in the report-facing boundary. The Selector/HB sidecar is derived from the exact grouped family, computed realizer table, and dependency table already accepted by M180, then its silence and closure consequences are combined with M180 Packet exclusion and the dependent M181 BCEL contradiction. The family and all terminal, budget, Packet, realizer, dependency, rank, and BCEL inputs remain supplied. This does not derive those inputs or constant activation from positive residual slack, complete the no-lower ledger, establish unconditional ZeroSlack, or prove polynomial PCCMin.

The checked finite SaturatePositive-to-BCEL-ready composition reruns the production finite classifier and accepts only its positive-projection branch with a computed ready anchor nucleus. Its proof-bearing result retains the exact classifier equality, safe trace, positive final and whole-support quantities, nontrivial nucleus, constant-cut equations, and local BN2 conclusions. The terminal problem and initial positive premise remain supplied; local failures are not globally routed, and BN3–BN6 inputs, constant activation, manuscript-wide SaturatePositive or BCELReady, ZeroSlack, and polynomial PCCMin remain open.

The same-candidate finite BCEL-ready and Packet carrier-coherence certificate indexes its terminal problem by the exact candidate and model of one accepted Packet/budget no-lower record. An injective and surjective anchor map plus a reflected exact list equality identifies the computed ready nucleus image with that record's grouped Packet carrier. This transfers the nontrivial-size bound without an independent Boolean and combines it with the same-family positive- Packet exclusion to rule out constant activation. The terminal problem, positive premise, family, map, payloads, tables, and ranks remain supplied; activation weights are not identified with projection excess, constant activation is not derived from positive residual slack, and unconditional ZeroSlack and polynomial PCCMin remain open.

The finite BCEL/Packet activation-coherence obstruction then checks the exact remaining numerical

seam. It compares the computed terminal defect with the declared Packet cut value and with every canonical nonempty proper-cut activation weight on that same family. Hypothetical acceptance reconstructs the mapped-cut projection equation, but the already checked same-family Packet exclusion proves rejection. A deterministic classifier returns either a proof-bearing cut-value mismatch or the first proper-cut activation mismatch. This diagnoses rather than repairs the missing bridge: the terminal problem, positive premise, family, map, activation data, payloads, tables, and ranks remain supplied; enumeration may be exponential, and unconditional ZeroSlack and polynomial PCCMin remain open.

| Machine field | Current value |
|---|---|
| `mathematicalTheoremEstablished` | **false** |
| `publicTheoremEmissionAllowed` | **false** |
| `finalTheoremReady` | **false** |
| `publicTheoremStatement` | `null` |
| `rootLeanTheorem` | `PNP.Main.p_eq_np` |
| `rootLeanTheoremPresent` | **false** |

## 3 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof. The compiler/refinement link to the raw machine kernel is formalized, but no deterministic polynomial CNF-SAT decider or concrete publication root is present. The direct CNF verifier proves NP membership only.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

| Gate subcheck | Result |
|---|---|
| standardComplexityModelEligible | **true** |
| concreteTargetPresent | **true** |
| concreteTargetIsDefinition | **true** |
| concreteTargetKernelTypeFingerprintConfigured | **false** |
| concreteTargetKernelTypeFingerprintMatches | **false** |
| concreteTargetKernelValueFingerprintConfigured | **false** |
| concreteTargetKernelValueFingerprintMatches | **false** |
| compatibilityRootPresent | **false** |
| compatibilityRootIsTheorem | **false** |
| compatibilityRootHasExactConcreteType | **false** |
| compatibilityRootKernelTypeFingerprintConfigured | **false** |
| compatibilityRootKernelTypeFingerprintMatches | **false** |
| axiomClosureFingerprintConfigured | **false** |
| axiomClosureFingerprintMatches | **false** |
| sourceClosureFingerprintConfigured | **false** |
| sourceClosureFingerprintMatches | **false** |
| axiomClosureUsesOnlyLeanStandardAllowlist | **false** |

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The compiled current project-axiom inventory contains 0 declarations:

- None. The compiled project-axiom inventory is empty.

### 3.1 Current concrete target and historical abstract bridge

The former abstract bridge compared witness classes whose language and algorithm structures carried string names or code handles rather than concrete execution semantics and proved polynomial bounds. A theorem of that former abstract type would not establish the standard P-versus-NP statement.

The current `PNP.PEqualsNP` compatibility name refers to `PNP.Concrete.PEqualsNP`; the eligible publication target is `PNP.Main.ConcretePEqualsNP`. These concrete finite charged-pipeline semantics carry proof-bearing P and NP witnesses, canonical input/certificate pairing, and polynomial runtime and output bounds. The recursive compiler/refinement link to the raw machine kernel is formalized. Concrete CNF-SAT NP-completeness is kernel checked. Deterministic CNF-SAT membership in P and the exact eligible root theorem remain absent; the publication fingerprints are unset and the publication gate remains false.

## 4 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, an axiom closure contained in the reviewed Lean standard-axiom allowlist, and per-name domain-separated kernel-type SHA-256 matches for 4534 detailed candidates, and the pinned whole-Lean source closure. No project-specific axiom is allowed; an earned row need not have an empty axiom closure. Human-readable scope remains conservative: type or source drift revokes credit.

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Wire-backed computational profiles and exposure balance | formalized-foundation-only | For arbitrary finite input, ordinary-output and computational-field widths, full comparison preserves the ordinary Boolean outputs and every actual wire-backed field at every input valuation. Quotient comparison masks only selected profile constraints and still preserves every ordinary output. Exact characterization, full-lift, attained-witness and universal lower-bound theorems establish quotient minimum <= full minimum <= physical gate count. Full slack plus projection defect equals physical size minus quotient minimum. Exposure masks that both retain a fixed comparison mask preserve that quotient minimum and the combined measure; nested masks transfer exactly the lost full slack into projection defect. Loss of positive full slack to zero yields an attained quotient witness with a strict deficit that cannot be used as a full witness. Forgetting all profile constraints recovers the ordinary semantic minimum. The actual all-gate source tuple, checked normalization and checked splicing connect this model to computational wire carriers without a supplied observer, semantic minimum or correctness certificate for the comparison model. | This is a proposed reconstruction of computational profile comparison, not the complete manuscript ten-role grammar or a derived governed terminal family. Ordinary outgoing Boolean interface wires cannot be forgotten. The all-gate field tuple is source-derived but is not asserted to be a canonical terminal profile. Conservation of full slack plus projection defect proves neither forced-cost transparency nor a Package E route, positive-slack activation, global rank decrease or route coverage. A quotient witness cannot be used as a full replacement without restoring every omitted field. Checked splicing still requires an equivalent open replacement and successful compilation; global discovery is not proved. Reference minima and attained witnesses use exhaustive finite minimization, not a polynomial algorithm. Unconditional SaturatePositive, BCELReady and ZeroSlack, exact polynomial PCCMin, encoded-input runtime, output and certificate bounds, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global proof gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-derived zero-cost alias exposure | formalized-foundation-only | For arbitrary finite input, ordinary-output and extra-field widths, a literal output extension adds only input, constant or existing-output aliases and a literal projection recovers the original outputs without allocating gates. Both constructions transfer arbitrary equivalent realizations, proving exact equality of the semantic reference minimum and residual slack. A source-derived recognizer scans the actual output wires; a gate field is accepted exactly when an ordinary output names that same wire. The tuple compiler derives every alias and accepts exactly when all requested fields have such literal references. The actual WireCarrier exposure therefore preserves physical gate count, semantic minimum and slack whenever this executable source check accepts. Constructors and recognition do not enumerate truth tables or minimum implementations and do not receive an observer, semantic minimum, route or correctness certificate from the caller. | This covers the literal input/constant/old-output alias class, not every semantically free exposure or arbitrary hidden internal wire. Refusal proves neither semantic impossibility nor a Package E gain or route. Adding no physical gates need not preserve the semantic minimum, and unique physical ownership or an extra recorded charge does not force an equal minimum increase. The concrete counterexamples are regression and obstruction evidence, not new global progress. The existing fresh-independent-input materializer theorem is reused, not re-awarded. Full manuscript profiles, positive-cost transparency for arbitrary materializers, terminal-family derivation, global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete encoded-input polynomial runtime, output and certificate bounds remain open. No fixed weighted checkpoint or global proof gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Structural reordering in complete source-only replacement programs | formalized-foundation-only | For arbitrary finite computational wire carriers, raw swap sequences, dependency graphs and complete offered replacement programs, computed structural actions preserve the literal exposed field sources, ordered output values and exact syntactic causal labels. The raw decoder validates the entire sequence against the current carrier, not the initial gate count. The exact pending snapshot function, charge count and removal count are preserved. Local physical ownership comes from the actual backward gate bijection, and the complete program lifts it through the prior ledger without resetting earlier allocations or removals. Structural actions interleave with primitive obligations and descendant-support programs while preserving full output and field semantics, creation-to-discharge bindings, unique event identities, causal invariants and physical ownership. The existing proper-support certificate verifier over this expanded action language still requires a complete accepted program, a closed final ledger and actual strict signed saving. Invalid later operations reject the whole program rather than returning a successful prefix; reordering alone earns no strict saving. Callers supply raw data, not correctness, transport, order, causality or ownership witnesses. | This is integration of checked offered programs, not a successful certificate-discovery strategy for every input. Finite executions are regression evidence, not theorem authority. The computational carrier does not establish full manuscript profile semantics or all R1-R9 and N1-N10 rules. Full manuscript VerifyDW, ChargeSoundness and Package E, global certificate discovery, terminal-family derivation and global route coverage remain open. There is no encoded-input polynomial runtime, output-size or certificate-size theorem for the complete construction. Unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computational recoding in complete source-only replacement programs | formalized-foundation-only | For arbitrary finite computational wire carriers, literal NAND encoder and decoder circuits, dependency graphs and complete offered replacement programs, raw recoding actions reuse the existing circuit format and validate both dimensions and every gate and output reference. A complete finite check derives both inverse equations. An independent exact syntactic dependency guard covers all ordinary outputs and hidden fields for every natural-valued input labelling. Actual encoder and decoder gates are appended, normalized and charged; actual compiler maps determine deletions and disjoint allocation identities. Recoding retains the entire pending snapshot function and cannot create or discharge an obligation. Complete-program execution lifts those physical identities through prior history, preserves full semantics, costs, causal invariants and creation-to-discharge bindings, and rejects an invalid later action rather than accepting a prefix. The proper-support certificate verifier still requires complete execution, a closed final ledger and strict actual saving. The existing structural decoder also has a constructive exact encoding of every finite gate bijection with at most one swap per gate. Callers supply raw data, not correctness, causal, ordering, ownership or cost authority. | This checks offered computational programs, not a successful certificate-discovery strategy for every input. The inverse checker enumerates all field valuations; finite termination and a bounded structural swap list do not establish uniformly polynomial encoded runtime or certificate size. Semantic reversibility does not imply physical cancellation or strict saving. Full manuscript profiles, all R1-R9 and N1-N10 rules, full VerifyDW, ChargeSoundness and Package E remain open. Terminal-family derivation, global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global proof gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Structural reordering and arbitrary-support replacement transport | formalized-foundation-only | For arbitrary finite computational wire candidates and checked finite raw index-swap sequences, structural reindexing constructs the raw graph and its successful compilation from the original circuit. Actual compiler placement and inverse placement determine literal gate-source pairs and ordered output references. For every descendant primitive-record list, including empty, full and duplicate-bearing lists, the construction derives the predecessor records and canonical boundary, interface, selected-gate and exterior bijections. Open support functions agree for every independent boundary valuation. A replacement is pulled back by literal input and output rewiring without padding; support and replacement counts agree, both matched signed surcharges are zero, and exact signed saving and strict gain are preserved. Complete raw splices have corresponding source pairs, outputs and dependency edges, with equivalent acyclicity and compiler success or rejection in both directions. Actual compiled-position maps preserve exterior and replacement ownership and literal sources. An actually accepted compatible descendant splice computes an accepted predecessor splice preserving whole-circuit semantics and exact local and whole signed savings. No predecessor compilation, order, transport map or source-fidelity witness is supplied. | This closes the computational structural-reordering component of arbitrary-support replacement transport, not full manuscript profile semantics: transported profile labels are not a proof of profile semantics. It does not establish completeness of the raw-swap encoding for every abstract permutation, all R1-R9 or N1-N10 rules, or the remaining normalization and materializer transport. Open replacement compatibility is required for semantic preservation, and actual descendant compiler acceptance is required for compiled pullback; a compatible arbitrary replacement need not be acyclic. Full manuscript VerifyDW, ChargeSoundness and Package E, global certificate discovery, terminal-family derivation and global route coverage remain open. There is no encoded-input polynomial runtime, output-size or certificate-size theorem for the complete construction. Unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin, deterministic CNFSAT in P and the eligible root remain open. Finite execution fixtures are regression evidence, not theorem authority. No fixed weighted checkpoint or global gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Open obligations across complete descendant-support programs | formalized-foundation-only | For arbitrary finite computational wire carriers, raw outer support records and raw mixed programs, source-only verification computes the complete dependency order, including intrinsic creation references, and executes every primitive or actual descendant-support splice from its internally constructed initial state. The same pending creations and captured full-value snapshots persist across intervening support changes until genuine full-mode R6, R7 or R8 discharge; final ambient closure is required. Execution-derived causal bounds construct the literal outer splice. Acceptance is exactly complete decoding and finally closed execution on a proper physical support with strict final saving. The result preserves all ordinary outputs and literal fields, includes the exterior once, and reconstructs unique original/allocation ownership through the actual compiler positions. Computed outer-position namespaces distinguish nested event identities; historical charges and removals survive later allocation deletion. Temporary expansion is allowed; malformed, cyclic, missing or duplicate references, unfinished ledgers, failed tails, whole supports and final nondecrease reject. No intermediate carrier, schedule, snapshot, correctness, cost, owner, rank or splice witness is supplied. | This verifies offered arbitrary finite mixed programs in the computational wire-carrier language; inner descendant histories retain their existing closed-history boundary. It does not find an accepted certificate for every nonminimal input, establish local minimality after rejection, or construct a globally successful strategy. Full manuscript profiles and all R1-R9 and N1-N10 rule families, matched-kappa arbitrary-support Pull/Expand, full manuscript VerifyDW, ChargeSoundness and Package E, and terminal-family derivation remain open. There is no bound on temporary growth or encoded-input polynomial runtime, output size or certificate size. Global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin, deterministic CNFSAT in P and the eligible root remain open. Finite runtime fixtures are regression evidence, not theorem authority. No fixed weighted checkpoint or global gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete machine and cost kernel | formalized-foundation-only | One literal finite four-stage pipeline handles every raw bitstring. The total framer uses exactly 4 work steps on empty input, 4 * k * k + 9 * k + 7 for complete two-bit cells, and 4 * k * k + 9 * k + 5 when the final cell is partial; its compiled raw cost is bounded by 6 * m * m + 39 * m + 75. Every supplied exact n-step target run lifts to exactly 3 * n work steps. PipelineCompiler preserves one raw target's verdict and ordinary output at an explicit external polynomial. PipelineSequentialCompiler composes two raw targets in one literal finite table, passes either first verdict onward, preserves the second verdict and output, retains stuck-first timeout, and proves R(m) = PipelineRaw(p)(m) + 6 + PipelineRaw(q)(m + p(m) + 1). PipelineRefinement recursively applies that compiler to every FunctionProgram composition and DecisionProgram precomposition node. Its 16 audited declarations have empty axiom closure, and PolynomialTimeDecider.compileToMachine exposes the complete tree as one raw polynomial-time machine. | This closes the concrete complexity machine-link blocker only. It does not construct a deterministic polynomial-time CNF-SAT decider or an NP-completeness reduction. CNF-SAT in P, NP-completeness, and P = NP are not established; PNP.Main.p_eq_np remains absent and the publication gate remains false. |
| Concrete P, NP, and polynomial reductions | formalized | Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, recursively compiled exact raw-machine refinements, many-one reductions, and the NP-complete-in-P implication. | The recursive refinement compiler closes the charged-to-raw machine link, but it does not construct raw paired verifiers beyond the existing CNF-SAT membership witness, prove CNF-SAT is in P or NP-complete, or establish the publication root theorem. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete universal CNF-SAT verifier | formalized-np-membership-only | An unconditional formula-grammar outcome, universal bounded work outcome, exact accept/reject semantics, no-timeout theorem, literal finite-machine paired verifier, and CNF-SAT membership in NP. | This proves CNF-SAT membership in NP only; it does not prove CNF-SAT is in P, NP-completeness, or P = NP. |
| Concrete Cook-Levin dimensions and variable layout | formalized-foundation-only | Fuel padding preserves designated halts and stuck timeouts; every literal machine state is below an explicit finite ceiling; the input, time, and tape dimensions are executable NatPolynomial expressions; and tape-symbol, head, state, certificate-bit, and certificate-length variables occupy proved disjoint in-range blocks. Elementary at-least-one and pair-exclusion CNF clauses have exact propositional semantics. All 20 reviewed theorems have empty axiom closure. | This is the finite layout substrate only. It does not yet construct a transition tableau, prove tableau soundness or completeness, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |
| Concrete fixed-certificate execution tableau | formalized-foundation-only | For one fixed source input, certificate, finite-rule machine, and fuel budget, the canonical raw execution trace has exactly fuel + 1 configurations. Intrinsic first-match transition validity is equivalent to equality with that trace, every valid endpoint equals the ordinary run endpoint, and accept/reject/timeout verdicts agree exactly with boundedDecide. All 30 declarations and eight reviewed theorem types have empty axiom closure. | This is the semantic fixed-certificate tableau only. It does not yet quantify a variable certificate, encode tape windows or tableau rows as CNF, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Uniform bounded verifier tableaux | formalized-foundation-only | For an arbitrary proof-bearing verifier in the concrete NP model and one source input, the complete finite decision pipeline is compiled to one raw machine. Every certificate inside the explicit certificate polynomial shares one answer-independent raw fuel obtained by evaluating the compiled raw-time polynomial at the maximum encoded input size. Per-certificate fuel is below that bound, padding follows a proved halt, exact verdicts are preserved, and language membership is equivalent to an existential bounded accepting semantic tableau. All 41 declarations and nine reviewed theorem types have empty axiom closure. | This is a uniform semantic verifier-tableau equivalence only. It does not encode configurations, transition windows, or variable-length certificates as Boolean clauses; emit CNF; prove emitter size or runtime polynomials; define a polynomial reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |
| Finite local Cook-Levin CNF compiler | formalized-foundation-only | Width-indexed literals materialize exact Boolean assignments and compile to the concrete CNF semantics without out-of-range variables. Unit constraints, finite implications, and pairwise exactly-one groups have exact satisfaction equivalences; local programs have exact recursive clause counts, satisfiability reflection, and a proved well-scoped formula. All 75 declarations and nine reviewed theorem types have empty axiom closure. | This is a local clause compiler only. It does not enumerate the verifier tableau, encode initialization, transition windows, certificate length, or acceptance as a complete formula; prove an all-input emitter size/runtime polynomial; define a reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite whole-tableau Cook-Levin CNF syntax | formalized-foundation-only | A uniform bounded verifier-tableau problem is compiled answer-independently into one finite, well-scoped concrete CNF formula. The program emits one-hot symbol/head/state rows, first-match control updates, untouched-cell preservation, exact input-only or symbolic paired-certificate initialization, and a designated final accept constraint. Canonical encoding/decoding, exact local satisfaction reflection, and exact emitted clause counting are proved. All 81 declarations and ten reviewed theorem types have empty axiom closure. | This is finite formula syntax and local-clause reflection only. It does not yet prove that formula satisfiability is equivalent to an accepting raw verifier execution, prove external encoded-output-size or construction-runtime polynomials, define a concrete polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |
| Finite whole-tableau Cook-Levin CNF semantics | formalized-foundation-only | Every satisfying assignment decodes to a unique-symbol, unique-head, unique-state finite row at each bounded time. The decoded initial row is exact in both verifier modes, each adjacent row is the literal deterministic first-match successor, and the final state is accepting. Conversely, every such intrinsic finite accepting tableau constructs a satisfying assignment. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to this finite semantics. All 161 explicit declarations are axiom-audited; the six reviewed theorem types depend only on the permitted Lean standard axioms Quot.sound and propext, with no project axiom. | This milestone itself proves equivalence only with an intrinsic finite-row model; the following raw-tape milestone supplies the execution bridge. External encoded-output-size and construction-runtime polynomials, a concrete polynomial reduction to CNFSAT, CNFSAT NP-completeness, CNFSAT in P, and P = NP remain absent. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite tableau to raw Tape semantics | formalized-foundation-only | The finite Cook-Levin rows are proved exactly equivalent to ordinary focused raw Tape execution. Absolute two-sided tape observations cover explicit and implicit blanks; the local successor matches designated halts, missing-rule stutter, and the literal first matching raw rule; a centered head invariant rules out finite-window clamping; and both verifier input modes have exact initial rows, including reversible bounded-certificate reconstruction. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to the concrete verifier language. All 54 explicit declarations are axiom-audited, and the nine reviewed theorem types use only the permitted Lean standard axiom allowlist with no project axiom. | This proves exact semantic reduction correctness only. It does not yet prove external encoded-formula-size or formula-construction-runtime polynomials, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, CNFSAT in P, or P = NP. |
| External Cook-Levin encoded-formula size | formalized-foundation-only | The actual canonical unary-indexed CNF bitstring generated for a fixed concrete polynomial-time verifier is bounded by an explicit NatPolynomial evaluated only at the external source-input length. Exact codec lengths, both verifier input modes, every concrete tableau constraint family, program and emitted-clause counts, clause width, and the mode-sensitive exact formula-variable width polynomial are covered. All 110 explicit declarations are axiom-audited; the ten reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom. | This does not implement or time a raw finite formula builder, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Rectangular Cook-Levin formula schedule | formalized-foundation-only | For every concrete verifier-tableau problem, a finite answer-independent rectangular schedule allocates exact constraint, clause, token, and raw-bit slots. Removing empty slots in order reproduces the existing canonical local program, bounded clauses, CNF tokens, and encoded formula. The raw-bit schedule length is exactly the previously proved encodedFormulaSizePolynomial evaluated at external source-input length. All 79 explicit declarations are axiom-audited; the eight reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom. | This is a pure schedule specification. It does not interpret a slot as a constant-time raw-machine action, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Direct Cook-Levin formula cursor | formalized-foundation-only | For every concrete verifier-tableau problem and natural coordinate, direct constraint, clause, token, and raw-bit decoders agree pointwise with the canonical rectangular schedules while preserving out-of-range, valid-padding, and populated states. Fuelled bit traversal proves exact bounded prefixes, complete traversal, one-step-short behavior, terminal behavior, excess-fuel stability, the external encoded-size polynomial, and canonical emitted output. A token-level specification cursor additionally exposes exact in-range, done, and terminal single-step laws. All 136 explicit declarations are axiom-audited; the 16 reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom. | This is a Lean specification cursor. It does not prove constant-time raw slot interpretation, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Literal Cook-Levin builder input-length tally | formalized-foundation-only | A fixed 19-rule work machine starts at the proved all-input framer endpoint, preserves every external source bit, appends exactly one unary tally symbol per source bit in fresh right-side workspace, and returns to the source head. It accepts after exactly 2*n*n + 4*n + 2 work steps; the compiled raw machine reaches the encoded endpoint after exactly 12*n*n + 24*n + 12 steps. Malformed internal scan symbols and one-step-short fuel both time out. All 39 public declarations are axiom-audited; the ten reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project or choice axiom. | This is only the input-length preparation stage. It does not emit formula bits, interpret direct cursor slots as raw transitions, compose a complete raw formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP. |
| Executable Cook-Levin builder input prefix | formalized-foundation-only | One literal finite work machine contains collision-free renamed copies of the total all-input framer and fixed 19-rule unary tally, with a total nine-symbol tape/head-preserving launch between them. Every raw bitstring reaches the exact preserved-input tally endpoint in totalInputFramerWorkSteps(input) + 1 + 2*n*n + 4*n + 2 work transitions and within the external compiled polynomial 18*n*n + 63*n + 93. All 40 public declarations are axiom-audited: 29 have empty closure, one uses only propext, and ten use only propext and Quot.sound. | This is still only an executable input-preparation prefix. It emits no formula bit, performs no raw formula-cursor interpretation, supplies no complete formula builder or construction-time RawRefinement, packages no concrete PolynomialReduction, and establishes neither CNFSAT NP-completeness, CNFSAT in P, nor P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Standalone Cook-Levin builder token appender | formalized-foundation-only | A fixed 59-rule work machine appends any of the four canonical two-bit CNF tokens selected only by its finite control state after a represented source word and exact unary input-length tally. For source length n and k existing tokens, it accepts at the exact endpoint after 2 * (max 1 n + n + k + 3) work steps. Its distinguished start state appends T, the first canonical formula header token, within the external compiled bound 24*n + 48; the two emitted bits are proved equal to the first two bits of every concrete verifier-tableau encoding. Malformed tally/output phases and one-step-short fuel remain timeouts. All 68 public declarations are axiom-audited: 42 have empty closure, 13 use only propext, and 13 use only propext and Quot.sound. | This milestone audits the token appender independently of its later composed use. It does not compute the remaining width header, interpret a dynamic formula cursor coordinate, emit a complete formula, construct a builder RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Composed Cook-Levin builder first-token prefix | formalized-foundation-only | One literal 184-rule finite work machine contains all 116 executable input-prefix rules, nine total symbol-preserving bridge rules, and all 59 token-appender rules under injective disjoint state renamings. Every raw bitstring runs through total framing, exact unary input tallying, the bridge, and the appender to preserve the workspace and emit exactly the first T header token. The exact all-input work trace compiles within 18*n*n + 87*n + 147 raw steps; the final two bits equal the first two bits of every canonical encoded verifier-tableau formula. Prefix-endpoint, malformed tally/output, and one-step-short negative cases time out. All 37 public declarations are axiom-audited: 21 have empty closure, three use only propext, and 13 use only propext and Quot.sound, with no project axiom or Classical.choice. | This emits exactly the fixed answer-independent first T token, hence only the first two canonical formula bits. It does not compute the remaining width header, implement a dynamic cursor, emit a complete formula, construct a builder RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Composed Cook-Levin complete width header | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine contains the 184-rule raw-input/first-token prefix, a structurally generated unary NatPolynomial evaluator, a 16-rule unary-root controller, two complete 59-rule token appenders, and five total nine-symbol bridges under injective pairwise-disjoint state renamings. Its table has exactly 363 plus the evaluator rule count rules. Every raw input follows an exact all-input trace, evaluates the verifier's mode-sensitive formula width into unary scratch space without calling NatPolynomial.eval in executable rules, and emits exactly FormulaWidth copies of T followed by F. The resulting bits are the complete canonical encoded-formula width header, and an external NatPolynomial bounds the compiled raw run. The evaluator's 74 and composition's 84 public declarations are completely axiom-audited: every closure is empty or uses only propext and Quot.sound, with no project axiom or Classical.choice. | This endpoint is the complete answer-independent width header only. It does not implement the dynamic formula cursor or body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Composed Cook-Levin body-start prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete width-header machine, a structurally generated unary evaluator for the next canonical token slot, one complete 59-rule token appender, and two total nine-symbol bridges under injective pairwise-disjoint state renamings. Its table has exactly 440 plus the width-evaluator and next-token-slot-evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary input tally and header workspace, retains next token coordinate FormulaVariableSlotBound + 2 and bit cursor twice that coordinate, and emits exactly FormulaWidth copies of T followed by F and Sep. The resulting bits are the canonical encoded-formula prefix through the first body separator; the compiled run is bounded by the complete-header bound plus 72, six times the unary next-slot evaluator work count, 24*n, and 12*width. All 60 public declarations are axiom-audited: 30 have empty closure, five use only propext, and 25 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed answer-independent body-opening separator after the complete width header and retains its coordinate as data. It does not implement a dynamic formula cursor or subsequent body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Composed Cook-Levin first-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete body-start-prefix machine, a structurally generated unary evaluator for token coordinate FormulaVariableSlotBound + 4, two complete 59-rule token appenders, and three total nine-symbol bridges under four injective pairwise-disjoint state renamings. Its table has exactly 585 plus the width, body-start next-slot, and first-literal next-slot evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary tally and earlier token workspace, retains the corresponding doubled bit cursor, and emits exactly FormulaWidth copies of T followed by F, Sep, T, and F. The final T/F pair is constructively pinned to the positive sign and unary-zero terminator of the first scheduled literal, positive variable zero, and all emitted bits equal encodedFormula.take (2 * (FormulaWidth + 4)). The compiled run is bounded by the body-start bound plus 174, six times the new unary evaluator work count, 48*n, and 24*width. All 74 current public declarations are axiom-audited: 21 have empty closure, three use only propext, and 50 use only propext and Quot.sound, with no project axiom or Classical.choice; the added halt-separation interface supports the reviewed first-clause composition. | This milestone emits only the first canonical literal after the body-opening separator and retains the following coordinate as data. It does not implement a dynamic formula cursor or remaining body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Composed Cook-Levin first-clause prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete first-literal-prefix machine, a structurally generated unary evaluator for token coordinate FormulaVariableSlotBound + 12, and a fixed 535-rule tail made from eight complete 59-rule token appenders and seven total nine-symbol bridges under disjoint injective state renamings. Its table has exactly 1138 plus the width, body-start next-slot, first-literal next-slot, and first-clause next-slot evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary tally and earlier token workspace, retains the corresponding doubled bit cursor, and emits exactly FormulaWidth copies of T followed by F, Sep, T, F, T, T, F, T, T, T, F, and Finish. The final eight tokens are constructively pinned to the remainder of the complete positive at-least-one shape clause on variables zero, one, and two; the following direct token slot is proved to be valid padding. All emitted bits equal encodedFormula.take (2 * (FormulaWidth + 12)). The compiled run is bounded by the first-literal bound plus 1158, six times the new unary evaluator work count, 192*n, and 96*width. All 79 public declarations are axiom-audited: 38 have empty closure, 13 use only propext, and 28 use only propext and Quot.sound, with no project axiom or Classical.choice; the 80-line combined audit additionally covers the predecessor halt-separation theorem. | This milestone emits only the complete first canonical clause after the width header and retains the following coordinate as data. It does not implement a dynamic formula cursor or remaining formula-body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Literal Cook-Levin token-cursor padding step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first-clause prefix with a total nine-symbol bridge and a fixed 45-rule cursor-advance table under disjoint nested state images. Its table has exactly 1192 plus the four inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete first-clause output, and advances the retained unary token coordinate from FormulaVariableSlotBound + 12 to FormulaVariableSlotBound + 13. The direct decoder proves that the consumed coordinate is the first in-range padding slot, so the token-level specification cursor advances without emitting a token. The cursor suffix takes exactly 2*cursorWord.length + 8 work steps including launch, and the compiled run is bounded by FirstClausePrefix.rawTimeBound + 48 + 12*cursorWord.length. All 47 public declarations, including two reviewed dead-state dispatch facts used downstream, are axiom-audited: 14 have empty closure, seven use only propext, and 26 use only propext and Quot.sound, with no project axiom or Classical.choice. | This is one fixed padding-coordinate transition after the complete first clause. It is not a general dynamic cursor loop or raw decoder for arbitrary schedule coordinates, emits no additional formula token, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin first-clause remaining-padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the preceding first-clause cursor step with structurally generated unary evaluators for D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6) and the second-clause coordinate, plus one fixed 25-rule countdown controller. Its table has exactly 1244 plus the six inherited and generated evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete first-clause output, traverses all D remaining first-clause padding coordinates without emitting a token, and reaches FormulaVariableSlotBound + 1 + FormulaTokensPerClause, where direct schedule lookup is proved to return Sep. The recursive specification run agrees at every padding coordinate and the following specification step observes Sep. The compiled exact run has an explicit external input-size polynomial bound, accepts within that bound, and remains fail-closed for malformed countdown scratch/root states, the unlaunched prefix endpoint, and one-step-short fuel. All 83 public declarations plus one predecessor transport theorem are axiom-audited: 37 have empty closure, 11 use only propext, and 36 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone executes exactly the remaining first-clause padding block and identifies the second-clause separator boundary. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the second clause or remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Uniform Cook-Levin full-schedule cursor controller | formalized-foundation-only | For every concrete polynomial-time verifier and raw input, the exact polynomial body-opportunity count equals the complete token schedule after the padded unary header. A semantic cursor returns the full canonical schedule and emitted formula. One deterministic finite raw machine composes the complete header, generated count and terminal-coordinate evaluators, and a fixed positive countdown table. It consumes the arbitrary body count exactly, materializes the unique terminal coordinate, accepts within an explicit input-size polynomial bound, and remains fail-closed for malformed countdown geometry, the unlaunched header endpoint, and one-step-short fuel. | This milestone provides a uniform control scaffold for the complete input-dependent Cook-Levin token schedule, but the raw loop does not decode or emit the visited body entries. It is not the complete raw formula builder or its FunctionProgram.RawRefinement, does not package the concrete Cook-Levin PolynomialReduction, does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, does not close a global gate, and does not prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Uniform Cook-Levin arbitrary-slot header router | formalized-foundation-only | For every concrete polynomial-time verifier and every coordinate below its complete direct token schedule, one fixed 54-rule deterministic work machine compares the coordinate with the exact problem-derived first-body boundary. Its exact trace accepts precisely the header branch and rejects equality or the post-header branch. The semantic route is definitionally faithful to formulaTokenSlotDirect; the compiled execution is bounded by the verifier-derived polynomial 36 * terminalSlotPolynomial^2; one-step-short fuel and malformed symbols remain timeout. All 51 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone routes only the top-level header boundary. It does not decode the post-header quotient and remainder into a clause and within-clause token slot, does not emit a body token, and is not the complete raw formula builder or its FunctionProgram.RawRefinement. It does not package the concrete Cook-Levin PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a global gate, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Uniform Cook-Levin arbitrary-slot post-header decoder | formalized-foundation-only | For every concrete polynomial-time verifier and every coordinate below its complete direct token schedule, the M209 outer route is refined by a structurally recursive all-coordinate post-header decoder. Every body result gives typed finite clause and within-clause coordinates that reconstruct the exact remainder; the unique Finish and out-of-range cases are characterized; and the interpreted route agrees with direct token lookup. A checked reader recovers the exact shifted remainder from every M209 raw comparison result while preserving M209's exact raw trace. All 22 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone adds an unbounded semantic clause/within-clause decoder and extracts the exact shifted remainder from M209's checked raw result. It does not implement raw division, raw body-token emission, the complete raw formula builder or its FunctionProgram.RawRefinement. It does not package the concrete Cook-Levin PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a global gate, or prove P = NP. |
| Fixed Cook-Levin post-header raw quotient/remainder divider | formalized-foundation-only | For every natural dividend and positive unary width, one fixed 99-rule deterministic work machine reaches the exact natural quotient and strict remainder tape, reconstructs the dividend, compiles with exactly six raw transitions per certified work step, times out one step short, and satisfies an explicit quadratic bound in the complete unary encoded input length. The zero-width dispatcher returns none. M210 body coordinates instantiate the same decoded quotient and remainder. All 57 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone proves a standalone fixed unary divider. It is not spliced onto M209's checked raw result, does not classify Finish or out-of-range routes, does not emit or append a Cook-Levin body token, and does not complete the raw formula builder or its FunctionProgram.RawRefinement. It does not package the concrete Cook-Levin PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a global gate, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-coordinate Cook-Levin post-header raw launch | formalized-foundation-only | For every concrete verifier tableau problem and every natural coordinate, executable Lean orchestration reads the exact shifted remainder from M209's checked raw terminal configuration and conditionally launches M211's fixed positive-width divider. The exact M209 and M211 traces are retained; every M210 body, Finish, and out-of-range result is recovered; every coordinate inside the complete token schedule excludes the out-of-range post-header branch; and the summed compiled work is bounded by one verifier-derived polynomial in the source-input length. All 24 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone composes M209's checked result reader and M211's exact divider through executable Lean orchestration. It is not a literal raw tape-to-tape bridge, does not preserve the complete builder workspace through such a rewrite, does not emit or append a Cook-Levin body or Finish token, and does not complete the raw formula builder or its FunctionProgram.RawRefinement. It does not package the concrete Cook-Levin PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a global gate, or prove P = NP. |
| Literal Cook-Levin post-header raw tape bridge | formalized-foundation-only | For every concrete verifier tableau problem, arbitrary exterior workspace, and in-range coordinate, one fixed collision-free 351-rule bridge consumes the exact M209 equality or greater-than terminal tape, derives zero or the exact positive shifted remainder, copies the positive problem width, and reaches a shielded M211 input. Exact bridge and divider traces preserve the complete exterior, recover the exact quotient and remainder, compile at six raw steps per work step, time out one step short, and fit one verifier-derived source-size polynomial. All 78 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone closes the literal M209-to-M211 physical tape handoff only. It does not select, emit, or append a Cook-Levin body or Finish token, iterate the complete token schedule, complete the raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Literal Cook-Levin post-divider raw route classifier | formalized-foundation-only | For every concrete verifier tableau problem, arbitrary safe exterior prefix and builder workspace, and in-range coordinate, one fixed collision-free 180-rule bridge consumes the exact M213 equality or greater-than divider-terminal tape, copies the exact problem-derived clause count from a restored sidecar, preserves the complete exterior and remainder ledger, and constructs a shielded M209 comparison input from the literal quotient marks. Exact bridge and comparator traces cannot cross their boundaries, compile at six raw steps per work step, time out one step short, agree with every M210 body or Finish route, and fit one verifier-derived source-size polynomial. All 85 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone closes the literal post-divider body-versus-Finish route-classification handoff only. It does not inspect, select, emit, or append a Cook-Levin token, iterate the complete token schedule, complete the raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-coordinate Cook-Levin selected-token launch | formalized-foundation-only | For every concrete verifier tableau problem and every post-header schedule coordinate, the canonical schedule alone derives padding, the exact body token, or the unique Finish token without a supplied route or token. M214's physical classifier contract is retained, every populated entry runs the existing fixed 59-rule appender to the exact next emitted prefix, exact work and compiled traces time out one step short, and the combined staged work fits one verifier-derived source-size polynomial. All 30 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone derives each canonical post-header padding, body-token, or Finish entry and launches the existing fixed token appender for populated entries through executable Lean orchestration. The selection handoff is not yet a literal raw tape rewrite. It does not iterate the complete token schedule, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |
| Complete semantic Cook-Levin schedule iteration | formalized-foundation-only | For every concrete verifier tableau problem, structural recursion consumes every verifier-derived post-header schedule opportunity without a supplied coordinate, token, route, trace, schedule, or precomputed formula. Every bounded run equals the canonical emitted prefix, the complete run equals encodeCNFTokens of the generated formula, every coordinate retains M215's exact classifier/appender evidence for arbitrary workspace, and the aggregate staged compiled work fits one source-size polynomial. All 12 public declarations are axiom-audited with only the approved Lean-standard closure and no project axiom or Classical.choice. | This milestone composes M215's proof-carrying per-coordinate stages through executable Lean recursion. It is not one literal raw-machine loop and does not prove a physical tape-to-tape handoff between successive stages, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Physical optional-token dispatch | formalized-foundation-only | For every raw input, arbitrary exterior workspace, emitted prefix, and optional token request, one fixed collision-free 64-rule machine reads one of five physical request symbols, restores the canonical builder left boundary, and either accepts without changing padding output or enters the existing renamed 59-rule token appender. Exact work and six-for-one compiled traces, one-step-short nonhalting, malformed blank-request timeout, canonical scheduleEntry specialization at every post-header coordinate, and one verifier-derived source-size polynomial are proved. All 49 public declarations are axiom-audited: 23 have empty closure, 13 use only propext, and 13 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone physically dispatches a supplied tape-resident padding or CNF-token request into the existing fixed appender. It does not derive that request from M214's raw coordinate classifier, connect the classifier output directly to this request cell, iterate one physical selector/dispatcher loop through the complete token schedule, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-coordinate physical dispatch schedule | formalized-foundation-only | For every concrete verifier problem and every canonical post-header coordinate, a total recursion applies M217 optional-token semantics, agrees at every bounded prefix with the canonical emitted token stream, and reaches the complete encoded formula. Every coordinate and arbitrary exterior retains M217 exact work, compiled and one-step-short physical traces together with M214 classifier evidence. The sum of all exact dispatch costs is bounded by the problem-derived body-slot count times M217 uniform bound, one polynomial in encoded source-input length. All 16 public declarations are axiom-audited: two have empty closure and fourteen use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone composes exact physical dispatch traces over every canonical request, but each request is still constructed from the Lean schedule. It does not derive a request from M214 raw classifier cells, connect successive final and initial tapes, implement one literal raw-machine loop, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Physical Finish-request handoff | formalized-foundation-only | For every concrete verifier problem, the unique final post-header coordinate is derived internally and M214 proves its in-range equal-classifier result. The canonical builder workspace is protected beyond the comparator end marker; one fixed writer produces M217's tape-resident Finish request, and one collision-free 137-rule composition runs the comparator, writer and dispatcher. Exact work, six-for-one compiled and one-step-short traces hold for arbitrary classifier exterior, and one verifier-derived polynomial bounds compiled work. All 49 public declarations are axiom-audited: 23 have empty closure, six use only propext, and 20 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone physically derives only the unique canonical Finish request. It does not derive body-token or padding requests from raw coordinates, preserve the builder suffix through every preceding classifier stage, connect successive schedule configurations into one literal raw-machine loop, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-coordinate physical classifier pipeline | formalized-foundation-only | For every concrete verifier problem, every canonical post-header coordinate and arbitrary protected builder workspace, one fixed collision-free 711-rule machine composes M213's router-to-divider bridge, M211's divider, M214's divider-to-comparator bridge and comparator, and three fixed launch transitions. Three exact physical tape handoffs preserve the workspace suffix, the final raw state agrees with M214's typed body-or-Finish semantics, and exact work, six-for-one compiled execution, one-step-short nonhalting, component decomposition and one source-size polynomial bound are proved. All 63 public declarations are axiom-audited: 44 have empty closure, four use only propext, and 15 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone physically composes the complete suffix-preserving classifier pipeline for every canonical post-header coordinate. It does not derive body-token or padding request symbols, connect the resulting body or Finish classification to M217's request dispatcher, iterate one literal raw-machine schedule loop, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Full-classifier Finish-request cell | formalized-foundation-only | For every concrete verifier problem and arbitrary protected builder workspace, the canonical Finish coordinate is derived internally. M220's full 711-rule classifier has its terminal verdict names swapped without changing transition semantics, then is chained through one fixed launch table to M219's one-cell writer as a collision-free 721-rule machine. The classifier reaches the end marker, the writer changes exactly that focused cell to M217's Finish request while preserving the rest of the tape, and exact work, six-for-one compiled execution, one-step-short nonhalting and one source-size polynomial bound are proved. All 36 public declarations are axiom-audited: nine have empty closure, three use only propext, and 24 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone connects only the unique canonical Finish terminal of the full physical classifier to one literal M217-compatible request cell. It does not derive body-token or padding request symbols, orient the preserved workspace for the dispatcher, run M217 from the M220 endpoint, iterate one literal raw-machine schedule loop, construct the complete raw formula builder or its FunctionProgram.RawRefinement, or package the concrete Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Full-classifier Finish-workspace orientation | formalized-foundation-only | For every concrete verifier problem, M222 prepends one blank sentinel to the canonical builder workspace, derives the complete M220/M221 classifier prefix, proves that prefix contains no blank, and chains M221 to one fixed ten-rule left scanner. The resulting collision-free 740-rule machine crosses exactly the classifier prefix and halts at the sentinel with a tape equal to the spatial mirror of M217's canonical Finish request entry, retaining classifier evidence as exterior data. Exact work, six-for-one compiled execution, one-step-short nonhalting, a derived prefix-size bound and one source-input-size polynomial bound are proved. All 57 public declarations are axiom-audited: 27 have empty closure and 30 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone closes only the physical workspace-orientation edge for the unique full-classifier Finish path. Equality with a spatial mirror of M217's canonical request entry does not prove that M217's existing machine executes on the mirrored representation. It does not derive body-token or padding requests, run a mirrored dispatcher, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Full-classifier reflected Finish dispatch | formalized-foundation-only | For every concrete verifier problem, M223 reflects M217's fixed 64-rule dispatcher by exchanging left and right moves while preserving rule queries and states, and proves that reflection transports exact finite-machine execution. Chained after M222, the resulting collision-free 813-rule machine executes the complete classifier's unique Finish path from raw classifier entry through workspace orientation and the reflected dispatcher to an appender endpoint containing the complete canonical CNF token encoding. Exact work, six-for-one compiled execution, one-step-short nonhalting and one source-input-size polynomial bound are proved. All 51 public declarations are axiom-audited: 23 have empty closure, seven use only propext, and 21 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone closes only reflected M217 execution for the unique full-classifier Finish path. It does not derive body-token or padding request symbols, connect every classifier outcome to the dispatcher, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| First-body separator physical dispatch | formalized-foundation-only | For every concrete verifier problem, M224 derives first-body coordinate zero and its canonical separator entry, runs the complete 711-rule classifier through the populated-body terminal, crosses the positive unary clause-count suffix with a fixed two-rule request writer, reorients the protected builder workspace with a fixed ten-rule scanner, and executes M223's reflected 64-rule dispatcher. One collision-free 814-rule machine reaches the exact canonical prefix ending in the first clause separator, with exact work, six-for-one compiled execution, one-step-short nonhalting and one source-input-size polynomial bound. All 121 public declarations are axiom-audited: 53 have empty closure, one uses only propext, and 67 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone closes only the first populated full-classifier body route. It does not select arbitrary body tokens, derive padding requests, connect every classifier outcome, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-body staged-request physical dispatch | formalized-foundation-only | For every concrete verifier problem and every coordinate in its complete clause-token body rectangle, M225 runs the complete 711-rule classifier while preserving an explicitly staged canonical optional-token request, relays that request through one fixed 14-rule scanner, and executes M223's reflected 64-rule dispatcher. One collision-free 807-rule composition covers populated and padding coordinates and reaches each exact next canonical emitted prefix, with exact work, six-for-one compiled execution, one-step-short nonhalting and one source-input-size polynomial bound. The current M227 audit covers all 82 public declarations: 33 have empty closure, nine use only propext, and 40 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone stages each canonical body or padding request explicitly on protected tape. It does not synthesize that request from raw classifier state, include the unique Finish route in the same machine, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-route physical classifier terminal join | formalized-foundation-only | For every concrete verifier problem, every coordinate in its complete post-header schedule and arbitrary protected workspace, M226 injectively embeds M220's complete 711-rule classifier and adds one total nine-symbol, tape-preserving redirect from its unique Finish/reject terminal to the same continuation-ready accepting state reached by all body coordinates. The resulting collision-free 720-rule machine takes zero additional steps on body routes and exactly one redirect step on Finish, with exact work, six-for-one compiled execution, one-step-short nonhalting and one source-input-size polynomial bound. All 34 public declarations are axiom-audited: 22 have empty closure, three use only propext, and nine use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone normalizes only the complete physical classifier's terminal control flow. It does not synthesize, stage or dispatch a body-token request, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-clause separator step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first-clause padding run with a selected 59-rule Sep appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance under collision-free nested state images. Its table has exactly 1366 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical token prefix through the Sep beginning clause two, and advances the retained unary coordinate by one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 13)), and direct schedule lookup proves that the following token is F. The compiled run is bounded by BuilderFirstClausePaddingRun.rawTimeBound + 246 + 24*n + 12*FormulaWidth + 12*cursorWord.length. Malformed appender tally/output, malformed cursor scratch, both unlaunched endpoints, and one-step-short total fuel remain timeout. All 54 new public declarations plus two reviewed predecessor dispatch facts are axiom-audited: 15 have empty closure, 11 use only propext, and 30 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed populated Sep transition beginning clause two and advances to the following F coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit that F or the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-clause first-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause-separator prefix with two selected 59-rule F appenders, four total symbol-preserving bridges, and two copies of the existing 45-rule cursor advance under collision-free nested state images. One F/cursor component has 113 rules, the two-component suffix has 235 rules, and the global table has exactly 1610 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable zero in clause two, and retains the negative-sign coordinate on variable one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 15)); direct schedule lookup proves the sign, unary-zero terminator, and following sign are all F. The compiled run is bounded by BuilderSecondClauseSeparatorStep.rawTimeBound + 564 + 48*n + 24*FormulaWidth + 24*cursorWord.length. All four pre-launch endpoints, malformed tally/output in both appenders, malformed scratch in both cursors, and one-step-short fuel remain timeout. All 85 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 25 have empty closure, 18 use only propext, and 44 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed negative literal on variable zero in clause two and advances to the following negative-sign coordinate. It does not complete clause two, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-clause second-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-two first-literal prefix with selected 59-rule F, T, and F appenders, six total symbol-preserving bridges, and three copies of the existing 45-rule cursor advance under collision-free nested state images. The selected T/cursor component has 113 rules, the T/F tail has 235 rules, the complete three-token suffix has 357 rules, and the global table has exactly 1976 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable one in clause two, and retains the following Finish coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 18)); direct schedule lookup proves the sign F, unary unit T, terminator F, and following Finish. The compiled run is bounded by BuilderSecondClause-FirstLiteralPre-fix.rawTimeBound + 1026 + 72*n + 36*FormulaWidth + 36*cursorWord.length. All six pre-launch endpoints, malformed tally/output in all three appenders, malformed scratch in all three cursors, and one-step-short fuel remain timeout. All 113 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 34 have empty closure, 25 use only propext, and 56 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed negative literal on variable one in clause two and advances to the following Finish coordinate. It does not emit the clause terminator, complete clause two, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionPro-gram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin complete second-clause prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-two second-literal prefix with a selected 59-rule Finish appender, two total symbol-preserving bridges, and one copy of the existing 45-rule cursor advance under collision-free state images. The fixed suffix has exactly 113 rules and the global table has exactly 2098 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete second clause, and retains its first in-range padding coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 19)); direct schedule lookup proves the executed opportunity is Finish and the retained next opportunity is padding. The compiled run is bounded by BuilderSecondClauseSecondLiteralPrefix.rawTimeBound + 390 + 24*n + 12*FormulaWidth + 12*cursorWord.length. Both pre-launch endpoints, malformed appender tally/output, malformed cursor scratch, and one-step-short fuel remain timeout. All 55 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 15 have empty closure, 10 use only propext, and 32 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed Finish terminator that completes clause two and advances to its first padding coordinate. It does not traverse clause-two padding, reach clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-clause remaining-padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause prefix with structurally generated unary evaluators for D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6) + 5 and the third-clause coordinate, plus the reused fixed 25-rule PaddingCountdown controller. Three total symbol-preserving WorkChain bridges yield a table with exactly 2150 plus the six inherited/generated predecessor evaluator rule counts and the two new evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete second-clause output, traverses all D = FormulaTokensPerClause - 7 remaining second-clause padding coordinates without emitting a token, and reaches FormulaVariableSlotBound + 1 + 2 * FormulaTokensPerClause, where direct schedule lookup is proved to return Sep. The recursive specification run agrees at every padding coordinate and the following specification step observes Sep. The emitted bits remain encodedFormula.take (2 * (FormulaWidth + 19)). The compiled exact run is bounded by BuilderSecondClausePrefix.rawTimeBound + 18 + 6 * countEvaluator.workSteps + 6 * (D * (2 * countRootPrefixLength + 8) + D * D) + 6 * targetEvaluator.workSteps, accepts within that external input-size polynomial, and remains fail-closed for malformed countdown scratch/root states, the unlaunched | This milestone executes exactly the remaining second-clause padding block and identifies the third-clause separator boundary without emitting a token. It reaches clause three only as a retained coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the third-clause separator or remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin third-clause separator step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause padding run with the reused selected 59-rule Sep appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance under collision-free nested state images. Its table has exactly 2272 plus eight inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical token prefix through the Sep beginning clause three, and advances the retained unary coordinate by one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 20)), and direct schedule lookup proves that the following token is F. The compiled run is bounded by BuilderSecondClausePaddingRun.rawTimeBound + 330 + 24*n + 12*FormulaWidth + 12*cursorWord.length. All 48 new public declarations plus eight reviewed suffix interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed populated Sep transition beginning clause three and advances to the following F coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit that F or the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
| --- | --- | --- | --- |
| Cook-Levin third-clause first-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause-separator prefix with the reused 235-rule two-F appender/cursor suffix behind one total nine-symbol bridge under collision-free nested state images. One F/cursor component has 113 rules, the two-component suffix has 235 rules, and the global table has exactly 2516 plus the eight inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable zero in clause three, and retains the negative-sign coordinate on variable two. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 22)); direct schedule lookup proves the sign, unary-zero terminator, and following sign are all F, with the third excluded pair constructively identified as variables zero and two. The compiled run is bounded by BuilderThirdClauseSeparatorStep.rawTimeBound + 732 + 48*n + 24*FormulaWidth + 24*cursorWord.length. All four pre-launch endpoints, malformed tally/output in both appenders, malformed scratch in both cursors, and one-step-short fuel remain timeout. All 74 new public declarations plus eleven reviewed suffix declarations and two reviewed cursor dead-loop facts are axiom-audited: 24 have empty closure, 18 use only propext, and 45 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed negative literal on variable zero in clause three and advances to the following negative-sign coordinate. It does not emit that following F, complete clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin third-clause second-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause-first-literal prefix with a fixed 479-rule F/T/T/F appender/cursor suffix behind one total nine-symbol bridge under collision-free nested state images. The nested component tables have 113, 235, 357, and 479 rules, and the global table has exactly 3004 plus the eight inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable two in clause three, and retains the following Finish coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 26)); direct schedule lookup proves the sign F, both unary units T, the terminator F, and the following Finish. The compiled run is bounded by BuilderThirdClause-FirstLiteralPre-fix.rawTimeBound + 1752 + 96*n + 48*FormulaWidth + 48*cursorWord.length. All eight pre-launch endpoints, malformed tally/output in all four appenders, malformed scratch in all four cursors, and one-step-short fuel remain timeout. All 145 public declarations are axiom-audited: 46 have empty closure, 32 use only propext, and 67 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed negative literal on variable two in clause three and advances to the following Finish coordinate. It does not emit the following Finish, complete clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionPro-gram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin complete third-clause prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-three second-literal prefix with a selected 59-rule Finish appender, two total symbol-preserving bridges, and one copy of the existing 45-rule cursor advance under collision-free state images. The fixed suffix has exactly 113 rules and the global table has exactly 3126 plus the eight inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete third clause, and retains its first in-range padding coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 27)); direct schedule lookup proves the executed opportunity is Finish and the retained next opportunity is padding. The compiled run is bounded by BuilderThirdClauseSecondLiteralPrefix.rawTimeBound + 498 + 24*n + 12*FormulaWidth + 12*BuilderThirdClauseSeparatorStep.cursorWord.length. Both pre-launch endpoints, malformed appender tally/output, malformed cursor scratch, and one-step-short fuel remain timeout. All 55 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 14 have empty closure, 10 use only propext, and 33 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits only the fixed Finish terminator that completes clause three and advances to its first padding coordinate. It does not traverse clause-three padding, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin third-clause remaining-padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause prefix with structurally generated unary evaluators for D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6) + 4 and the fourth-clause coordinate, plus the reused fixed 25-rule PaddingCountdown controller. Three total symbol-preserving WorkChain bridges yield a table with exactly 3178 plus the eight inherited/generated predecessor evaluator rule counts and the two new evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete third-clause output, traverses all D = FormulaTokensPerClause - 8 remaining third-clause padding coordinates without emitting a token, and reaches FormulaVariableSlotBound + 1 + 3 * FormulaTokensPerClause, where direct schedule lookup is proved to return Sep. The recursive specification run agrees at every padding coordinate and the following specification step observes Sep. The emitted bits remain encodedFormula.take (2 * (FormulaWidth + 27)). The compiled exact run is bounded by BuilderThirdClausePrefix.rawTimeBound + 18 + 6 * countEvaluator.workSteps + 6 * (D * (2 * countRootPrefixLength + 8) + D * D) + 6 * targetEvaluator.workSteps, accepts within that external input-size polynomial, and remains fail-closed for malformed countdown scratch/root states, the unlaunched | This milestone executes exactly the remaining third-clause padding block and identifies the fourth-clause separator boundary without emitting a token. It reaches clause four only as a retained coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the fourth-clause separator or remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin fourth-clause separator step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause padding run with the reused selected 59-rule Sep appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 3300 plus the ten inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the fourth-clause Sep; preserves encodedFormula.take (2 * (FormulaWidth + 28)); and retains coordinate FormulaVariableSlotBound + 1 + 3 * FormulaTokensPerClause + 1, whose direct next schedule token is F. The external compiled bound evaluates to BuilderThirdClausePaddingRun.rawTimeBound + 426 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused separator/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the fixed Sep that starts clause four. It observes but does not emit the following F, does not complete clause four, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin fourth-clause first-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete fourth-clause separator prefix with the reused 357-rule F/T/F appender/cursor suffix through one outer total nine-symbol WorkChain bridge. The global table has exactly 3666 plus the ten inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits the complete first negative literal on variable one in clause four; preserves encodedFormula.take (2 * (FormulaWidth + 31)); and retains coordinate FormulaVariableSlotBound + 1 + 3 * FormulaTokensPerClause + 4, whose direct next schedule token is F. The external compiled bound evaluates to BuilderFourthClauseSeparatorStep.rawTimeBound + 1422 + 72 * n + 36 * FormulaWidth + 36 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, all six unlaunched-endpoint, and one-step-short results are proved. All 97 new public declarations, 16 reviewed reused suffix interfaces, and two cursor dead-state facts are axiom-audited: 33 have empty closure, 25 use only propext, and 57 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly the fixed first negative literal on variable one in clause four. It observes but does not emit the following second-literal F, does not complete clause four, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin fourth-clause second-literal prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete fourth-clause first-literal prefix with the reused 479-rule F/T/T/F appender/cursor suffix through one outer total nine-symbol WorkChain bridge. The global table has exactly 4154 plus the ten inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits the complete second negative literal on variable two in clause four; preserves encodedFormula.take (2 * (FormulaWidth + 35)); and retains coordinate FormulaVariableSlotBound + 1 + 3 * FormulaTokensPerClause + 8, whose direct next schedule token is Finish. The external compiled bound evaluates to BuilderFourthClauseFirstLiteralPrefix.rawTimeBound + 2232 + 96 * n + 48 * FormulaWidth + 48 * BuilderFourthClauseSeparatorStep.cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, all eight unlaunched-endpoint, and one-step-short results are proved. All 124 new public declarations, 21 reviewed reused suffix interfaces, and two cursor dead-state facts are axiom-audited: 46 have empty closure, 32 use only propext, and 69 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly the fixed second negative literal on variable two in clause four. It observes but does not emit the following Finish, does not complete clause four, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin complete fourth-clause prefix | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the fourth-clause second-literal prefix with a selected 59-rule Finish appender, one existing 45-rule cursor advance, and two total nine-symbol WorkChain bridges. The selected suffix has exactly 113 rules and the global table has exactly 4276 plus the ten inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits the Finish that completes clause four; preserves encodedFormula.take (2 * (FormulaWidth + 36)); and retains coordinate FormulaVariableSlotBound + 1 + 3 * FormulaTokensPerClause + 9, whose direct next schedule token is padding. The external compiled bound evaluates to BuilderFourthClauseSecondLiteralPrefix.rawTimeBound + 618 + 24 * n + 12 * FormulaWidth + 12 * BuilderFourthClauseSeparatorStep.cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 55 new public declarations and two cursor dead-state facts are axiom-audited: 14 have empty closure, 10 use only propext, and 33 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly the fixed Finish terminator that completes clause four and advances to its first padding coordinate. It does not traverse clause-four padding, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin fourth-clause remaining-padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete fourth-clause prefix with two structurally generated unary evaluators, the reused 25-rule PaddingCountdown machine, and three total nine-symbol WorkChain bridges. Its table has exactly 4328 plus twelve inherited/generated unary-evaluator rule counts. For every raw bitstring it traverses exactly FormulaTokensPerClause - 9 padding opportunities without emitting a token, preserves encodedFormula.take (2 * (FormulaWidth + 36)), and retains coordinate FormulaVariableSlotBound + 1 + 4 * FormulaTokensPerClause. Direct schedule lookup and the specification cursor both prove that this first opportunity in the intentionally empty fifth fixed-width clause slot is padding. The external compiled bound is BuilderFourthClausePrefix.rawTimeBound + 18 + six times the count-evaluator work, countdown bound, and target-evaluator work. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-countdown, unlaunched-predecessor, and one-step-short results are proved. All 65 new public declarations and three reused countdown interfaces are axiom-audited: 26 have empty closure, 9 use only propext, and 33 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone traverses only the remaining padding in clause four and retains the first opportunity in the intentionally empty fifth clause rectangle. It does not traverse that empty rectangle, reach the next constraint, emit another token, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin empty fifth-clause padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the fourth-clause padding predecessor with two structurally generated unary evaluators, the reused 25-rule PaddingCountdown machine, and three total nine-symbol WorkChain bridges. Its table has exactly 4380 plus fourteen inherited/generated unary-evaluator rule counts. For every raw bitstring it traverses exactly FormulaTokensPerClause padding opportunities in the intentionally empty fifth fixed-width clause rectangle without emitting a token, preserves encodedFormula.take (2 * (FormulaWidth + 36)), and retains coordinate FormulaVariableSlotBound + 1 + 5 * FormulaTokensPerClause. Direct schedule lookup and the specification cursor prove that every traversed fifth-slot opportunity and the first opportunity in the intentionally empty sixth slot are padding. The external compiled bound is BuilderFourthClausePaddingRun.rawTimeBound + 18 + six times the count-evaluator work, countdown bound, and target-evaluator work. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-countdown, unlaunched-predecessor, and one-step-short results are proved. All 65 new public declarations and three reused countdown interfaces are axiom-audited: 28 have empty closure, 9 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone traverses only the intentionally empty fifth clause rectangle and retains the first opportunity in the intentionally empty sixth clause rectangle. It does not traverse that sixth rectangle, reach the next constraint, emit another token, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin remaining first-constraint padding run | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the fifth-clause padding predecessor with two structurally generated unary evaluators, the reused 25-rule PaddingCountdown machine, and three total nine-symbol WorkChain bridges. Its table has exactly 4432 plus sixteen inherited/generated unary-evaluator rule counts. For every raw bitstring it traverses exactly (FormulaVariableSlotBound - 2) * (FormulaVariableSlotBound + 2) * FormulaTokensPerClause = (FormulaClauseSlotsPerConstraint - 5) * FormulaTokensPerClause remaining empty token opportunities of the first scheduled constraint without emitting a token, preserves encodedFormula.take (2 * (FormulaWidth + 36)), and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause. Direct schedule lookup and the specification cursor prove that every traversed opportunity is padding and the endpoint is the Sep beginning the second scheduled constraint. The external compiled bound is BuilderFifthClausePaddingRun.rawTimeBound + 18 + six times the count-evaluator work, countdown bound, and target-evaluator work. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-countdown, unlaunched-predecessor, and one-step-short results are proved. All 65 new public declarations and three reused countdown interfaces are axiom-audited: 26 have empty closure, 9 use only | This milestone traverses only the remaining empty clause rectangles of the first scheduled constraint and retains the Sep beginning the second scheduled constraint. It observes but does not emit that separator, does not emit the next constraint's first literal, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint separator step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first-constraint padding run with the reused selected 59-rule Sep appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 4554 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the Sep beginning the second scheduled constraint; preserves encodedFormula.take (2 * (FormulaWidth + 37)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 1, whose direct next schedule token is T. The external compiled bound evaluates to BuilderFirstConstraintPaddingRun.rawTimeBound + 534 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused separator/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the fixed Sep that starts the second scheduled constraint. It observes but does not emit the following T, does not emit the first literal or traverse the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal sign step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint separator step with the reused selected 59-rule T appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 4676 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the positive sign beginning the second scheduled constraint's first literal; preserves encodedFormula.take (2 * (FormulaWidth + 38)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 2, whose direct next schedule token is the first unary T of a nonzero variable index. The external compiled bound evaluates to BuilderSecondConstraintSeparatorStep.rawTimeBound + 546 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused true-token/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the fixed positive T sign that begins the second scheduled constraint's first literal. It observes but does not emit the following unary T, does not complete that literal or traverse the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal first unary-unit step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal sign step with the reused selected 59-rule T appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 4798 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the first unary T of the second scheduled constraint's first variable index; preserves encodedFormula.take (2 * (FormulaWidth + 39)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 3. A constructive schedule proof establishes that the index is at least three, so the direct next schedule token is the second unary T. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralSignStep.rawTimeBound + 558 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused true-token/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the first unary T of the second scheduled constraint's first variable index. It observes but does not emit the following second unary T, does not complete that literal or traverse the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal second unary-unit step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal first-unary-unit step with the reused selected 59-rule T appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 4920 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the second unary T of the second scheduled constraint's first variable index; preserves encodedFormula.take (2 * (FormulaWidth + 40)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 4. A constructive schedule proof establishes that the index is at least three, so the direct next schedule token is the third unary T. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralFirstUnaryUnitStep.rawTimeBound + 570 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused true-token/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the second unary T of the second scheduled constraint's first variable index. It observes but does not emit the following third unary T, does not complete that literal or traverse the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal third unary-unit step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal second-unary-unit step with the reused selected 59-rule T appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 5042 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the third and final unary T of the second scheduled constraint's first variable index; preserves encodedFormula.take (2 * (FormulaWidth + 41)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 5. A constructive schedule case split proves the selected variable index is exactly three in both the width-one head-variable and wider position-one blank-symbol branches, so the direct next schedule token is the terminating F. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralSecondUnaryUnitStep.rawTimeBound + 582 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused true-token/cursor interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, | This milestone emits exactly one token: the third and final unary T of the second scheduled constraint's first variable index. It observes but does not emit the following terminating F, does not complete that literal or traverse the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal terminator step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal third-unary-unit step with the reused selected 59-rule F appender and 45-rule cursor advance through one outer total nine-symbol WorkChain bridge. The global table has exactly 5164 plus the sixteen inherited/generated unary-evaluator rule counts. Every raw input follows exact prefix, appender, cursor, bridge, suffix, and combined traces; emits exactly the terminating F of the second scheduled constraint's first literal; preserves encodedFormula.take (2 * (FormulaWidth + 42)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 6. A constructive schedule case split proves the direct next schedule token is Finish when tapeWidth is one and the positive T beginning the next literal at wider widths. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralThirdUnaryUnitStep.rawTimeBound + 594 + 24 * n + 12 * FormulaWidth + 12 * cursorWord.length. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, malformed-workspace, both unlaunched-endpoint, and one-step-short results are proved. All 48 new public declarations plus eight reviewed reused false-token/cursor and dead-state interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone emits exactly one token: the terminating F of the second scheduled constraint's first literal. It observes but does not emit the following Finish in the width-one case or the following positive T in wider cases, does not emit the remainder of the second constraint or traverse that constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint first-literal successor token step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal terminator with the represented-width unary evaluator, one 93-rule finite width branch that enters a single reused 59-rule token appender at either Finish or T, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5284 plus the eighteen inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, branch/appender, retained-coordinate, bridge, suffix, and combined traces; emits Finish exactly when tapeWidth is one and T at every wider width; preserves encodedFormula.take (2 * (FormulaWidth + 43)); and retains coordinate FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 7. Direct lookup and the specification cursor prove the following opportunity is padding at width one and unary T at wider widths. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralTerminatorStep.rawTimeBound + 600 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, predecessor-unlaunched-endpoint, and one-step-short results are proved while component malformed-workspace behavior remains fail-closed. All 80 new public declarations plus two strengthened | This milestone emits exactly one width-selected token after the terminating F of the second scheduled constraint's first literal: Finish at width one or positive T at wider widths. It observes but does not emit the following padding opportunity at width one or unary T at wider widths, does not emit the remainder of the second constraint or traverse that constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint padding-or-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-constraint first-literal successor-token predecessor with the represented-width unary evaluator, one 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5404 plus the twenty inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width T-appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the first unary T of the second literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 1)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 8. Direct lookup and the specification cursor prove the following slot is again padding at width one and the second unary T at wider widths. The external compiled bound evaluates to BuilderSecondConstraintFirstLiteralSuccessorTokenStep.rawTimeBound + 612 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, predecessor-unlaunched-endpoint, and one-step-short results are | This milestone consumes exactly one width-selected schedule opportunity after the second scheduled constraint first literal: padding with no emitted token at width one or the first unary T of the second literal at wider widths. It observes but does not consume the following padding opportunity at width one or second unary T at wider widths, does not complete the second literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint second padding-or-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first padding-or-unary-opportunity predecessor with the represented-width unary evaluator, the same reviewed 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5524 plus the twenty-two inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width T-appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the second unary T of the second literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 2)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 9. Direct lookup and the specification cursor prove the following slot is again padding at width one and the third unary T at wider widths. The external compiled bound evaluates to BuilderSecondConstraintPaddingOrUnaryOpportunityStep.rawTimeBound + 624 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, predecessor-unlaunched-endpoint, and | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint first literal: padding with no emitted token at width one or the second unary T of the second literal at wider widths. It observes but does not consume the following padding opportunity at width one or third unary T at wider widths, does not complete the second literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint third padding-or-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second padding-or-unary-opportunity predecessor with the represented-width unary evaluator, the same reviewed 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5644 plus the twenty-four inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width T-appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the third unary T of the second literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 3)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 10. Direct lookup and the specification cursor prove the following slot is again padding at width one and the fourth unary T at wider widths. The external compiled bound evaluates to BuilderSecondConstraintSecondPaddingOrUnaryOpportunityStep.rawTimeBound + 636 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, predecessor-unlaunched-endpoint, and | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint first literal: padding with no emitted token at width one or the third unary T of the second literal at wider widths. It observes but does not consume the following padding opportunity at width one or fourth unary T at wider widths, does not complete the second literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint fourth padding-or-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third padding-or-unary-opportunity predecessor with the represented-width unary evaluator, the same reviewed 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5764 plus the twenty-six inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width T-appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the fourth unary T of the second literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 4)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 11. Direct lookup and the specification cursor prove the following slot is padding at width one and the terminating F at wider widths. The external compiled bound evaluates to BuilderSecondConstraintThirdPaddingOrUnaryOpportunityStep.rawTimeBound + 648 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, non-timeout, predecessor-unlaunched-endpoint, and | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint first literal: padding with no emitted token at width one or the fourth unary T of the second literal at wider widths. It observes but does not consume the following padding opportunity at width one or terminating F at wider widths, does not complete the second literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint fifth padding-or-terminator opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete fourth padding-or-unary-opportunity predecessor with the represented-width unary evaluator, a new reviewed 93-rule finite optional-terminator table over the audited token-appender rules, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 5884 plus the twenty-eight inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width F-appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the terminating F of the second literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 5)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 12. Direct lookup and the specification cursor prove the following slot is padding at width one and the opening unary T of the following literal at wider widths. The external compiled bound evaluates to BuilderSecondConstraintFourthPaddingOrUnaryOpportunityStep.rawTimeBound + 660 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint first literal: padding with no emitted token at width one or the terminating F of the second literal at wider widths. It observes but does not consume the following padding opportunity at width one or opening unary T at wider widths, does not complete the following literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint sixth padding-or-opening-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete fifth padding-or-terminator-opportunity predecessor with the represented-width unary evaluator, the reviewed 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 6004 plus the thirty inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width opening-T appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the opening positive T of the following literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 6)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 13. Direct lookup and the specification cursor prove the following slot is padding at width one and the first unary-index T of the following literal at wider widths. The external compiled bound evaluates to BuilderSecondConstraintFifthPaddingOrTerminatorOpportunityStep.rawTimeBound + 672 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact, bounded, blank-equivalent, accepting, | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint second literal: padding with no emitted token at width one or the opening positive T of the following literal at wider widths. It observes but does not consume the following padding opportunity at width one or first unary-index T at wider widths, does not complete the following literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Cook-Levin second-constraint seventh padding-or-unary opportunity step | formalized-foundation-only | For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete sixth padding-or-opening-unary-opportunity predecessor with the represented-width unary evaluator, the reviewed 93-rule finite optional-appender table, and the retained-coordinate unary evaluator through three total nine-symbol WorkChain bridges. The global table has exactly 6124 plus the thirty-two inherited/generated unary-evaluator rule counts. Every raw input follows exact predecessor, width-evaluator, width-one skip or wider-width first-unary-index-T appender, retained-coordinate, bridge, suffix, and combined traces. At tapeWidth one it consumes padding and emits no token; at every wider width it appends exactly the first unary-index T of the following literal. The output equals encodedFormula.take (2 * (FormulaWidth + 43 + if tapeWidth = 1 then 0 else 7)) and the retained coordinate is FormulaVariableSlotBound + 1 + FormulaClauseSlotsPerConstraint * FormulaTokensPerClause + 14. Direct lookup and the specification cursor prove the following slot is padding at width one and the second unary-index T of the following literal at wider widths. The external compiled bound evaluates to BuilderSecondConstraintSixthPaddingOrOpeningUnaryOpportunityStep.rawTimeBound + 684 + 24 * n + 12 * FormulaWidth + 12 * width + 12 * widthRootPrefixLength + 6 * widthWorkSteps + 6 * targetWorkSteps. Compiled exact bounded | This milestone consumes exactly one additional width-selected schedule opportunity after the second scheduled constraint following literal opening: padding with no emitted token at width one or the first unary-index T of the following literal at wider widths. It observes but does not consume the following padding opportunity at width one or second unary-index T at wider widths, does not complete the following literal or traverse the remainder of the second constraint, does not implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Typed direct-wire NAND semantics | formalized | Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics. | This does not establish circuit minimization, SAT, or P = NP. |
| Finite enumeration, equivalence, and reference minimum | formalized | Exhaustive finite Boolean direct-wire search under the empty-profile reference model. | No polynomial-runtime result is obtained from this exhaustive reference search. |
| Concrete framed replacement and slack | formalized | The concrete serial framed context with support outputs and explicit bypass wires. | This is not an arbitrary-support replacement theorem for the report's global family. |
| Locked-NAND local candidates and baseline accounting | formalized | Typed local macro candidates, source-derived counts, and five finite local square baselines. | Local minima are not a global BaselineDistinct or locked-NAND threshold theorem. |
| Locked-NAND global carrier and trace equivalence | formalized | Exact X/T/O/R/L/z carrier separation and both trace-equivalence directions for arbitrary finite topological NAND circuits. | Carrier/trace equivalence does not assemble the complete exposed candidates, prove cross-instance BaselineDistinct or final-output laws, construct the uniform polynomial builder, or establish the locked-NAND threshold. |
| Locked-NAND global baseline and four-gate candidate assembly | formalized | Exact source-derived B/B baseline and B+4/B+1 extension for arbitrary finite topological NAND circuits. | Candidate assembly alone does not prove global BaselineDistinct, either conditional final-output branch law, the locked-NAND threshold, residual slack at most four, or a uniform polynomial bitstring builder. |
| Locked-NAND global baseline distinctness | formalized | All exposed baseline outputs are nonconstant, nonprojections, pairwise semantically distinct, and have exact exhaustive reference minimum B for arbitrary finite topological NAND circuits. | BaselineDistinct does not prove either whole-carrier final-output branch law, instantiate the complete threshold premise package, establish the locked-NAND threshold or global residual-slack bound, or construct the uniform polynomial bitstring builder. |
| Locked-NAND global unsatisfiable final-zero branch | formalized | For every finite topologically ordered NAND circuit, unsatisfiability makes the full final coordinate identically false on the whole carrier and fixes the exhaustive reference minimum at B. | This does not prove satisfiable FinalLockSeparation, instantiate the complete threshold package, establish the global locked-NAND threshold or residual-slack bound, or construct the uniform polynomial builder. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Locked-NAND satisfiable separation and typed semantic threshold | formalized | For every finite topologically ordered NAND circuit, one answer-independent full candidate instantiates all six semantic premises, has residual slack at most four, and crosses the exact source-derived minimum threshold exactly when the source circuit is satisfiable. | This typed semantic theorem does not construct or compile the report's encoded polynomial-time SAT-to-locked-NAND builder, establish CNFSAT in P, prove NP-hardness transport, discharge the abstract locked-NAND threshold axiom, or prove P = NP. |
| Encoded locked-NAND semantic boundary | formalized-semantic-boundary | A strict version-zero bit grammar round-trips normalized NAND circuits and complete locked-NAND candidates; the pure all-bitstring transformation is fail-closed and preserves source satisfiability at the exact target threshold. | This is not a parser/validator machine, emitter machine, RawRefinement, PolynomialReduction, construction-runtime or output-size bound, abstract locked-NAND threshold discharge, CNFSAT-in-P result, or P = NP. |
| Concrete strict-v0 locked-NAND source parser | formalized-foundation-only | One literal nine-symbol finite work machine validates every strict version-zero source bitstring: it accepts exactly ValidEncodedCircuit, preserves valid bytes, clears invalid bytes, cannot time out within the proved compiled cubic bound, and supplies polynomial-time machine/function witnesses plus the validator's exact leaf RawRefinement. | This source parser alone does not emit the locked-NAND target or establish the source-to-target PolynomialReduction. The downstream emitter now supplies its own runtime/output bounds and strict composition, but the abstract locked-NAND threshold assumption, CNFSAT-in-P result, NP-hardness or NP-completeness transport, and P = NP remain absent. |
| Concrete strict-v0 locked-NAND target emitter | formalized-foundation-only | One literal 1,387,921-rule grammar-only controller emits the exact direct locked-NAND target on every grammar-decoded circuit, rejects malformed grammar with empty output, cannot time out within an explicit all-input polynomial, has an explicit quadratic output-size bound, and supplies compiled polynomial-time machine/function witnesses, exact leaf RawRefinement, and strict parser/emitter composition computing buildLocked-NANDInstance. | The standalone emitter intentionally accepts every grammar-decoded raw circuit, including intrinsically invalid references; strict fail-closed semantics come from parser composition. The standalone emitter does not itself package the language equivalence as PolynomialReduction; the downstream concrete reduction milestone now does. The abstract locked-NAND threshold assumption, CNFSAT-in-P result, NP-hardness transport, and P = NP remain absent. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete strict-v0 locked-NAND polynomial reduction | formalized-polynomial-reduction | The existing strict parser/emitter composition is packaged as a concrete polynomial many-one reduction from EncodedNANDSAT to EncodedLocked-NANDThreshold, with exact function identity, exact output, all-bitstring language equivalence, a ReducesTo witness, and recursive raw-machine refinement. | The downstream all-input CNF compiler now identifies CNFSAT with this concrete source language through a fixed finite machine and a direct polynomial reduction, then composes with this reduction. This milestone does not discharge the abstract target-language assumption, prove the report-level locked-NAND threshold theorem, put CNFSAT in P, complete residual minimization or ZeroSlack, or prove P = NP. |
| Concrete CNF-to-NAND semantic compiler | formalized-semantic-boundary | A total answer-independent compiler transforms every strict canonical CNF formula into an intrinsically topological well-formed NAND circuit, preserves satisfiability exactly, proves the exact gate count and a quadratic serialized-output bound, fails closed on every malformed bitstring, and composes semantically with the concrete locked-NAND threshold builder. | This milestone is the pure semantic and size-bound layer; the subsequent all-input milestone supplies the finite-machine, PolynomialTimeFunction, RawRefinement, and PolynomialReduction interfaces. Neither layer decides CNF-SAT, proves CNFSAT is in deterministic polynomial time, discharges the abstract report-level locked-NAND premise, completes ZeroSlack/PCCMin, or proves P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete all-input CNF-to-NAND polynomial reduction | formalized-polynomial-reduction | One fixed 135,070-rule three-node parser/carrier/controller work graph halts on every bitstring, rejects malformed CNF words with empty output, emits exactly compileEncodedCNFToNAND on every valid source, has one external encoded-input polynomial, compiles to a non-timeout PolynomialTimeFunction, retains literal RawRefinement, packages a direct PolynomialReduction from CNFSAT to EncodedNANDSAT, and composes it with the strict locked-NAND reduction to EncodedLockedNANDThreshold. | This syntax-directed compiler does not itself decide CNF-SAT, put CNFSAT in deterministic polynomial time, establish SAT NP-hardness or CNFSAT NP-completeness, connect the concrete locked-NAND target to the abstract report-level threshold theorem, complete residual minimization or ZeroSlack/PCCMin, discharge any project assumption, or prove P = NP. |
| Conditional locked-NAND threshold boundary | formalized-with-premises | A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition. | The premises are not instantiated by a uniform SAT-to-locked-NAND builder. |
| Fail-closed explicit-list residual routes | formalized-explicit-list-only | Executable strict-gain search over a caller-supplied finite implementation list. | Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack. |
| Universal verified residual-gain chain bound | formalized-iteration-bound-only | Every finite proof-bearing or executably verified chain of adjacent strict equivalent gains preserves semantics and the exhaustive reference minimum, while its endpoint residual slack plus its length is at most its starting residual slack. For the complete locked-NAND candidate, the existing residual-slack-at-most-four theorem specializes this to at most four verified gain steps. | This milestone bounds only a disclosed, independently verified sequence. It does not find the next gain, prove route or candidate-list completeness, justify stopping after fewer than the bound, construct ZeroSlack, compute an exact minimizer, establish polynomial checker or PCCMin runtime, put SAT in P, discharge a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Global strict-gain stopping specification | formalized-semantic-stopping-only | For every finite direct-wire implementation, positive exhaustive-reference residual slack is equivalent to the existence of some strictly smaller semantically equivalent implementation; zero slack and semantic minimality are each equivalent to global absence of such an implementation. A verified chain endpoint with separately proved global no-gain evidence therefore has zero slack and packages an exact minimum result. | This is a semantic stopping criterion, not a stopping algorithm. It uses the exhaustive reference minimum as a mathematical witness and requires a proof quantifying over every finite implementation at the endpoint. It does not derive global absence from a finite scan, generate a route, prove candidate-list or route completeness, construct the manuscript's ZeroSlack certificate, establish polynomial checking or PCCMin runtime, put SAT in P, discharge a project assumption, or prove P = NP. |
| Terminal full-carrier residual bridge | formalized-terminal-full-mode-semantic-bridge | For every finite direct-wire implementation, terminalization preserves the exact whole implementation, gate count, and semantics at every input/output coordinate. An independently stated terminal minimum is attained, universally lower-bounds every complete terminal realization, and equals the exhaustive semantic reference minimum. Positive residual slack is equivalent to a cheaper whole-span full realization, every such realization gives strict residual descent, and zero slack is equivalent to absence of one. | This is the direct-wire terminal full-mode specialization of the manuscript bridge. It does not formalize the quotient carrier or quotient-to-full firewall, proper or governed supports, SaturatePositive, BCEL/BN2-BN6, packet or selector completeness, route generation, the ZeroSlack certificate, PCCMin, polynomial minimum search or checking, SAT in P, discharge a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal quotient/full mode firewall | formalized-terminal-mode-firewall | For every finite direct-wire implementation, a computed finite profile observer records the ten terminal carrier roles and an explicit forgetful projection selects the quotient coordinates. Projection retains the exact implementation, gate count, and complete multi-output Boolean semantics. A quotient comparison has a checked full lift exactly when every forgotten profile coordinate agrees, lossless projections lift directly, and obligation discharge transports across a checked lift. | This is a terminal comparison/lifting firewall only. It supplies no proper or governed supports, arbitrary quotient construction, support or projection-defect minimum, saturation, Package E, BCEL/BN2-BN6, packet or selector completeness, global residual route, ZeroSlack certificate, PCCMin exactness or polynomial runtime, SAT-in-P result, discharged project assumption, or proof that P = NP. |
| Terminal full/quotient projection minima | formalized-terminal-projection-minimum | For every finite direct-wire implementation, computed finite terminal-profile observer, and explicit forgetful projection, complete enumeration through the current gate count computes an attained full-profile minimum and an attained quotient-profile minimum. Both minima universally lower-bound every matching realization, forgetting coordinates cannot increase the minimum, the full minimum decomposes as the quotient minimum plus a nonnegative projection defect, and that defect is zero exactly when an attained quotient minimum has a checked full lift. | These are exhaustive finite reference minima through the supplied implementation size. This milestone proves no polynomial runtime, proper or governed support construction, arbitrary manuscript quotient carrier, SaturatePositive, Package E, BCEL/BN2-BN6, complete residual routing, ZeroSlack certificate, PCCMin exactness, SAT-in-P result, discharged project assumption, or proof that P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal projection transfer identity | formalized-terminal-projection-transfer | For every finite direct-wire four-corner terminal-profile family sharing one computed observer and one explicit projection, signed full and quotient minimum deltas obey the exact Section 5.2 transfer identity. The projection excess is the quotient delta minus the full delta; if meet and both side defects are zero while the join defect is D, the excess equals D and is positive whenever D is positive. | This is signed arithmetic over four supplied corners. It does not construct or certify a proper governed support square, prove SaturatePositive, discharge Package E or BCEL/BN2-BN6, generate a complete residual route, prove ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Terminal saturation closure | formalized-terminal-saturation-closure | For every finite terminal primitive-record universe and every explicit Boolean dependency system tagged by the manuscript's ten closure mechanisms, the generated reflexive transitive closure contains the seed, is dependency-closed, is least among closed supersets, is monotone and idempotent, and has exactly the closed supports as fixed points. | This closure theorem does not derive the dependency relation from an arbitrary circuit, construct proper support, prove support completion or square legitimacy, instantiate a projection-compatible square, prove SaturatePositive or BCELReady, discharge Package E or BCEL/BN2-BN6, generate a complete residual route, prove ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Terminal executable and physical support completion | formalized-terminal-physical-support-completion | For every finite direct-wire candidate, explicit terminal dependency system, and finite seed list, a deterministic finite work list computes exactly the inductive saturation, then the actual program computes canonically ordered incoming boundary and outgoing interface wires. Lean proves no crossing wire is omitted or added and the composed physical support is compatible. | The terminal dependency system remains explicit data rather than an extracted profile frontier. This milestone does not construct proper positive support, prove support completion in the manuscript's full sense or square legitimacy, instantiate the required projection square, prove SaturatePositive, discharge Package E or BCEL/BN2-BN6, generate a complete residual route, prove ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Arbitrary terminal support extraction | formalized-terminal-support-extraction | For every finite direct-wire candidate and finite terminal record list, including noncontiguous selections, the actual program is structurally extracted over its exact canonical incoming boundary and ordered outgoing interface. The extracted candidate equals an independently defined open-support function for every boundary valuation and recovers the original interface values on whole-circuit-induced boundaries; the construction also composes with executable terminal saturation. | The record list and terminal dependency system remain explicit inputs rather than the manuscript's derived profile frontier. This milestone does not construct a proper positive support, prove full governed support completion or square legitimacy, instantiate the required projection square, prove SaturatePositive, discharge Package E or BCEL/BN2-BN6, generate a complete residual route, prove ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Governed proper-positive terminal support search | formalized-governed-proper-positive-support-search | For every finite direct-wire candidate and explicit terminal dependency system, Lean enumerates the complete canonical finite universe of primitive-record seeds, saturates and physically completes each seed, extracts its exact open support, and computes exact local gain from the exhaustive semantic reference minimum. The search returns a proof-bearing nonempty proper support with positive gain whenever one exists in that canonical seed universe, and its none result is equivalent to the absence of such a seed. | The terminal dependency system remains explicit input rather than a profile frontier derived from the circuit, and the search is exhaustive reference computation rather than a polynomial algorithm. This milestone does not prove global gain completeness, full manuscript support completion or square legitimacy, instantiate a projection-compatible square, prove SaturatePositive or BCELReady, discharge Package E or BCEL/BN2-BN6, generate a complete residual route, prove ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Saturated terminal support-square closure | formalized-terminal-saturated-support-square-closure | For every finite direct-wire candidate, every explicit terminal dependency system, and every pair of finite terminal seeds, Lean computes saturated left and right corners, their canonical closed meet, and their closed saturated-union join. It proves the exact greatest-lower-bound and least-upper-bound laws, seed extensionality, computed physical compatibility, exact gate count, open-support semantics, and induced whole-circuit recovery for all four corners. | The terminal dependency system remains explicit input rather than a profile frontier derived from the circuit. This milestone proves finite closed-corner algebra and computed physical extraction, not the manuscript's obstruction routing, frontier pushout, projection-compatible square, side-tight four-corner minima, BN2 square legitimacy, SaturatePositive, Package E, BCEL/BN2-BN6, complete residual routing, ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |
| Governed terminal support completion | formalized-terminal-governed-support-completion | For every finite direct-wire candidate, explicit terminal dependency system, finite seed list, and computed saturated support-square corner, Lean computes the exact physical boundary, ordered interface, and partition of selected profile coordinates among all ten terminal profile roles. It proves exact membership, no duplicates, pairwise disjointness, record coverage, dependency closure, physical compatibility, and retention of each exact corner. | The terminal dependency system remains explicit input rather than a profile frontier derived from the circuit. This milestone computes a governed finite completion of each saturated support-square corner, not the manuscript's obstruction routing, frontier pushout, projection-compatible square, side-tight four-corner minima, BN2 square legitimacy, SaturatePositive, Package E, BCEL/BN2-BN6, complete residual routing, ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Governed terminal frontier pushout | formalized-terminal-governed-frontier-pushout | For every finite direct-wire candidate, explicit terminal dependency system, and computed saturated support square, Lean constructs the governed boundary, interface, and role-preserving profile pushout from the two side completions alone. The independently completed join frontier equals that gluing, the meet profile is the exact shared overlap, and every side physical coordinate is either retained externally or witnessed as internalized. | The terminal dependency system remains explicit input rather than a profile frontier derived from the circuit. This milestone proves exact frontier gluing for computed saturated support squares, not projection compatibility, side-tight four-corner minima, BN2 square legitimacy, SaturatePositive, Package E, BCEL/BN2-BN6, complete obstruction routing, ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |
| Governed terminal projection square | formalized-terminal-governed-projection-square | For every finite direct-wire candidate, explicit terminal dependency system, computed saturated support square, and every forgetful terminal projection, Lean retains the exact physical frontier, filters all ten role profiles exactly, proves projected meet is the shared side overlap, and proves projected join is the side-only projected pushout without reading the join corner. | The terminal dependency system remains explicit input rather than a profile frontier derived from the circuit. This milestone proves structural projection commutation for computed saturated support squares, not side-tight four-corner minima, BN2 square legitimacy, SaturatePositive, Package E, BCEL/BN2-BN6, complete obstruction routing, ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |
| Side-tight four-corner minimum arithmetic | formalized-residual-terminal-side-tight-minimum-arithmetic | For every finite terminal projection four-corner family and every independently attained typed full or quotient basis, Lean proves componentwise minimum bounds and the exact signed four-slack identity. A fail-closed Boolean and Option gate returns the corresponding existing delta only when meet, left, right, and join all attain their exact minima; both canonical independently attained minimum bases pass. | The canonical corner minima are independently attained. This milestone proves numerical arithmetic and fail-closed exactness, not construction of one coherent four-corner basis, coherent completion, maximization over a finite tight family, BN2 square legitimacy, SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked four-corner carrier transport | formalized-residual-terminal-four-corner-carrier-transport | For every finite computed saturated terminal support square, direct-wire candidate, and forgetful terminal projection, Lean derives all four exact governed and extracted endpoints in common ambient coordinates, proves duplicate-free boundary, interface, and profile lists, transports meet and join profiles exactly, and classifies each present side physical coordinate as identically retained or constructively internalized through fail-closed queries. | This milestone supplies a checked common ambient carrier for the computed square. It does not transport four optimum realizers, prove the full four-corner optimum carrier-compatibility obligation, construct a coherent four-corner optimum, prove side-tight completion or BN2 square legitimacy, establish SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |
| Four-corner optimum carrier compatibility | formalized-residual-terminal-four-corner-optimum-carrier-compatibility | For every finite computed saturated terminal support square and every explicit observer, Lean embeds all four exact corner candidates into one common ambient carrier, proves reversible semantic and gate-count preservation, proves exact ambient and corner reference minima agree, and localizes canonical full and quotient optima from one shared observer and projection without changing their exact minimum counts. | This milestone compares independently attained full and quotient optima on one reversible common carrier. It does not prove coherent transport along the square legs, construct a coherent four-corner optimum, prove sideTightCompletionExists or BN2 square legitimacy, derive the terminal dependency system, establish SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |
| Four-corner optimum coherence dichotomy | formalized-residual-terminal-four-corner-optimum-coherence-dichotomy | For every finite computed terminal support square, every explicit observer, every terminal projection, and either full or quotient coherence mode, Lean checks the four square legs in a deterministic order and returns either one coherent canonical optimum tuple with exact transport, side-tight, and incidence facts or the exact deterministic first failure. | This milestone classifies coherent transport or its exact first failure. It does not prove that every square is coherent, construct the later no-outcome route, prove sideTightCompletionExists or BN2 square legitimacy, establish SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, removal of a project assumption, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Four-corner side-tight completion under local route silence | formalized-residual-terminal-four-corner-side-tight-completion-under-local-route-silence | For every finite computed terminal support square, every explicit observer, and either full or quotient coherence mode, the exact first local coherence query returns a proof-bearing sound route or, under computed local route silence, Lean supplies the complete checked side-tight coherent optimum tuple with exact minimum incidence value while retaining the separate quotient-promotion firewall. | This milestone closes only the local completion edge under computed local route silence. It does not prove universal route silence, connect a local obstruction to the complete global no-outcome route system, prove BN2 square legitimacy, derive the terminal dependency system, enumerate or maximize the complete tight-basis family, establish SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, remove a project assumption, or prove P = NP. |
| Complete four-corner BN2 tight-basis maximum | formalized-residual-terminal-four-corner-complete-tight-basis-maximum | For every finite computed terminal support square, every explicit observer, and either full or quotient mode, Lean enumerates the complete finite tight-basis family, retains every exact profile-constrained minimum implementation at each corner, filters the full Cartesian product with the arbitrary-family coherence query, and proves under exact local route silence that the signed maximum equals the selected delta. | This milestone closes the remaining local BN2 tight-basis maximum under computed local route silence. It does not prove universal route silence, connect a local obstruction to the complete global no-outcome route system, prove BN2 square legitimacy, derive the terminal dependency system, establish SaturatePositive, Package E, BCELReady or BCEL/BN2-BN6, complete obstruction routing, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed terminal BN2 square legitimacy | formalized-residual-terminal-computed-bn2-square-legitimacy | For every finite computed terminal support square built from two finite seeds under one explicit terminal dependency system, direct-wire candidate, observer, and forgetful projection, Lean constructs the exact compatible governed frontier and projection square, keeps full and quotient minimum quantities on the same carrier, and returns either the complete local conclusion under exact local route silence or the deterministic full-then-quotient proof-bearing first coherence route. | This milestone packages computed structural legitimacy and the exact local no-route conclusion. It does not derive the terminal dependency system from an arbitrary circuit, prove universal route silence, connect a local failure to the complete global no-outcome route system, identify a BCEL anchor square, establish SaturatePositive, Package E, BCELReady or later BCEL/BN2-BN6 conclusions, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, remove a project assumption, or prove P = NP. |
| Computed terminal BCEL anchor nucleus and cut-square dichotomy | formalized-residual-terminal-computed-bcel-anchor-nucleus | For every finite direct-wire candidate, explicit terminal dependency system, computed governed proper-positive support, forgetful projection, executable ambient observer, and positive whole-support projection defect, Lean computes the canonical minimum-cardinality positive anchor nucleus and returns either an insufficient nucleus, the exact first anchor-algebra mismatch, the exact first proper-cut defect mismatch, the proof-bearing first full-before-quotient local route, or exact constant-cut and local BN2 conclusions for every proper cut. | This milestone assumes a positive whole-support projection defect and an explicit terminal dependency system. It does not derive either premise, identify manuscript activation or charge equivalence classes absent from the terminal model, connect a local failure to the complete global no-outcome route system, establish SaturatePositive, Package E, BCELReady or later BCEL/BN2-BN6 conclusions, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed terminal saturation-positivity firewall | formalized-residual-terminal-saturation-positivity-firewall | For every finite direct-wire candidate, explicit terminal dependency system, computed governed proper-positive support, forgetful projection, and executable ambient observer, Lean computes the whole-support defect: zero projection defect returns an attained quotient minimum with a checked full lift, while positive defect delegates exactly to the existing fail-closed BCEL anchor-nucleus classifier. | This closes only projectionPositivityNotLostSilently in the current finite terminal model. It assumes an explicit terminal dependency system and an already computed governed proper-positive support. It does not discharge transparentSaturationCostBalanced, interfaceExposureRoutesToE, originKernelObligationClosureRouted, or firstNontransparentStepRecorded; establish full SaturatePositive, Package E, BCELReady or later BCEL/BN2-BN6 conclusions; prove ZeroSlack, PCCMin, polynomial runtime, SAT in P; remove a project assumption; or prove P = NP. |
| Candidate-derived terminal saturation cost balance | formalized-residual-terminal-candidate-saturation-cost-balance | For every finite direct-wire candidate, executable ambient observer, forgetful projection, and finite terminal seed, Lean computes the candidate-derived dependency system and deterministic rule-labelled saturation trace, then returns proof that every event is exactly cost-balanced with preserved full slack and nondecreasing projection defect, or records the exact first nontransparent event and complete transparent prefix. | This closes only the finite terminal forms of transparentSaturationCostBalanced and firstNontransparentStepRecorded. The executable observer and forgetful projection remain explicit model inputs, and a nontransparent event is recorded rather than routed. It does not discharge interfaceExposureRoutesToE or originKernelObligationClosureRouted; establish full SaturatePositive, Package E, BCELReady or later BCEL/BN2-BN6 conclusions; prove ZeroSlack, PCCMin, polynomial runtime, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite terminal interface-exposure routing | formalized-residual-terminal-interface-exposure-routing | For every finite direct-wire candidate, executable ambient observer, forgetful projection, and finite terminal seed, Lean recognizes only an exact candidate-derived interface-consumer edge. Each recognized event is transparently cost-balanced or produces a proof-bearing local E-route; the production trace result records the exact first nontransparent event and complete transparent prefix, while non-interface first failures remain fail-closed. | This closes only the finite local form of interfaceExposureRoutesToE. The proof-bearing local E-route is an exposure-obligation coordinate, not a full Package E VerifyDW acceptance, a verified global gain, or global route completeness. The executable observer and forgetful projection remain explicit model inputs. It does not discharge originKernelObligationClosureRouted; establish full SaturatePositive, Package E, BCELReady or later BCEL/BN2-BN6 conclusions; prove ZeroSlack, PCCMin, polynomial runtime, SAT in P; remove a project assumption; or prove P = NP. |
| Finite terminal SaturatePositive composition | formalized-residual-terminal-finite-saturate-positive-composition | For every finite direct-wire candidate, executable ambient observer, forgetful projection, and proof-bearing candidate BCEL anchor problem whose normalized seed has positive full slack, Lean recognizes exact candidate-derived origin, kernel, and obligation closures in both gate/profile orientations; checks cost transparency, obligation discharge, and forgotten-profile stability; preserves positive full slack across an all-safe trace into the checked-lift or BCEL firewall; or returns the exact first interface, closure, or other fail-closed nontransparent route with its complete safe prefix. | This closes the finite local form of originKernelObligationClosureRouted and composes the five reconstructed terminal sub-obligations only for an explicit proof-bearing problem. A local route is not a complete global outcome, Package E VerifyDW acceptance, verified gain, or global route-completeness result. The positive initial full-slack premise remains explicit. It does not establish manuscript-wide SaturatePositive, BCELReady, RankWF, ZeroSlack, PCCMin, polynomial runtime, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked finite SaturatePositive-to-BCEL-ready composition | formalized-residual-terminal-finite-bcel-ready-composition | For every finite direct-wire candidate, executable terminal saturation model, and proof-bearing candidate BCEL anchor problem with explicit initial positive full slack, Lean reruns the recomputed finite SaturatePositive classifier and accepts only its positive-projection branch with a computed BCEL-ready anchor nucleus. The certificate retains the exact classifier equality, complete safe trace, positive final slack, positive whole-support defect, at-least-two anchor bound, exact constant-cut equation, and local full/quotient BN2 conclusion for every proper cut. | The terminal candidate, executable model, candidate-derived anchor problem, and initial positive full-slack premise remain explicit. Rejection of another finite branch is not a mapping into the complete global route system. This does not derive positivity from residual slack, construct BN3–BN6 data or a grouped family, derive constant activation, establish manuscript-wide SaturatePositive or BCELReady, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P; remove a project assumption; or prove P = NP. |
| Same-candidate finite BCEL-ready and Packet carrier coherence | formalized-residual-terminal-finite-bcel-packet-carrier-coherence | For every accepted finite Packet/budget no-lower certificate, Lean indexes a checked finite BCEL-ready problem by the same candidate and model, retains its computed ready nucleus, and uses a bijective anchor map with explicit injectivity and surjectivity proofs plus a Boolean-reflected exact list equality to identify that nucleus with the exact grouped Packet carrier. The inherited nontrivial-size bound, positive-Packet exclusion, and constant-activation exclusion therefore hold on one coherently bound family consumed by the report-facing ZeroSlack boundary. | The terminal problem, initial positive full-slack premise, grouped Packet family, bijective anchor map, Packet payloads, budget, typed realizer table, dependency table, and rank maps remain supplied proof-bearing inputs. The carrier identity does not identify Packet-family activation weights with proper-cut projection excess or derive constant activation from positive residual slack. It does not construct BN3–BN6 data from terminal data, complete the no-lower ledger, prove unconditional ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite BCEL/Packet activation-coherence obstruction | formalized-residual-terminal-finite-bcel-packet-activation-obstruction | For every arbitrary finite M184 carrier-coherence certificate, Lean computes the exact selected M183 nucleus defect and exhaustively checks the M180 Packet cut value plus every canonical nonempty proper Packet-cut activation weight. Acceptance reconstructs full finite activation coherence and identifies each mapped terminal cut's activation with its projection excess. The same-family Packet exclusion forces rejection, and a deterministic classifier returns either the declared cut-value mismatch or a proof-bearing proper-cut activation mismatch; the report-facing ZeroSlack record derives the same obstruction without duplicate family data or a caller flag. | The checker proves that the numerical bridge fails on the current accepted finite family; it does not prove activation coherence. The terminal problem, initial positive full-slack premise, grouped Packet family, bijective anchor map, activation cells and masses, payloads, budget, typed realizer table, dependency table, and rank maps remain supplied proof-bearing inputs. Exhaustive proper-cut enumeration may be exponential. This milestone does not derive the family or activation weights from positive residual slack, construct BN3–BN6 data from every valid input, complete the global route or no-lower systems, prove unconditional SaturatePositive, BCELReady, ZeroSlack or PCCMin, establish polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Residual terminal RankWF | formalized-residual-terminal-rank-wf | For the fixed manuscript residual rank of exactly ten natural coordinates in the stated witness-type, span-type, mode, frontier-defect, projection-defect, saturation-defect, anchor-count, charge-size, profile-size, canonical-code priority order, Lean provides the exact lexicographic proposition, an equivalent executable comparison, all ten priority witnesses, proof-bearing descent, accessibility, induction, and kernel-checked well-foundedness. | This establishes the fixed residual rank domain and RankWF only. It does not map the current finite terminal routes into the manuscript's complete global outcome system, prove that any existing route strictly decreases the rank, establish route completeness or Package E, remove the explicit positive premise from the finite composition, establish full manuscript-wide SaturatePositive or BCELReady, prove ZeroSlack, PCCMin, polynomial runtime, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Candidate-derived finite BN3 request envelope | formalized-residual-terminal-bn3-request-envelope | From every successful computed finite BCEL anchor nucleus, Lean uses one canonical duplicate-free primitive-record identity list across every proper cut; gives exact executable monotone request membership stable under extensional transport and exact singleton minimal consumers; accounts active incidences without duplicates; selects one canonical full or quotient side-tight coherent basis for every proper cut; and preserves all upstream proof-bearing classifier failures in a total outcome. | This establishes one exact candidate-derived finite BN3 envelope only after the existing computed BCEL anchor-nucleus classifier succeeds. Proper cuts are enumerated through all subsets, so the construction is exponential reference computation rather than a polynomial algorithm. It does not derive the terminal dependency system, map local routes into the manuscript's complete global outcome system, construct BN4-BN6, prove selector or realizer completeness, establish global ZeroSlack or PCCMin, prove SAT in P, remove a project assumption, or prove P = NP. |
| Finite BN4 activation-exact cancellation kernel | formalized-residual-terminal-bn4-activation-cancellation | After a successful computed finite BN3 envelope, Lean gives every request atom a canonical singleton activation code; proves activation-code equality exactly equivalent to equality of activation functions without enumerating cuts; checks equality of a complete typed key containing the atom, explicit semantic signature, and explicit transport type; totals positive and negative natural mass only at that same complete key; classifies a canonical balanced, positive, or negative residual; proves exact integer mass conservation, complete-key preservation, positive residual mass, and absence of opposite-sign residual pairs; computes duplicate-free ledger keys; and preserves all upstream failure branches while rejecting foreign request atoms. | This is a finite cancellation kernel over an explicit typed cell ledger. It does not derive cells, semantic signatures, or transport types from four-corner bases and is not the full historical BN4 theorem. It supplies no polynomial construction or size bound; does not construct PkgC or BN6; does not complete global routes, selectors, or realizers; does not establish ZeroSlack or PCCMin; does not put SAT in P; does not remove a project assumption; and does not prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite BN5 full-shadow localization kernel | formalized-residual-terminal-bn5-full-shadow-localization | For every explicit finite negative-unit refinement and quotient-shadow ledger, Lean preserves the complete exact-coordinate data, validates the negative mass refinement, computes whether the cut is silent, and otherwise returns either complete multiplicity coverage or a strict Hall deficit with a literal smaller shadow-neighbor fibre, complete-coordinate preservation, and a proof-bearing local X1 route that prevents active unmatched units from disappearing silently. | This kernel starts from explicit finite inputs: one complete BN4 key, a negative cancellation result, a payload list, a cut, and a quotient-shadow coordinate list. It does not derive payloads or shadows from four-corner bases, connect complete matching back to a BN4 contradiction, or prove the full CritC/Q/E/L/X2/X3/X4 diagnosis, so it is not the full historical BN5 theorem. It does not construct PkgC or BN6; complete global routes, selectors, or realizers; establish polynomial generation or runtime, ZeroSlack, or PCCMin; put SAT in P; remove a project assumption; or prove P = NP. |
| Finite PkgC separating-consumer restoration dichotomy | formalized-residual-terminal-pkgc-separating-consumers | For an arbitrary finite explicit minimal-consumer antichain, Lean canonically scans for the first disjoint pair that is not singleton-singleton. Absence proves exactly V54's singletonization premise. A found pair's atoms are canonically indexed into exact-coordinate quotient units and an explicit full-restoration universe is classified into complete multiplicity coverage or a strict Hall deficit with a deterministic local Q route. | The restoration coordinate universe remains explicit. This theorem does not derive consumers or restorations from a terminal candidate, connect complete coverage back to a BN4 or BN5 contradiction, embed the Hall route into the complete global outcome system, prove global route silence, or establish the full historical PkgC theorem. It does not prove full BN6 or Packet selector-realizer completeness, polynomial generation or runtime, ZeroSlack or PCCMin, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite PkgC typed restoration realization | formalized-residual-terminal-pkgc-typed-restoration | For an arbitrary finite explicit minimal-consumer antichain and a typed coordinate-preserving restoration operation, Lean materializes typed full-restoration candidates for every atom of the canonical first disjoint nonsingleton pair, proves exact candidate count and positional coordinate preservation, derives complete equality-fibre multiplicity coverage, excludes a strict Hall deficit for that graph, and otherwise proves V54 singletonization. | The typed restoration operation remains explicit caller data. This milestone does not construct it from a terminal candidate or prove its full semantic adequacy. It does not connect complete restoration to a BN4 or BN5 contradiction, embed local routes into the complete global outcome system, prove global PkgC route silence or the full historical PkgC theorem, establish full BN6 or Packet selector-realizer completeness, polynomial generation or runtime, ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Finite PkgC typed restoration same-key cancellation | formalized-residual-terminal-pkgc-same-key-cancellation | For an arbitrary finite explicit minimal-consumer antichain and typed exact-BN5-coordinate restoration operation, Lean mechanically pairs every quotient atom with its restored full candidate as opposite-sign unit cells, proves the complete BN5 coordinate gives the same nested BN4 key, proves exact cell count and positive/negative multiplicity equality at every BN4 key, computes an empty canonical residual and zero signed mass at every key, and derives V54 singletonization from exact absence of every such proof-bearing cancellation outcome. | The typed restoration operation and its complete coordinate maps remain explicit inputs, and the generated opposite-sign cells are not yet proved to be the cells of the terminal candidate's ambient BN4 ledger. This milestone does not construct semantic restorations from a terminal candidate, embed cancellation or Hall outcomes into the complete global route system, prove global route silence or the full historical PkgC theorem, establish full BN6 or Packet selector-realizer completeness, polynomial generation or runtime, ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite PkgC ambient BN4 ledger embedding | formalized-residual-terminal-pkgc-ambient-bn4-ledger | For arbitrary finite explicit BN4 cell ledgers, Lean proves that a proof-bearing exact multiset embedding identifies the generated PkgC opposite-sign cancellation ledger with an ambient subledger and preserves every duplicate. Positive and negative mass decompose at every complete key; removing the balanced generated subledger leaves the ambient signed mass and executable residual signed contribution exactly equal to an explicit remainder. A successful candidate-derived BN4 kernel additionally proves every embedded generated cell uses its canonical request-atom space, and complete bindings plus exact absence of every computed bridge imply V54 singletonization. | The ambient ledger, typed restoration operation, exact permutation certificate or canonical serialization, and successful candidate-derived BN4 kernel remain explicit proof-bearing inputs. This milestone does not derive the ambient ledger or restorer from a terminal candidate, prove the restorer's semantic adequacy, embed local cancellation or Hall outcomes into the complete global route system, prove global PkgC route silence or the full historical PkgC theorem, establish full BN6 or Packet selector-realizer completeness, polynomial generation or runtime, ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Finite PkgC ambient BN4 residual reduction | formalized-residual-terminal-pkgc-ambient-bn4-residual-reduction | For arbitrary finite explicit ambient BN4 ledgers, Lean proves that removing an exactly embedded balanced PkgC generated subledger preserves the executable residual cell at every complete key and the complete canonical executable residual ledger over the ambient key universe. Every remainder key occurs in that universe; a fail-closed canonical classifier constructs the exact reduction without caller-provided proof bits; and an empty remainder yields an empty ambient residual ledger, including for the existing candidate-derived computed bridge. | The ambient ledger, typed restoration operation, exact embedding, and explicit remainder remain proof-bearing inputs. This milestone does not derive those inputs from an arbitrary terminal candidate, prove that the remainder is empty or route-producing, establish restoration semantic adequacy or complete global route silence, reconstruct the full historical PkgC/BN6/Packet path, prove polynomial generation or runtime, establish ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite V54 consumer-antichain normal form | formalized-residual-terminal-v54-consumer-antichain-normal-form | For an arbitrary finite carrier and its explicit minimal-consumer antichain, Lean proves monotonicity and empty-request inactivity, proves that nonzero two-sided cut activation is equivalent to the existence of a disjoint consumer pair, and under the exact singletonized-disjoint-pair premise proves literal equality with the corresponding footprint cut indicator. | This finite kernel starts from an explicit minimal-consumer antichain and an explicit proof that every disjoint consumer pair is singletonized. It does not construct PkgC, derive that singletonization premise, or connect the footprint back to the full BN6 proof. It does not construct complete global routes, selectors, or realizers; establish polynomial generation or runtime, ZeroSlack, or PCCMin; put SAT in P; remove a project assumption; or prove P = NP. |
| Finite V53 constant-cut hypergraph rigidity | formalized-residual-terminal-v53-constant-cut-hypergraph-rigidity | For an arbitrary finite duplicate-free carrier and sparse nonnegative weighted hypergraph with positive listed cells, exact equality of every nonempty proper cut proves the complete V53 q=2, q=3, and q>=4 classification: full-span weight D; one common pair weight p with w_A + 2p = D; or zero weight on every proper footprint with full-span weight D. | This theorem consumes an explicit sparse positive hypergraph and an explicit proof that every nonempty proper cut has the same positive value. It does not construct PkgC, derive the hypergraph from a terminal candidate or the V54 consumer system, build BN6 cells or payloads, complete global routes, selectors, or realizers, establish polynomial generation or runtime, prove ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite BN6 grouped hypergraph packet bridge | formalized-residual-terminal-bn6-hypergraph-packet | For an arbitrary finite duplicate-free anchor carrier and explicit already-grouped family of positive payload-bearing survivor cells, V54 activation is transported exactly into the constructed V53 hypergraph cut sum. A positive BCEL constant-cut premise then yields the complete pair, mixed three-anchor balanced-triple/full-span, or four-or-more-anchor full-span classification with original payload witnesses. | This finite bridge consumes explicit exact footprint grouping, PkgC singletonization proofs, positive atom ledgers, payload data, and the BCEL constant-cut equation. It does not construct PkgC, derive or group survivors from a terminal candidate, establish full historical BN6 or Packet selector/realizer completeness, complete global routes, prove polynomial generation or runtime, ZeroSlack or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Finite Packet selector-seed extraction | formalized-residual-terminal-packet-selector-seeds | For an arbitrary finite exact BN6 packet conclusion, Lean extracts a carrier-contained payload-backed raw selector seed at the positive pair footprint, at every positive pair footprint of a balanced triple, or at the positive full-span footprint. The mixed three-anchor positive alternative is handled without asserting that both masses are positive, and the construction fixes no carrier cardinality. | This milestone consumes an exact finite BN6 packet conclusion and preserves only carrier containment, selector-relevant footprint size, and original grouped cell-and-atom payload evidence. It does not prove selector-universe membership, selector faithfulness or compatibility, construct a realizer or route, establish enumeration or polynomial generation/runtime, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Finite Packet payload-selector universe membership | formalized-residual-terminal-packet-selector-universe | For every arbitrary finite explicit grouped BN6 family, Lean enumerates the exact duplicate-free list of grouped footprints, proves membership equivalent to an original grouped cell at that footprint, and upgrades every payload-backed pair, balanced-triple pair, or full-span seed to membership in that same finite universe. | This finite universe is exactly the grouped-footprint list already present in an explicit BN6 family. It is not the manuscript's encoded or polynomial selector universe, and payload retention is not manuscript-level selector faithfulness. It does not prove selector compatibility, construct a realizer or route, derive the grouped family from a terminal candidate, establish polynomial enumeration or size bounds, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Canonical finite Packet selector handles | formalized-residual-terminal-packet-selector-handles | For every arbitrary finite explicit grouped BN6 family, Lean defines canonical indexed grouped-footprint handles, proves exact decoding is injective and every payload selector has a unique handle, retains original payload evidence, and proves every decoded footprint remains carrier-contained and has length at least two across the pair, balanced-triple, and full-span Packet branches. | These handles are input-relative list positions, not the manuscript's bit encoding or a polynomially enumerable selector universe. The milestone does not prove manuscript-level selector faithfulness or compatibility, construct a selector realizer or route, derive or group BN6 survivors from a terminal candidate, establish polynomial encoding length or runtime, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical fail-closed Packet selector-handle codec | formalized-residual-terminal-packet-selector-codec | For every arbitrary finite explicit grouped BN6 family, Lean gives each canonical input-relative selector handle a unary bitstring with fail-closed total decoding of missing delimiters, trailing data, and out-of-range indices, proves exact round trip, injectivity, canonical successful decoding, exact and explicit-universe-bounded length, retains payload, carrier, size, cell, and atom evidence, and gives every payload selector one unique accepted code across the pair, balanced-triple, and full-span Packet branches. | The code-length bound is relative to the explicit grouped-family list and does not bound that list by encoded circuit size. This milestone does not prove polynomial enumeration or runtime, encode atom or payload data, prove manuscript-level selector faithfulness or compatibility, construct a selector realizer or route, derive or group BN6 survivors from a terminal candidate, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Total fail-closed Packet source-payload realization | formalized-residual-terminal-packet-selector-payload-realization | For every arbitrary finite explicit grouped BN6 family, Lean defines a total fail-closed function that maps each accepted canonical selector code to its exact decoded handle, original source cell, decoded footprint, and a canonical original positive payload atom. Successful results re-encode to the exact input, remain in the supplied family, retain strict atom positivity, are equivalent to the finite payload-selector predicate, and preserve the pair, balanced-triple, and full-span Packet branches. | This is source-payload materialization relative to a supplied explicit grouped family, not the manuscript's gain-or-blocker selector realizer. The unary code still encodes only a list position and does not serialize atom or payload data. This milestone does not construct a replacement circuit, prove selector faithfulness or compatibility, return a gain or typed blocker route, derive or group BN6 survivors from a terminal candidate, bound the selector family by encoded circuit size, prove polynomial generation or runtime, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet selector candidate-gain scan | formalized-residual-terminal-packet-selector-gain-scan | For every arbitrary finite explicit grouped BN6 family whose payloads are direct-wire implementations, Lean decodes each accepted canonical Packet selector, scans every original candidate payload in the exact selected source cell with the executable strict-equivalent-gain checker, and returns only a genuine source-atom StrictEquivalentGain or proof that the selected cell has no such candidate. Every gain strictly decreases residual slack, decoder rejection is exact, and the pair, balanced-triple, and full-span Packet alternatives are preserved. | The candidate implementations and grouped BN6 family remain explicit input data. A local no-gain result excludes only payload candidates in one selected source cell; it is not a manuscript BotHN, BotBUD, or lower-rank BotSeed and does not imply global minimality or ZeroSlack. This milestone does not construct replacement candidates, prove selector faithfulness or compatibility, connect payload mass to charge surplus, derive or group survivors from a terminal candidate, bound the selector family by encoded circuit size, prove polynomial generation or runtime, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |
| Exhaustive Packet selector-universe gain scan | formalized-residual-terminal-packet-selector-universe-gain-scan | For every arbitrary finite explicit grouped BN6 family whose payloads are direct-wire implementations, Lean enumerates every canonical input-relative selector handle, scans every original candidate payload in each exact source cell with the executable strict-equivalent-gain checker, and returns only a canonical source-atom StrictEquivalentGain or proof that the complete supplied selector universe has no such candidate. Every gain retains a canonical accepted code and strictly decreases residual slack, while the pair, balanced-triple, and full-span Packet alternatives are preserved literally. | The grouped BN6 family and candidate implementations remain explicit inputs. Family-wide no-gain is silence only for that supplied input-relative selector universe; it is not a manuscript BotHN, BotBUD, or lower-rank BotSeed, does not establish selector faithfulness or compatibility, and does not imply global minimality or ZeroSlack. This milestone does not construct replacement candidates, connect payload mass to charge surplus, derive or group survivors from terminal data, bound the family by encoded circuit size, prove polynomial enumeration or runtime, complete PkgC, ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Conditional Packet selector gain-or-ZeroSlack coverage | formalized-residual-terminal-packet-selector-gain-coverage | For every arbitrary finite explicit grouped BN6 family whose payloads are direct-wire implementations, an explicit proof-bearing coverage certificate requires every strict equivalent gain from the current implementation to occur as an original payload atom in an exact canonical selector source cell. Under precisely that premise, the exhaustive scan returns either a source-atom gain with strict residual descent or a proof-bearing semantic minimum and zero residual slack, while preserving the pair, balanced-triple, and full-span Packet alternatives literally. | The explicit gain-coverage certificate, grouped BN6 family, and candidate implementations remain inputs. This milestone does not construct the coverage certificate from terminal data, prove selector faithfulness or compatibility, construct or polynomially enumerate replacement candidates, establish encoded-size or runtime bounds, produce typed blockers or HB/rank closure, or complete global PkgC, ZeroSlack, or PCCMin. It is conditional and not unconditional ZeroSlack; it does not put SAT in P, remove a project assumption, or prove P = NP. |
| Finite Packet charge-surplus realizer kernel | formalized-residual-terminal-packet-charge-surplus | For arbitrary finite support and replacement charge ledgers, exact multiplicity-preserving occurrence pairing and pairwise weight preservation leave an unmatched positive support charge and derive strict occurrence count and strict total replacement weight. When the totals exactly account for current and replacement NAND gates and semantic equivalence is proved independently, Lean constructs a genuine StrictEquivalentGain and strict reference-residual descent without assuming either strict inequality. | The finite ledgers, exact occurrence pairing, gate accounting, and semantic equivalence remain explicit inputs. This milestone does not construct a replacement or its charge ledger from a Packet blueprint or terminal data, prove selector faithfulness or compatibility, produce BotHN, BotBUD, or a lower-rank BotSeed, close HB/rank routing, establish encoded-size or polynomial-runtime bounds, or complete global PkgC, ZeroSlack, or PCCMin. It is not unconditional ZeroSlack; it does not put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet unit-charge blueprint realizer | formalized-residual-terminal-packet-unit-charge-blueprint-realizer | For arbitrary direct-wire input and output arities and every finite explicit grouped BN6 family of replacement blueprints, Lean validates canonical unit-charge gate-occurrence ledgers with a constructive exact permutation checker, a nonempty unmatched current-gate remainder, and semantic equivalence. Acceptance mechanically constructs the generic charge-surplus realization, a genuine StrictEquivalentGain, and strict residual descent without accepting a gate inequality. Every original blueprint atom behind every canonical handle is scanned, and the complete pair, balanced-triple, and full-span Packet conclusion is retained literally. | The grouped BN6 family, candidate implementations, replacement blueprints, occurrence pairings, and unmatched lists remain explicit inputs. Supplied-family validator silence is not BotHN, BotBUD, a lower-rank BotSeed, global absence of strict gains, semantic minimality, or ZeroSlack. This milestone does not derive blueprints from terminal data, establish manuscript selector faithfulness or compatibility, close HB/rank routing, establish encoded-size or polynomial-runtime bounds, or complete global PkgC, ZeroSlack, or PCCMin. It does not put SAT in P, remove a project assumption, or prove P = NP. |
| Checked Packet typed-realizer contract | formalized-residual-terminal-packet-typed-realizer-contract | For arbitrary selector types, finite selector lists, positive finite rank carriers, executable faithfulness and blocker-activity tables, and data-only realizer claims, Lean accepts a faithful selector row exactly as a checked unit-charge gain, an active same-or-lower-rank HN bot, an active same-or-lower-rank budget bot, or a faithful strictly lower-rank seed bot. The list validator covers every faithful member, and its grouped-BN6 specialization covers every canonical input-relative Packet handle. | The selector family, finite rank assignment, faithfulness predicate, realizer claims, hereditary activity table, and budget activity table remain explicit inputs. Finite indices are not the manuscript's tuple-valued packet ranks, and an invalid faithful row is rejected rather than reinterpreted as a bot. This milestone does not construct blueprints or blockers from terminal data, prove selector faithfulness or compatibility, establish blocker semantics or HB acyclicity, derive global selector silence, or establish encoded-size or polynomial-runtime bounds. It does not complete PkgC, unconditional ZeroSlack, or PCCMin, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked exact-rank HB blocker-graph acyclicity | formalized-residual-terminal-hb-blocker-graph-acyclicity | For arbitrary finite HN/BUD data-edge graphs, Lean exhaustively checks that the supplied finite rank indices embed strictly into the exact ten-coordinate residual rank and that every supplied dependency edge strictly descends that rank. Acceptance derives accessibility and well-foundedness of the exact supplied dependency relation, proves that it has no nonempty directed cycle, upgrades valid lower-seed bots to exact-rank descent, and composes these facts with every faithful canonical handle in an accepted Packet typed-realizer table. | The graph, edges, finite-to-exact rank mapping, selector family, faithfulness predicate, blocker activity tables, and realizer claims remain explicit inputs. This milestone does not prove dependency completeness, blocker semantics, that every active HN or budget row records all dependencies, or construction of the graph or rank mapping from terminal data. It does not establish selector compatibility, rank-complete selector silence, the full HB.NegativeClosure theorem, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked total-table HB dependency closure | formalized-residual-terminal-hb-dependency-table-closure | For every arbitrary finite rank carrier, Lean enumerates all HN and budget nodes, assigns each node one total data-only dependency row, and materializes the graph mechanically from all rows without accepting a second edge list. The checker exhaustively validates the finite-to-exact ten-coordinate rank embedding and strict exact-rank descent for every row dependency. Acceptance proves exact row-to-edge coverage, accessibility, well-founded rank induction for arbitrary predicates with an explicit local premise, absence of every nonempty dependency cycle, covered HN/BUD bot rows, and exact-rank descent for lower seeds. | The dependency table, finite-to-exact rank mapping, selector family, faithfulness predicate, blocker activity tables, and realizer claims remain explicit inputs. Total table coverage means only that every finite HN/BUD node has a row and every dependency listed in that row becomes a graph edge. It does not prove blocker semantics, semantic dependency completeness relative to terminal data, or the local invariant premise needed by generic rank induction. It does not establish selector compatibility, rank-complete selector silence, the full HB.NegativeClosure theorem, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked HB active-dependency closure | formalized-residual-terminal-hb-active-dependency-closure | For every arbitrary finite rank carrier and supplied typed-realizer environment, Lean exhaustively checks that every active HN or budget node has an active dependency in its own total row and independently checks strict exact-rank descent for every row dependency. Well-founded induction then proves every supplied HN and budget activity bit is false. Composition with every faithful canonical grouped-BN6 handle eliminates HN/BUD typed bots while preserving a genuine checked strict gain or a faithful strictly lower-rank seed. | The activity bits, dependency rows, finite-to-exact rank mapping, selector family, faithfulness predicate, and realizer claims remain explicit inputs. The local check does not derive blocker activity, blocker semantics, or semantic dependency completeness from terminal data. It eliminates HN/BUD bots only after the supplied tables pass; a checked gain or faithful lower-rank seed remains. This milestone does not establish selector compatibility, gain exclusion, lower-seed closure, rank-complete selector silence, the full HB.NegativeClosure theorem, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Conditional HB selector-silence rank closure | formalized-residual-terminal-hb-selector-silence-closure | For every arbitrary finite accepted typed-realizer table, checked HN/BUD active-dependency closure and semantic exclusion of every strict equivalent gain leave a faithful selector only if there is a faithful selector at strictly lower finite rank. Strong induction proves every canonical selector in the supplied grouped-BN6 family nonfaithful. A second theorem obtains the global gain-exclusion premise from the existing explicit gain-coverage certificate and exact source-cell no-gain evidence. | The global semantic gain exclusion is an explicit proof-bearing premise. The coverage specialization still requires a supplied certificate covering every strict equivalent gain plus exact source-cell no-gain evidence. The grouped family, finite rank and faithfulness tables, realizer claims, blocker activity, dependency rows, and rank map remain explicit inputs. This does not establish selector faithfulness or compatibility, construct those inputs from terminal data, derive blocker semantics or semantic dependency completeness, prove the unconditional HB.NegativeClosure theorem, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P, remove a project assumption, or prove P = NP. |
| Executable HB selector-silence induction | formalized-residual-terminal-hb-executable-selector-silence-induction | For every arbitrary finite accepted typed-realizer table, one exhaustive executable check proves that every canonical realizer claim is a typed bottom and retains faithful-row validity. Checked HB active-dependency closure eliminates HN and budget bottoms, while strong induction on the supplied finite rank eliminates faithful strictly lower-rank seeds. Every canonical selector is therefore nonfaithful without global semantic no-gain as a theorem premise. | The grouped family, finite rank and faithfulness functions, realizer claim function, blocker activity functions, dependency rows, and finite-to-exact rank map remain explicit data inputs. The checker validates these data but does not construct them from terminal candidates or prove selector faithfulness, selector compatibility, blocker semantics, or semantic dependency completeness. This milestone does not establish the full unconditional HB.NegativeClosure theorem, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Packet selector-faithfulness routing and HB contradiction | formalized-residual-terminal-packet-selector-faithfulness-routing | For every arbitrary finite explicit grouped BN6 family, exhaustive route-clear canonical payload checks plus exact binding to the supplied HB faithfulness table turn every positive Packet conclusion into a faithful canonical handle. Accepted executable HB selector silence and active-dependency closure prove that same handle nonfaithful, yielding a contradiction with selector silence. A fixed first-failure classifier exposes colour, frontier, charge, obligation, activation, direction, budget, rank, exact-route, or descent failure. | The grouped family, payload field Booleans, finite rank tags, route-clear payload checks, exact HB binding, realizer claims, blocker activity, dependency rows, and finite-to-exact rank map remain explicit terminal-relative inputs. The checkers do not derive those inputs from a terminal candidate or prove their external manuscript semantics. This milestone does not derive positive slack, SaturatePositive, BCELReady, the grouped family, or the no-lower ledger; establish unconditional HB.NegativeClosure or complete route silence; prove unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |
| Canonical Packet faithfulness-table construction | formalized-residual-terminal-packet-selector-faithfulness-table | For every arbitrary finite explicit grouped BN6 family and finite rank carrier, Lean canonicalizes a typed-realizer table by replacing its free faithfulness function with the canonical positive source-payload computation while preserving rank, HN activity, budget activity, and every realizer claim exactly. Exhaustive faithfulness binding accepts by construction, and a route-clear positive Packet contradicts accepted executable HB selector silence without an independent binding premise. | The grouped family, ten payload field Booleans, finite rank tags and rank assignment, route-clear acceptance, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The constructor does not derive those inputs from a terminal candidate or prove their external manuscript semantics. This milestone does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Total Packet selector first-route outcome | formalized-residual-terminal-packet-selector-first-route-outcome | For every arbitrary finite explicit grouped BN6 family and finite rank carrier, Lean proves the canonical payload first-route classifier total: no route exactly on acceptance and one earliest typed route exactly on rejection. Every positive Packet under the canonicalized HB table, accepted executable selector silence, and accepted active-dependency closure yields a first typed route without route-clear or binding premises. | The grouped family, ten payload field Booleans, finite rank tags and rank assignment, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The result does not prove any route's external semantics and does not map reported routes into a decreasing complete global outcome system. It does not construct those inputs from a terminal candidate, establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Exact Packet first-route failure semantics | formalized-residual-terminal-packet-selector-first-route-semantics | For every arbitrary finite payload rank and all ten route constructors, Lean proves the executable first route is equivalent to the exact earliest failed supplied field, with every preceding condition accepted. The failure is unique, rejection is equivalent to exact failure existence, and the canonical positive Packet/HB outcome carries the route and exact field-failure proof without route-clear or binding premises. | The grouped family, ten payload field Booleans, finite rank tags and rank assignment, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The result does not derive the payload fields or family from terminal data, does not prove their external manuscript semantics, and does not map a reported failure into a decreasing complete global outcome system. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Rank-reflected Packet descent route | formalized-residual-terminal-packet-descent-route-reflection | For every arbitrary finite grouped BN6 family and selector-rank carrier, the final strict-descent payload condition is computed from the exact ten-coordinate RankWF comparison while the preceding fields are preserved. Accepted computed payloads carry actual rank descent, a final descent failure carries actual nondecrease, and the positive Packet/HB endpoint returns either an earlier exact field route or proof that the supplied transition is nondecreasing, without route-clear or descent-binding premises. | The first nine payload fields, before/after residual ranks and their handle assignment, grouped family, finite selector-rank map, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The result does not construct the grouped family or ranks from terminal data, prove external manuscript semantics for the earlier fields, map those other nine routes into the complete global outcome system, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical Packet rank-tag route reflection | formalized-residual-terminal-packet-rank-route-reflection | For every arbitrary finite grouped BN6 family and selector-rank carrier, the payload rank tag is copied from the authoritative handle rank while final residual descent remains computed from the exact ten-coordinate RankWF comparison. The canonical first-route classifier cannot return rank; a final descent route proves the supplied transition is nondecreasing, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The finite rank map, before/after residual ranks and their handle assignment, seven earlier Boolean payload fields, exactRouteClear, grouped family, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The result does not construct the grouped family or rank map from terminal data, prove external manuscript semantics for the eight remaining routes, map those routes into the complete global outcome system, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical Packet exact-route reflection | formalized-residual-terminal-packet-exact-route-reflection | For every arbitrary finite grouped BN6 family and selector-rank carrier, each canonical handle selects an original positive payload atom from its exact grouped cell and footprint. That canonical handle-to-cell-to-payload source route is marked clear by construction while the authoritative handle rank is copied and residual descent is computed from the exact ten-coordinate RankWF comparison. The first-route classifier cannot return exactRoute or rank; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The seven semantic Boolean payload fields, finite rank map, before/after residual ranks, grouped family, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. This internal route is not an external exact minimum or a proof of manuscript exact-minimum semantics. The seven remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, derive the seven fields from a terminal candidate, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical Packet charge-route reflection | formalized-residual-terminal-packet-charge-route-reflection | For every arbitrary finite grouped BN6 family and selector-rank carrier, each canonical handle selects a strictly positive source payload atom. That positive-source-charge fact is reflected by construction while the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison remain canonical. The first-route classifier cannot return charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The six remaining semantic Boolean payload fields, finite rank map, before/after residual ranks, grouped family, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. Strictly positive source mass is not the full external charge-surplus, budget, replacement, or selector-compatibility semantics. The six remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, derive those six fields from a terminal candidate, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical Packet colour-route reflection | formalized-residual-terminal-packet-colour-route-reflection | For every arbitrary finite grouped BN6 family and selector-rank carrier, each canonical handle has a grouped footprint proved to lie in the family carrier and to have selector-relevant size. That internal grouped-footprint colour check is computed by construction while positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison remain canonical. The first-route classifier cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The internal colour check proves only canonical grouped-footprint eligibility: selector-relevant size with carrier-sublist membership retained as a separate theorem. It is not the full external manuscript colour equivalence. The five remaining semantic Boolean payload fields, finite rank map, before/after residual ranks, grouped family, realizer claims, HN/BUD activity, dependency rows, and finite-to-exact rank map remain explicit inputs. The five remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, derive those five fields from a terminal candidate, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical Packet typed-frontier route reflection | formalized-residual-terminal-packet-frontier-route-reflection | For every arbitrary finite grouped BN6 family, selector-rank carrier, and typed frontier-signature domain with decidable equality, the canonical payload computes frontier acceptance from exact equality of the supplied source and selector signatures while retaining grouped colour, positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison. A frontier first route is exactly typed signature inequality; equal signatures exclude it. The first-route classifier also cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The source and selector frontier signatures remain explicit inputs: this milestone does not construct the supplied signatures from terminal data, bind them to a BN5 coordinate, or prove the manuscript's full frontier-faithful comparison. Obligation, activation, direction, and budget remain supplied Boolean fields. Those four remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| BN5-bound Packet frontier and obligation route reflection | formalized-residual-terminal-packet-bn5-obligation-route-reflection | For every arbitrary finite grouped BN6 family, selector-rank carrier, and typed terminal BN5 coordinate, the canonical payload computes frontier and obligation acceptance from exact equality of the corresponding source and selector BN5 fields while retaining grouped colour, positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison. A frontier first route is exactly frontier inequality; an obligation first route is prior frontier equality together with exact obligation inequality. The first-route classifier also cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The source and selector terminal BN5 coordinates remain explicit inputs: this milestone does not construct those coordinates from terminal data or prove the manuscript's complete BN5 or Packet adequacy bridge. Activation, direction, and budget remain supplied Boolean fields. Those three remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| BN4 activation-exact Packet route reflection | formalized-residual-terminal-packet-bn4-activation-route-reflection | For every arbitrary finite grouped BN6 family, selector-rank carrier, and typed terminal BN5 coordinate with decidable equality of activation atoms, the canonical payload computes frontier and obligation acceptance from the corresponding BN5 fields and activation acceptance from equality of the nested BN4 activation atoms while retaining grouped colour, positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison. Activation-atom equality is equivalent to equality of the canonical BN4 activation predicates on every cut. An activation first route is prior frontier and obligation equality together with exact activation-atom inequality. The first-route classifier also cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The source and selector terminal BN5 coordinates remain explicit inputs: this milestone does not construct those coordinates from terminal data or prove the manuscript's complete BN4, BN5, or Packet adequacy bridge. Direction and budget remain supplied Boolean fields. Those two remaining routes still lack complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Typed Packet direction-route reflection | formalized-residual-terminal-packet-direction-route-reflection | For every arbitrary finite grouped BN6 family, selector-rank carrier, and domain of typed direction values with decidable equality, the canonical payload computes direction acceptance from exact source and selector direction equality while retaining computed BN5 frontier, obligation, and BN4 activation acceptance, grouped colour, positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison. A direction first route is prior frontier, obligation, and activation equality together with exact typed-direction inequality. The first-route classifier also cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The source and selector direction values remain explicit inputs: this milestone does not construct those direction values from terminal data or prove the manuscript's complete Dir(u) or Packet adequacy bridge. Budget is the sole remaining supplied Boolean field. That sole remaining route still lacks complete external semantics and global integration. The result does not construct the grouped family or rank map from terminal data, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Typed Packet budget-route reflection | formalized-residual-terminal-packet-budget-route-reflection | For every arbitrary finite grouped BN6 family, selector-rank carrier, and domain of typed budget values with decidable equality, the canonical payload computes budget acceptance from exact source and selector budget equality while retaining computed BN5 frontier, obligation, BN4 activation, and typed direction acceptance, grouped colour, positive charge, the internal source route, authoritative handle rank, and exact ten-coordinate descent comparison. A budget first route is prior frontier, obligation, activation, and direction equality together with exact typed-budget inequality. The first-route classifier cannot return colour, charge, rank, or exactRoute; a final descent route proves nondecrease, and the positive Packet/HB endpoint carries exact failure evidence without route-clear or binding premises. | The source and selector budget values remain explicit inputs: this milestone does not construct those budget values from terminal data, prove the manuscript's complete Bud(u) budget-envelope or Packet adequacy bridge, or identify local Packet budget coherence with BudgetResolve or HB budget activity. Computing every local Packet classifier field does not establish that terminal data supplies adequate values or that all routes are globally excluded. The result does not construct the grouped family or rank map from terminal data, prove that a decreasing transition exists, or construct the no-lower ledger. It does not establish full external selector compatibility, complete route silence, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet budget/HB activity binding | formalized-residual-terminal-packet-budget-hb-activity-binding | For every arbitrary finite grouped BN6 family, the executable binding checker exhaustively requires each typed source/selector budget mismatch to imply activity of the HB budget node at the table-owned handle rank. When that check and the existing checked well-founded HB no-outcome closure both pass, every canonical handle has equal typed budgets, the Packet first-route classifier excludes the budget route, and the positive Packet endpoint is confined to frontier, obligation, activation, direction, or exact residual nondecrease. | The Packet-to-HB budget binding is still an explicit checked input over a supplied grouped family, rank map, activity environment, and dependency table; this milestone does not construct that binding, the typed budget values, or a manuscript Bud(u) envelope from terminal data. It does not implement BudgetResolve, prove blocker semantic completeness, derive the HB table or its local closure premise from terminal data, or exclude the remaining frontier, obligation, activation, direction, and descent routes. It does not establish full Packet adequacy, unconditional HB.NegativeClosure, positive slack, SaturatePositive, BCELReady, unconditional ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet semantic/HN activity binding | formalized-residual-terminal-packet-semantic-hn-activity-binding | For every arbitrary finite grouped BN6 family, the executable binding checker exhaustively requires any failure of simultaneous frontier, obligation, activation, and direction agreement to imply activity of the HN node at the table-owned handle rank. When that check and the existing checked well-founded HB no-outcome closure both pass, all four typed semantic fields agree. Combined with the separately checked budget/HB binding and executable selector silence, every semantic first route is excluded and the positive Packet endpoint has the sole exact residual-nondecrease route. | The Packet-to-HN semantic binding is still an explicit checked input over a supplied grouped family, typed BN5 coordinates, activation atoms, direction values, rank map, activity environment, and dependency table; this milestone does not construct that binding or those values from terminal data. It does not establish HN blocker semantics, semantic dependency completeness, a decreasing transition, a no-lower contradiction, complete external Packet adequacy, or unconditional HB negative closure. It does not prove ZeroSlack, PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |
| Checked Packet descent no-lower binding | formalized-residual-terminal-packet-descent-no-lower-binding | For every arbitrary finite grouped BN6 family and selector-rank carrier, the executable local Packet descent no-lower checker scans every canonical handle and accepts exactly when no fully computed first route is residual nondecrease. Under a supplied positive Packet conclusion, checked semantic/HN and budget/HB bindings, executable selector silence, and checked well-founded HB no-outcome closure, the checker is forced to reject; assuming that same local row accepted yields a contradiction. | This is one checked local Packet no-lower row over a supplied finite grouped family, rank map, residual ranks, typed fields, realizer table, dependency table, and accepted binding, silence, and closure checks. It does not construct those inputs from terminal data or complete the manuscript's no-lower ledger. It does not cover HResolve, BudgetResolve, normalization, named descent routes, saturation, or replay; derive unconditional HB closure; establish complete Packet adequacy or unconditional ZeroSlack; prove PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet no-lower ledger | formalized-residual-terminal-packet-no-lower-ledger | For every supplied arbitrary finite grouped BN6 family, typed realizer table, HB dependency table, finite rank map, and before/after residual ranks, one Boolean recomputes the five executable checks for semantic/HN binding, budget/HB binding, selector silence, HB no-outcome closure, and Packet descent/no-lower. Its reflection theorem characterizes exact acceptance, a positive Packet forces rejection, and an accepted ledger excludes a positive Packet conclusion for those same inputs. | This closes the Packet branch only, not the complete no-lower ledger. The grouped family, BN5 coordinates, activation atoms, direction and budget values, rank map, residual ranks, realizer claims, activity environment, dependency rows, and all terminal data remain supplied. It does not implement HResolve, BudgetResolve, normalization, other named obstructions, saturation, or replay; derive unconditional HB closure; establish unconditional ZeroSlack; prove PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |
| Checked finite HResolve coverage ledger | formalized-residual-terminal-hresolve-coverage-ledger | For every arbitrary finite supplied HResolve candidate family with decidable equality and arbitrary decidable exact, gain, and blocker predicates, Lean computes a fixed-priority exact, gain, blocked, or unresolved route for every candidate, generates a sound and complete route ledger, and checks both duplicate-free enumeration and all-candidate blocked coverage. The checker accepts exactly a NoHereditary sidecar for that supplied family, and acceptance excludes exact and gain routes for every enumerated candidate. | The finite HResolve candidate family and its decidable exact, gain, and blocker predicates remain supplied. This milestone does not construct the governed hereditary universe from terminal data or prove that those predicates implement the manuscript's HN grammar, BWL exactness, H-disjointness, exact-minimum semantics, strict-gain semantics, or blocker dependency semantics. It does not discharge the full historical HResolve theorem, implement BudgetResolve, normalization, the complete no-lower ledger, saturation, or replay; establish unconditional HB closure or ZeroSlack; prove PCCMin, encoded-size or polynomial-runtime bounds, SAT in P; remove a project assumption; or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal-derived HResolve support resolver | formalized-residual-terminal-hresolve-support-resolver | For every finite direct-wire candidate and profile-width model, Lean constructs the complete duplicate-free family of canonical terminal support seeds, saturates each seed in the candidate-derived terminal system, and computes either an exact route with semantic-minimum evidence or a gain route with a strict-equivalent-gain witness. | The reference search is exhaustive over the complete finite terminal support universe and may be exponential in the number of primitive records. This milestone does not implement the manuscript's HN grammar, BWL exactness, H-disjointness, blocker or NoHereditary semantics, polynomial HResolve, BudgetResolve, the complete no-lower ledger, unconditional ZeroSlack, PCCMin, polynomial runtime, SAT in P, or P = NP. |
| Terminal budget-envelope resolver | formalized-residual-terminal-budget-envelope-resolver | For every finite direct-wire candidate, candidate-derived saturation model, and supplied natural resource caps, Lean scans the canonical terminal support universe, recomputes nonempty gate and interface feasibility plus gate and saturated-record caps, and returns a feasible semantic minimum, a feasible strict equivalent gain, or complete NoBudget exclusion for every canonical seed. | The budget caps remain supplied, and the complete terminal subset scan, candidate-derived saturation, and reference minimization are exhaustive and may be exponential. This milestone does not implement the manuscript's HN/BUD grammar, BWL or budget-envelope dynamic program, blocker dependency semantics, polynomial BudgetResolve, the complete no-lower ledger, unconditional ZeroSlack, PCCMin, polynomial runtime, SAT in P, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal budget no-lower ledger | formalized-residual-terminal-budget-no-lower-ledger | For every finite direct-wire candidate, candidate-derived saturation model, and supplied natural resource caps, Lean classifies every canonical terminal support as exact, strict gain, or NoBudget, materializes the complete route ledger, and proves that accepted exhaustive coverage makes every budget-feasible governed support a semantic minimum while excluding every feasible strict equivalent gain. | The budget caps remain supplied, and complete terminal subset enumeration, candidate-derived saturation, and reference minimization are exhaustive and may be exponential. This closes only the finite terminal-derived budget branch. It does not implement the manuscript's HN/BUD grammar, BWL or budget-envelope dynamic program, blocker dependency semantics, polynomial BudgetResolve, Packet or complete no-lower composition, unconditional ZeroSlack, PCCMin, polynomial runtime, SAT in P, or P = NP. |
| Terminal Packet-budget no-lower composition | formalized-residual-terminal-packet-budget-no-lower-composition | For every finite direct-wire candidate, candidate-derived saturation model, supplied caps, and supplied Packet family and tables over that same direct-wire candidate, one Boolean recomputes both finite ledgers. Acceptance makes every governed budget-feasible canonical support a semantic minimum, excludes every feasible strict equivalent gain, and excludes a positive Packet conclusion. | The caps, Packet family, typed payloads, ranks, realizer claims, activity environment, and dependency tables remain supplied. This is a finite two-branch composition, not the complete no-lower ledger. It does not construct Packet data from terminal data, implement HN/BUD grammar, polynomial HResolve or BudgetResolve, cover normalization, remaining named routes, saturation, or replay, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal HResolve maximal H-disjoint family | formalized-residual-terminal-hresolve-maximal-h-disjoint-family | For every arbitrary finite duplicate-free family of supplied hereditary footprints over eight decidable coordinate domains, Lean deterministically constructs a governed duplicate-free maximal pairwise H-disjoint family. Every rejected governed candidate has a selected blocker carrying the exact first support, frontier, origin, kernel, obligation, prefix-tail, charge, or interface interference route. | The hereditary footprints remain supplied inputs. This milestone does not derive them from a terminal candidate, formalize the HN pair/tripod/spine/non-flat grammar, prove BWL exactness or ParseOrExit, establish leaf tightness or solve a leaf, construct the full H0-H4 NoHereditary sidecar, connect blockers to HB ranks, implement full HResolve, complete the no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Terminal HN BWL certified-path minimum | formalized-residual-terminal-hn-bwl-certified-path-minimum | For every nonempty finite supplied family of certified hereditary paths, Lean computes the exact four-coordinate lexicographic minimum by realized cost, residual rank, frontier deviation, and direct-wire code. The selected member preserves semantic fidelity, frontier fidelity, nonempty block coverage, and one of four HN shapes; an explicit completeness premise extends its lower bound to every path in the supplied governed predicate. | The certified path family, governed predicate, and family completeness remain supplied inputs. This milestone does not derive accepted paths from terminal data, prove shape-grammar soundness or completeness, LN confluence, ParseOrExit, independent leaf tightness, or the full BWL theorem, solve a leaf, construct the H0-H4 NoHereditary sidecar, implement full HResolve, complete the no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial generation and runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Terminal HResolve certified-path family | formalized-residual-terminal-hresolve-certified-path-family | For every duplicate-free finite supplied family of proof-bearing hereditary candidates, Lean constructs a duplicate-free maximal pairwise H-disjoint selected family. Every selected candidate has an exact four-coordinate certified-path minimum preserving semantic, frontier, block, shape, and footprint-coherence evidence; every rejected candidate has a selected blocker carrying the exact first interference route. | The candidates, certified paths, hereditary footprints, governed predicates, family-completeness proofs, and path-to-footprint coherence proofs remain supplied inputs. This milestone does not derive them from terminal data, prove HN grammar soundness or completeness, LN confluence, ParseOrExit, independent leaf tightness, or the H0-H4 NoHereditary sidecar, connect blockers to HB rank semantics, implement full or polynomial HResolve, complete the no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Proof-bearing HResolve ZeroSlack sidecar | formalized-residual-terminal-hresolve-zeroslack-sidecar | For every arbitrary finite supplied governed HResolve family, the report-facing ZeroSlack boundary now consumes a checked proof-bearing NoHereditary sidecar: duplicate-free total blocked coverage is recomputed, exact and gain routes are excluded for every listed candidate, and those route predicates are separately bound to semantic minimum and strict equivalent gain propositions. | The governed candidate family, implementation map, exact, gain, and blocked predicates, their decidability, and blocker semantics remain supplied inputs. This milestone does not derive hereditary candidates from terminal data, prove HN grammar soundness or completeness, BWL, ParseOrExit, leaf tightness, H0-H4 blocker semantics, full or polynomial HResolve, complete the no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Proof-bearing Budget ZeroSlack sidecar | formalized-residual-terminal-budget-zeroslack-sidecar | For arbitrary finite direct-wire candidates and supplied natural caps, the report-facing ZeroSlack boundary now consumes a checked proof-bearing NoBudget sidecar. A failed exhaustive search over the complete canonical terminal support universe excludes every budget-feasible support, while separately checked exact and gain routes entail semantic minimum and strict equivalent gain propositions. | The natural terminal support caps are supplied inputs, and exhaustive enumeration, saturation, and reference minimization may be exponential. This milestone does not formalize the B0-B4 Budget grammar or blocker semantics, implement full or polynomial BudgetResolve, prove a complete no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Proof-bearing Selector/HB ZeroSlack sidecar | formalized-residual-terminal-selector-hb-zeroslack-sidecar | For arbitrary finite supplied grouped BN6, typed-realizer, and exact-rank HB dependency tables, the report-facing ZeroSlack boundary now consumes one checked proof-bearing Selector/HB sidecar. The exact executable checks yield all-canonical-selector nonfaithfulness, exact typed-bottom rows, complete no-outcome closure, all-node HN/BUD inactivity, and a well-founded dependency relation. | The grouped BN6 family, typed-realizer and dependency tables, environment, realizer claims, activity bits, dependency rows, and finite rank map remain supplied inputs. This milestone does not derive them from terminal data, prove selector faithfulness or compatibility, establish HN/BUD blocker semantics or semantic dependency completeness, connect selector silence to the BCEL contradiction, complete the no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Proof-bearing Packet/budget no-lower ZeroSlack sidecar | formalized-residual-terminal-packet-budget-no-lower-zeroslack-sidecar | For arbitrary finite supplied same-candidate Packet and terminal-budget data, the report-facing ZeroSlack boundary now consumes one checked proof-bearing Packet/budget no-lower sidecar. Its exact executable composition equation proves every governed budget-feasible support semantically minimum, excludes every such strict equivalent gain, and excludes a positive Packet conclusion for the supplied family. | The caps, candidate-derived model, grouped BN6 family, typed payloads, finite ranks, realizer claims, activity environment, dependency rows, and rank maps remain supplied inputs. This finite two-branch sidecar does not complete no-lower ledger coverage for the manuscript, derive its inputs from terminal data, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Proof-bearing BCEL/Packet no-lower ZeroSlack sidecar | formalized-residual-terminal-bcel-packet-no-lower-zeroslack-sidecar | For every arbitrary finite supplied M180 Packet/budget no-lower certificate whose grouped carrier passes the exact at-least-two-anchor check, the report-facing ZeroSlack boundary now consumes one checked proof-bearing BCEL/Packet contradiction sidecar. BCEL constant activation would construct a positive BN6 Packet, contradicting the accepted same-family no-lower exclusion. | The grouped BN6 family and all terminal, budget, Packet, realizer, dependency, and rank inputs inherited from M180 remain supplied. This milestone does not derive the family or BCEL constant activation from positive residual slack, construct BCELReady from a terminal candidate, complete the manuscript no-lower ledger, establish unconditional ZeroSlack, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |
| Same-family Selector/HB, Packet, and BCEL ZeroSlack coherence | formalized-residual-terminal-zeroslack-packet-selector-hb-coherence | For every arbitrary finite accepted M180 certificate and its dependent M181 boundary, the report-facing ZeroSlack layer now derives same-family Selector/HB, Packet, and BCEL evidence from the exact grouped family, computed realizer table, and dependency table. Definitional identities prevent a detached Selector/HB certificate from being paired with the checked Packet and BCEL exclusions. | The grouped family and all terminal, budget, Packet, realizer, dependency, rank, and BCEL data remain supplied inputs. This milestone does not derive those inputs or BCEL constant activation from positive residual slack, establish unconditional ZeroSlack, complete the manuscript no-lower ledger, prove PCCMin or polynomial runtime, put SAT in P, remove a project assumption, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Global locked-NAND construction and threshold | formalized-concrete-locked-nand-threshold | A uniform encoded polynomial-time SAT instance builder and the report-level locked-NAND threshold theorem linked to that builder. | This closes the uniform all-bitstring CNFSAT-to-concrete-locked-threshold builder in the finite charged-pipeline model. The downstream M186 compatibility milestone now activates that exact reduction in the report-facing bridge. Neither milestone puts the concrete locked threshold language in P, discharges residual-band minimization, ZeroSlack or PCCMin, proves concrete CNFSAT NP-hardness, or proves P = NP. |
| Concrete report-facing locked-NAND compatibility | formalized-concrete-legacy-locked-nand-compatibility | The report-facing language, decider, verifier, reduction, P/NP class, and P-equals-NP interfaces are exact compatibility names for the concrete finite-pipeline model. SAT and LockedNANDThreshold are definitionally the concrete CNFSAT and EncodedLocked-NANDThreshold languages, and the active bridge reuses the checked all-bitstring reduction without a caller-supplied trust field. | This removes the duplicate locked-NAND project axiom and caller-supplied reduction edge, but it does not put the concrete locked target in P, construct the residual-band reduction, prove PCCMin or ZeroSlack soundness, prove concrete CNFSAT NP-hardness, create the eligible root theorem, open any global gate, or prove P = NP. |
| Concrete report-facing residual-band compatibility | formalized-concrete-residual-band-compatibility | The report-facing residual-band endpoint is the concrete fail-closed direct-wire candidate/threshold language. Every intrinsically typed finite candidate has exact reference-minimum semantics after encoding, and the active locked-to-residual compatibility edge is the compiled identity polynomial reduction rather than caller-supplied trust. | This removes the residual-band language axiom and caller-supplied locked-to-residual reduction edge by identifying both endpoints with one concrete encoded exact-minimum threshold predicate. It does not construct an executable PCCMin algorithm, prove that exhaustive reference minimization is polynomial, establish residual-band promise bounds, prove unconditional ZeroSlack, put SAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Typed PCCPack generation and structural reflection | formalized-typed-pccpack-reflection | For every explicit proof-bearing PCCMin loop certificate, transparent Lean generation preserves that exact certificate, the canonical generated identifier is accepted by structural computation, every mismatched identifier is rejected, and the active bridge projects the supplied certificate directly rather than relying on package or checker axioms. | This removes the two opaque package/checker declarations and the caller-supplied reflection field from the active Lean bridge. It does not construct a PCCMinLoopCertificate, verify historical JavaScript package bytes, prove the semantic string fields, implement PCCMin, establish unconditional ZeroSlack or polynomial bounds, prove concrete SAT hardness, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |
| Proof-bearing PCCMin total-oracle loop | formalized-pccmin-total-oracle-loop | For every finite direct-wire implementation and every explicit proof-bearing total oracle, transparent well-founded recursion follows strict equivalent gains until exact-minimum or ZeroSlack evidence is returned, preserves semantics, returns a global minimum with zero residual slack, and bounds strict-gain iterations by the starting residual slack. | The total oracle remains an explicit proof-bearing argument, and the regression fixture uses exhaustive reference minimization. This milestone does not construct an executable PCCMin oracle, derive terminal families or routes, establish unconditional ZeroSlack, encode the loop as a raw machine, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Proof-bearing PCCMin normalization/oracle composition | formalized-pccmin-normalize-oracle-composition | For every finite direct-wire implementation, explicit proof-bearing total normalizer, and explicit proof-bearing total oracle, non-increasing semantic normalization lifts later oracle gains to strict gains from the original implementation, transports exact-minimum and ZeroSlack endpoints, and reuses the checked well-founded loop with its exact result and gain-iteration bound. | The total normalizer and total oracle remain explicit proof-bearing arguments, and the regression fixtures use exhaustive reference minimization. This milestone does not construct either executable stage, derive terminal families or routes, establish unconditional ZeroSlack, encode the stages as raw machines, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |
| Rank-ordered proof-bearing PCCOracle orchestration | formalized-pccmin-rank-ordered-oracle | For every finite direct-wire implementation, explicit proof-bearing normalizer, and explicit rank-ordered oracle builder, HResolve precedes BudgetResolve, every selector in every canonical finite rank row is scanned, gains return immediately, complete typed silence is required before ZeroSlack, and the resulting total oracle reuses the checked well-founded exact loop and gain-iteration bound. | The normalizer, HResolve and BudgetResolve algorithms, rank rows, selector universe, realizer, typed blocker semantics, and final ZeroSlack closure remain explicit proof-bearing inputs. The exhaustive reference fixture is not polynomial. This milestone does not construct executable PCCMin, derive terminal objects, establish unconditional ZeroSlack, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked Packet-backed rank-selector construction | formalized-pccmin-checked-packet-ranked-selector | For every finite direct-wire implementation and supplied checked Packet table, every canonical handle claim is validated as a data-only unit-charge gain or typed HN, budget, or lower-seed blocker; exact rank rows are derived by filtering the exhaustive canonical handle list with the table-owned rank map; and the resulting plan reuses the M191 rank-ordered oracle and checked well-founded loop. | The grouped terminal family, rank assignment, data-only claim table, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and complete-silence-to-ZeroSlack implication remain explicit supplied inputs. The exhaustive reference fixture is not polynomial. This milestone does not derive terminal objects or claims from an arbitrary implementation, construct executable PCCMin, establish unconditional ZeroSlack, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |
| Checked Packet/HB conditional ZeroSlack bridge | formalized-pccmin-checked-packet-hb-zeroslack-bridge | For every finite direct-wire implementation and supplied checked Packet/HB data, complete checked rank-row silence and checked HB no-outcome closure derive that every canonical handle is nonfaithful; one explicit positive-slack-to-faithful-selector bridge then yields conditional ZeroSlack, and the resulting adapter reuses the M192 selector construction and checked well-founded loop. | The positive-residual-slack-to-faithful-selector implication, grouped terminal family, rank assignment, data-only claim table, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The exhaustive reference fixture is not polynomial. This milestone does not derive terminal objects or the positive-slack bridge from an arbitrary implementation, construct executable polynomial PCCMin, establish unconditional ZeroSlack, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| BN6 computed-faithfulness checked Packet/HB ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-hb-zeroslack-bridge | For every finite direct-wire implementation and supplied checked Packet/BN6/HB data, positive residual slack reaches a general BN6 Packet through the explicit constant-activation boundary; route-clear payload checks construct a faithful selector in the canonicalized table; checked rank-row silence and checked HB closure then derive conditional ZeroSlack; and the resulting adapter reuses the M193 contradiction and checked well-founded loop. | The positive-residual-slack-to-constant-activation implication, grouped terminal family, carrier lower bound, rank assignment, route-clear payload checks, data-only claim table, HB dependency table, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The exhaustive reference fixture is not polynomial. This milestone does not derive terminal objects or constant activation from an arbitrary implementation, prove manuscript-wide SaturatePositive or BCELReady, construct executable polynomial PCCMin, establish unconditional ZeroSlack, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Same-candidate BCEL activation-route classifier and conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-activation-route | For every finite same-candidate checked BCEL nucleus and supplied grouped BN6 Packet family, a total classifier compares the exact carrier, cut value, and every canonical nonempty proper-cut activation weight. It returns one proof-bearing carrier, cut-value, or activation mismatch route, or derives coherent constant activation, identifies it with the BCEL projection excess, and reuses M194 to derive conditional ZeroSlack under checked selector silence. | The terminal problem, positive premise, checked finite BCEL-ready certificate, grouped Packet family, payloads, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The proper-cut classifier enumerates a finite powerset and is not proved polynomial. Its mismatch results are typed diagnostic routes, not constructed gains or globally decreasing transitions. This milestone does not derive the family or supporting data from every source input, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| BCEL-derived BN6 family skeleton and conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-derived-family | For every finite same-candidate checked BCEL nucleus and supplied grouped-cell ledger, the BN6 family carrier, duplicate-freedom, cut value, and cut-value positivity are constructed from the computed nucleus. M195's carrier and cut-value mismatch routes are impossible by construction; checked selector silence yields either an exact proper-cut activation mismatch or conditional ZeroSlack. | The terminal problem, positive premise, checked finite BCEL-ready certificate, grouped cells and payloads, exact grouping proofs, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The inherited proper-cut classifier enumerates a finite powerset and is not proved polynomial. Its remaining activation mismatch is a typed diagnostic route, not a constructed gain or globally decreasing transition. This milestone does not derive the grouped cells or supporting data from every source input, complete PkgC/BN3–BN6 integration, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical BN6 positive-cell grouping and conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-canonical-grouping | For every finite same-candidate checked BCEL nucleus and supplied raw positive support/payload ledger, supports are normalized inside the computed carrier, singleton consumer systems are constructed, duplicate footprints are coalesced, and every positive payload atom is preserved. The resulting canonical grouped family reuses M196, so checked selector silence yields either an exact proper-cut activation mismatch or conditional ZeroSlack. | The terminal problem, positive premise, checked finite BCEL-ready certificate, raw positive supports and payload atoms, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The inherited proper-cut classifier enumerates a finite powerset and is not proved polynomial. Its remaining activation mismatch is a typed diagnostic route, not a constructed gain or globally decreasing transition. This milestone does not derive the raw cells from BN3, BN4, BN5, PkgC, or every terminal input, complete PkgC/BN3–BN6 integration, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical BN6 raw cut-ledger conservation and conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-canonical-cut-ledger | For every finite same-candidate checked BCEL nucleus, supplied raw positive-cell ledger, and cut, canonical coalescing preserves the exact direct crossing-mass sum. The M197 grouped-family activation mismatch is therefore exposed directly against the raw ledger, so checked selector silence yields either an exact proper-cut raw activation mismatch or conditional ZeroSlack. | The terminal problem, positive premise, checked finite BCEL-ready certificate, raw positive cells and payloads, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and encoded-size bounds remain explicit supplied inputs. The inherited proper-cut classifier enumerates a finite powerset and is not proved polynomial. The direct raw-ledger mismatch is a typed diagnostic route, not a constructed gain or globally decreasing transition, and the constant-activation equation is not derived. This milestone does not derive the raw cells from BN3, BN4, BN5, PkgC, or every terminal input, complete PkgC/BN3–BN6 integration, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Canonical sparse V53 constant-cut basis and checked conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-canonical-constant-cut-basis | For every finite sparse V53 hypergraph with at least two anchors, a shape-specific basis is equivalent to the complete constant equation on every nonempty proper cut. A total classifier checks the two-anchor full weight, three singleton cuts, or four-plus full-span support and weight without enumerating carrier subsets. At the same-candidate checked BN6/BCEL/HB boundary, accepted evidence derives constant activation and conditional ZeroSlack under checked selector silence, while rejection preserves one typed structural route and every inspected cut retains M198's exact raw-ledger reflection. | The terminal problem, positive premise, checked finite BCEL-ready certificate, raw positive cells and payloads, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and complete encoded-size bounds remain explicit supplied inputs. A rejected basis is a typed structural diagnostic route, not a constructed gain or globally decreasing transition. Removing this endpoint's proper-cut powerset scan does not prove that upstream terminal or BCEL construction avoids subset enumeration or that the complete construction is polynomial. This milestone does not derive the raw cells from BN3, BN4, BN5, PkgC, or every terminal input, force basis acceptance or route every rejection to a gain, complete PkgC/BN3–BN6 integration, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Sparse V53 proper-cut activation route and checked conditional ZeroSlack bridge | formalized-pccmin-checked-packet-bn6-bcel-sparse-activation-route | For every finite sparse positive V53 hypergraph with at least two anchors, the duplicate-free singleton and order-preserving pair proper cuts form a quadratic-size test family equivalent to the complete constant equation on every nonempty proper cut. A total first-mismatch classifier either derives that equation or retains one exact small proper-cut mismatch. At the same-candidate checked BN6/BCEL/HB boundary, a mismatch is reflected through the direct raw positive-cell activation ledger, while coherence yields conditional ZeroSlack under checked selector silence. | The terminal problem, positive premise, checked finite BCEL-ready certificate, raw positive cells and payloads, claim table, rank map, route-clear equation, dependency table, checked HB closure, HResolve and BudgetResolve algorithms, normalizer, blocker semantics, and complete encoded-input bounds remain explicit supplied inputs. The singleton/pair family has a direct quadratic list-length bound, but this is not a complete polynomial construction or runtime theorem. A returned raw activation mismatch is an exact diagnostic route, not a verified gain or globally decreasing transition. This milestone does not derive the raw cells from BN3, BN4, BN5, PkgC, or every terminal input, map the mismatch into a decreasing route, complete PkgC/BN3–BN6 integration, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| PkgC same-key cancellation or derived BN6 positive cellization | formalized-residual-terminal-pkgc-bn6-positive-cellization | For every finite list of supplied active V54 consumer systems over one common carrier and supplied typed restorer, a total classifier returns the first exact same-key PkgC cancellation realization or proves all systems singletonized. The latter branch derives each raw BN6 positive-cell support and support-size fact from its active consumer data, preserves payload order, and conserves the exact V54 activation weight on every cut. | The terminal problem, source V54 consumer systems, active cuts, positive payload atoms, typed restorer, and all upstream BN3–BN5 construction remain explicit supplied inputs. A returned same-key cancellation is exact proof-bearing PkgC evidence, not a verified global gain or rank-decreasing transition. This milestone does not construct those sources from every valid terminal input, integrate them with the checked BCEL/Packet family, complete PkgC/BN3–BN6, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-derived PkgC/BN6 sparse BCEL route and checked conditional ZeroSlack | formalized-pccmin-checked-packet-pkgc-bn6-bcel-source-route | For every arbitrary finite supplied active PkgC source ledger over the exact checked BCEL nucleus, M201's total classifier either retains an exact source-member same-key cancellation or constructs the only raw BN6 positive-cell ledger admitted downstream. That derived ledger enters M200's canonical checked Packet/HB classifier, whose small-cut mismatch is reflected back through all-cut activation conservation to the original PkgC source ledger; the coherent branch yields genuine conditional ZeroSlack under supplied checked selector silence. | The terminal problem, checked finite BCEL-ready certificate, active V54 source systems and cuts, positive payload atoms, typed restorer, realizer table, accepted claims, rank assignment, dependency table, checked HB closure, route-clear result, and selector silence remain explicit supplied inputs. A returned PkgC cancellation or source-ledger activation mismatch is exact proof-bearing evidence, not a verified gain or globally rank-decreasing transition. This milestone does not construct the sources or downstream tables from every valid terminal input, construct upstream BN3–BN5 data, prove complete PkgC/BN3–BN6 route integration, derive blocker semantics or semantic dependency completeness, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed PkgC ambient-BN4 extraction and checked source-route composition | formalized-pccmin-checked-packet-pkgc-ambient-bn4-extraction-route | For every arbitrary finite supplied active PkgC source ledger and arbitrary finite ambient BN4 ledger tied to the same checked BCEL nucleus, constructive remove-first recursion preserves multiplicity and ambient order independence while computing the exact remainder, multiset embedding, and complete residual-ledger reduction, or proving that no exact remainder embedding exists. The four-way composition preserves M202's conditional ZeroSlack and source activation mismatch branches. | The terminal problem, checked finite BCEL-ready certificate, active V54 source systems and cuts, positive payload atoms, typed restorer, ambient BN4 ledger, realizer table, accepted claims, rank assignment, dependency table, checked HB closure, route-clear result, and selector silence remain explicit supplied inputs. The computed ambient residual reduction does not prove that its remainder is empty or contradict a surviving residual, and an exact no-embedding result is a compatibility failure rather than a verified gain or globally rank-decreasing transition. This milestone does not construct the ambient ledger, sources, payloads, restorer, or downstream tables from every valid terminal input, construct upstream BN3–BN5 data, prove complete PkgC/BN3–BN6 route integration, derive blocker semantics or semantic dependency completeness, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove encoded-size polynomial construction, runtime, output-size, or certificate-size bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| PkgC restoration coverage, exact Hall route, and computed ambient BN4 reduction | formalized-residual-terminal-pkgc-restoration-coverage-ambient-route | For every arbitrary finite supplied consumer system, restoration-coordinate universe and ambient BN4 ledger, the finite exact-coordinate classifier returns singletonization, the first proof-bearing Hall deficit with its qRestorationHall route, or complete coordinate coverage. Coverage constructs one balanced opposite-sign BN4 unit pair for every canonical quotient unit and exact arbitrary-order extraction computes the ambient remainder with complete residual-ledger equality or proves that no exact embedding exists. | The consumer system, finite restoration-coordinate universe, quotient coordinate map, full-restoration coordinate lists, and ambient BN4 ledger remain explicit supplied inputs. Complete coordinate coverage is equality-fibre multiplicity evidence and does not materialize semantic full candidates or prove restoration adequacy. A Hall deficit is an exact local qRestorationHall route, not a verified gain or globally rank-decreasing transition. The computed residual reduction does not prove that the remainder is empty, and no-embedding is a compatibility failure rather than a complete global route. This milestone does not derive the restoration universe or ambient ledger from every valid terminal input, complete PkgC/BN3–BN6 integration, derive blocker semantics or semantic dependency completeness, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove complete encoded-size polynomial bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Family-level PkgC restoration coverage and exact BN6 ledger cellization | formalized-residual-terminal-pkgc-restoration-coverage-bn6-ledger | For every arbitrary finite supplied active PkgC source ledger with per-source restoration and ambient BN4 data, the list-order classifier either proves all source systems singletonized and constructs the complete BN6 positive-cell ledger with exact length, payload order and all-cut activation conservation, or retains the first exact source-member Hall deficit, computed ambient reduction, or no-embedding witness. | The active PkgC source cells and cuts, positive payload atoms, per-source restoration-coordinate universes and maps, full-restoration coordinate lists, and ambient BN4 ledgers remain explicit supplied inputs. Complete coordinate coverage does not materialize semantic full candidates or derive restoration adequacy. A Hall deficit is not a verified gain or globally rank-decreasing transition, the computed remainder is not proved empty, and no-embedding is not a complete global route. This milestone does not derive the source ledger or restoration and ambient data from every valid terminal input, complete PkgC/BN3–BN6 integration, derive blocker semantics or semantic dependency completeness, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove complete encoded-size polynomial bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Candidate-bound PkgC restoration coverage and checked BN6/BCEL route | formalized-pccmin-checked-packet-pkgc-restoration-coverage-bn6-bcel-route | For every arbitrary finite M205 restoration-coverage source ledger tied to one computed BCEL nucleus, the total classifier preserves the first exact Hall deficit, ambient residual reduction, or no-embedding witness. Only the all-singletonized branch constructs the source-derived BN6 ledger and enters the checked sparse BN6/BCEL Packet/HB classifier, yielding conditional ZeroSlack or one exact small-cut mismatch reflected to the original enriched source activation ledger. | The terminal problem, checked BCEL-ready certificate, active PkgC source cells and cuts, restoration-coordinate universes and maps, ambient BN4 ledgers, ranks, claims, dependency table, checked HB closure, route-clear result, and selector silence remain explicit supplied inputs. The candidate binding does not materialize semantically adequate full candidates or derive these objects from every valid terminal input. A Hall deficit is not a verified global gain or rank-decreasing transition, the computed ambient remainder is not proved empty, and ambient incompatibility or source activation mismatch is not a complete global route. This milestone does not finish PkgC/BN3–BN6 integration, derive blocker semantics or semantic dependency completeness, prove manuscript-wide SaturatePositive or BCELReady, establish unconditional ZeroSlack, construct executable polynomial PCCMin, prove complete encoded-size polynomial bounds, put CNFSAT in P, open a global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| PkgC restoration coverage and exact ambient charge-coordinate descent | formalized-pccmin-checked-packet-pkgc-restoration-coverage-charge-descent | For every arbitrary finite exact ambient embedding returned by M206, unsigned BN4 charge is computed as the sum of cell masses. Exact permutation decomposes ambient charge into a strictly positive coverage-derived cancellation charge plus remainder charge. The remainder therefore strictly decreases the eighth chargeSize coordinate of the exact ten-coordinate TerminalResidualRank for every fixed surrounding context while preserving the complete canonical residual ledger. All other M206 outcomes remain explicit. | The terminal problem, checked BCEL-ready certificate, active PkgC source cells and cuts, restoration-coordinate universes and maps, ambient BN4 ledgers, semantic full candidates, Packet ranks and claims, dependency table, checked HB closure, route-clear result, and selector silence remain explicit supplied inputs. M207 proves exact charge-coordinate descent only for M206's successful ambient embedding. It does not prove the computed remainder is empty or turn Hall deficits, ambient incompatibility, or activation mismatches into complete global routes. This milestone does not finish terminal-input derivation, complete PkgC/BN3–BN6 integration, HN/BUD/HB semantic completeness, manuscript-wide SaturatePositive or BCELReady, unconditional ZeroSlack, executable polynomial PCCMin, complete encoded-size polynomial bounds, CNFSAT in P, a global gate, the eligible root theorem, or P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Global ZeroSlack, PCCMin, and polynomial runtime | not-formalized | Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds. | The finite candidate-derived BN3 envelope supplies stable request identities and one jointly side-tight canonical basis family; the finite BN4 kernel supplies activation-exact same-key integer cancellation over an explicit typed cell ledger; the finite BN5 kernel localizes explicit full/shadow multiplicity failure to a strict Hall deficit and local X1 route; the exhaustive Packet scan verifies strict gains or exact no-gain over every canonical selector in one supplied explicit grouped family; an explicit global gain-coverage certificate conditionally upgrades that silence to a proof-bearing ZeroSlack result; the generic finite R-ChargeSurplus kernel derives strict gain from exact ledgers, an unmatched positive support charge, exact gate accounting, and separately proved semantics; the checked unit-charge blueprint realizer derives canonical ledgers and gains for every valid blueprint in one supplied family; the checked typed-realizer contract rejects every faithful-table row except a genuine blueprint gain or an explicitly active bounded-rank HN, budget, or strictly lower faithful seed bot; the checked exact-rank HB graph contract validates every edge in a supplied finite HN/BUD dependency graph; the checked total-table HB contract gives every finite HN/BUD node one row, materializes every listed dependency as an edge, and derives well-founded induction and cycle exclusion for that supplied table; the checked HB active-dependency closure combines an exhaustive active-to-active row condition with strict |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete standard P-versus-NP target and root theorem | not-formalized | Raw-machine-linked complexity classes, concrete SAT completeness and a SAT decider, plus the publication root theorem. | The concrete target definition remains inactive: CNF-SAT has a raw-machine verifier, NP-membership proof, an all-input finite-machine polynomial reduction through NAND to the concrete locked target, and an exact report-facing compatibility link, but a deterministic polynomial-time CNF-SAT decider, SAT NP-hardness and CNFSAT NP-completeness transport, and PNP.Main.p_eq_np remain absent. |
| All-route staged-request physical dispatch | formalized-foundation-only | For every concrete verifier problem and every coordinate in its complete post-header schedule, M227 runs M226's fixed terminal-joined classifier over a protected canonical optional-token request, proves common body-or-Finish terminal geometry, crosses the blank-free classifier prefix with M225's fixed 14-rule request relay, and executes M217's reflected fixed 64-rule dispatcher to the exact next canonical emitted prefix. The resulting collision-free 816-rule composition has exact work, six-for-one compiled, one-step-short and source-input-size polynomial evidence. All 65 public declarations are axiom-audited: 23 have empty closure, four use only propext, and 38 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone keeps the canonical optional-token request explicitly staged on the protected initial tape. It does not synthesize the request from raw classifier state, connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-route derived-Finish physical request split | formalized-foundation-only | For every concrete verifier problem and every coordinate in its complete post-header schedule, M228 runs M226's terminal-joined classifier without a staged request cell. A fixed 20-rule relay reads the physical body-or-Finish terminal, crosses the exact blank-free classifier prefix, writes a collision-free body-pending marker for every body route, and writes the canonical Finish request only for the unique Finish route. A fixed 65-rule conditional reflected dispatcher rejects the body-pending marker and sends Finish to the exact next canonical emitted prefix. The collision-free 823-rule composition has exact work, six-for-one compiled, one-step-short and source-input-size polynomial evidence. All 91 public declarations are axiom-audited: 50 have empty closure, 18 use only propext, and 23 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone derives only the unique Finish request. Every body route halts at an explicit non-request pending marker; body-token and padding request synthesis remain open. It does not connect successive schedule configurations, implement one repeated raw-machine builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| All-route physical body-remainder split | formalized-foundation-only | For every concrete verifier problem and every coordinate in its complete post-header schedule, M229 runs M228 without a staged request, route, remainder or success certificate. It preserves the completed Finish endpoint and sends every body route through one fixed 36-rule physical scanner. The scanner crosses the retained clause-count and exterior boundaries, skips consumed-dividend marks, and distinguishes zero from positive remainder by the actual separator or unit symbol. The physical remainder equals the canonical body token coordinate. The collision-free 895-rule graph has exact work, six-for-one compiled execution, one-step-short nonhalting, and a verifier-input-size polynomial bound. All 71 public declarations are axiom-audited: 39 have empty closure, nine use only propext, and 23 use only propext and Quot.sound, with no project axiom or Classical.choice. | This milestone reads the physical body remainder but leaves clause occupancy and body-token and padding request synthesis open. Both body outcomes remain incomplete terminal configurations, distinguished by tape content. It does not connect successive schedule configurations, implement the repeated builder loop, prove builder FunctionProgram.RawRefinement, or package the Cook-Levin PolynomialReduction. It does not establish CNFSAT NP-hardness or NP-completeness transport or CNFSAT in P, close a fixed checkpoint or global gate, create the eligible root theorem, or prove P = NP. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Complete all-input Cook-Levin formula builder and reduction | formalized-foundation-only | For every concrete polynomial verifier and every ordinary raw bitstring, including empty and odd-length inputs, M230 runs the complete source-derived formula loop and physical output finalizer in one finite raw machine. Its exact ordinary output is the original canonical encoded Cook-Levin CNF formula, with total halting, runtime and output-size bounds polynomial in the original input length. The concrete PolynomialTimeFunction is a machine leaf; recursive FunctionProgram.RawRefinement preserves its output. The packaged PolynomialReduction to CNFSAT uses the already checked original formula semantics. No accepting certificate, execution trace, route, finite family, rank map or correctness certificate is supplied by the caller. | This completes the fixed all-input builder checkpoint, not a deterministic SAT algorithm. The separate named concrete NP-hardness or NP-completeness transport remains to be published. Unconditional residual minimization and ZeroSlack, complete polynomial PCCMin construction and certificate bounds, deterministic CNFSAT in P, and the eligible root theorem remain open. No global gate closes and P = NP is not proved. Execution theorem closures use only propext and Quot.sound; reduction-facing semantic closures additionally use the already allowed Lean standard axiom Classical.choice, never a project-specific axiom. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Concrete CNF-SAT NP-hardness and NP-completeness | formalized-foundation-only | For every language in the concrete bounded-certificate NP class, M231 extracts its polynomial-time verifier from NP membership and applies the complete source-derived all-input Cook-Levin PolynomialReduction to CNFSAT. Together with the existing concrete CNFSAT verifier, the closed theorem proves NPComplete CNFSAT in the selected finite-machine model. No reduction, execution trace, finite-instance restriction or correctness certificate is supplied. Both complete theorem interfaces are root-built and axiom-audited; their closures contain only the Lean standard axioms Classical.choice, Quot.sound and propext. | NP-completeness is hardness plus NP membership, not a deterministic polynomial-time SAT algorithm. This closes only the fixed concrete NP-hardness checkpoint. It does not close the separate final complexity transport to P = NP, unconditional residual minimization or ZeroSlack, complete polynomial PCCMin construction and certificate bounds, deterministic CNFSAT in P, or the eligible root theorem. All five global proof gates and the publication gate remain open; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Candidate-derived terminal profile dependency reflection and noninterference | formalized-foundation-only | For every finite direct-wire candidate size, executable terminal model, profile coordinate and gate, M232 proves that the computed profile-influence bit is true exactly when insertion changes the actual ambient observation in a canonical subset context. It identifies each computed profile-to-gate dependency with the coordinate's exact rule role and that influence bit. For every seed, a gate absent from the actual computed saturation cannot change a retained profile coordinate in any canonical context. The observation definition and all three general theorem interfaces are root-built and axiom-audited using only propext and Quot.sound. No dependency relation, coverage certificate or soundness proof is supplied in place of the existing computation. | The executable observer and profile model remain supplied data, and influence construction enumerates all subsets. This is a local semantic reflection and noninterference theorem, not an input-derived terminal family, arbitrary-record normalization, unconditional SaturatePositive or BCELReady, global route coverage or ZeroSlack, exact PCCMin construction, or an encoded-size polynomial runtime result. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Retained ambient-profile locality and whole-support preservation | formalized-foundation-only | For every finite direct-wire candidate size, executable terminal model, seed and retained profile coordinate, M233 proves that arbitrary primitive-record supports with the same computed-saturation gate membership have equal actual ambient observations. Structural extraction and ambient-implementation equalities account for record order, duplicates and non-gate metadata without an observer-congruence premise. General omitted-gate elimination extends M232 from canonical contexts to arbitrary supports. The actual saturated support therefore preserves each retained observation of the complete gate universe. All five theorem interfaces are root-built and axiom-audited using only propext and Quot.sound. No dependency relation, normalization certificate or correctness proof is supplied in place of the existing computation. | The executable observer and profile model remain supplied data, and influence construction enumerates all subsets. The theorem does not derive profile semantics or terminal families from every valid input, identify the ambient observer with the differently typed profileSystem.observe, prove transparent cost or complete route coverage, establish unconditional SaturatePositive, BCELReady or ZeroSlack, or supply an exact polynomial PCCMin construction and certificate bound. The retained-profile premise is necessary. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed saturation trace fidelity and metadata cost balance | formalized-foundation-only | For arbitrary finite terminal systems and seeds, M234 proves that the actual cost-accounting replay contains exactly the canonical computed saturated records and that every emitted event adds its required record with a valid selected dependency rule. For every finite direct-wire candidate and executable terminal model, the replay and canonical endpoints have equal ambient implementations and equal cost snapshot coordinates, with the stored record list kept explicit. Every generated non-gate metadata event is structurally cost-transparent. All five theorem interfaces are root-built and axiom-audited using only propext and Quot.sound. No trace-validity, replay-correctness, observer-congruence or balance certificate is supplied in place of the existing computation. | The executable observer and profile model remain supplied data. Influence and semantic minima use exhaustive finite reference constructions; no polynomial runtime is proved. Metadata cost balance does not establish physical-gate transparency, obligation discharge or complete closure safety. The theorem does not derive terminal families from every valid input, prove global route coverage, unconditional SaturatePositive, BCELReady or ZeroSlack, or construct the exact polynomial PCCMin algorithm and certificate bounds. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Actual physical saturation accounting and first-obstruction fidelity | formalized-foundation-only | For arbitrary finite terminal systems, seeds and actual generated events, M235 derives that the dependent is already active and the required record is genuinely new. For every finite direct-wire candidate and executable terminal model, the unchanged extractor charges exactly one physical gate for a generated gate event and zero for metadata, and the selected rule/dependent pair belongs to the computed active-owner list. Every generated nontransparent event is a physical insertion with a genuine nonunique-owner, full-minimum-growth or quotient-cost obstruction. The existing total classifier either preserves full slack and nondecreasing projection defect at canonical computed saturation or returns its exact first such obstruction with a transparent prefix. All five general interfaces are root-built and audited using only propext and Quot.sound. No freshness, charge-balance, active-owner, all-transparent-history or preselected-route certificate replaces the actual computation. | The executable observer and profile model remain supplied data. Influence and semantic minima use exhaustive finite reference constructions; no polynomial runtime is proved. An active dependency owner need not be unique. Exact physical support growth does not establish full-minimum growth, a quotient bound, obligation discharge, complete closure safety or a global named route. The first physical obstruction is not thereby a verified gain, an exact route or strict descent. This theorem does not derive terminal families from every valid input, prove global route coverage, unconditional SaturatePositive, BCELReady or ZeroSlack, or construct the exact polynomial PCCMin algorithm and certificate bounds. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed physical-charge partition and introduction provenance | formalized-foundation-only | For arbitrary finite terminal systems and seeds, M236 constructs a physical-NAND charge ledger directly from the normalized initial records and actual computed saturation trace. Its physical gate identifiers are duplicate-free and occur exactly when the corresponding gate records occur in the computed saturated support. Each entry has inherited seed provenance or an actual generating event with its exact rule and dependent, active context and fresh required gate. The deterministic lookup returns exactly the provenance of the corresponding ledger entry. For every finite direct-wire candidate and executable terminal model, the complete computed ledger length equals the unchanged extractor's support size at canonical saturation. All five general interfaces are root-built and audited using only propext and Quot.sound. No supplied charge list, coverage certificate, freshness premise or ownership certificate replaces the actual computation. | This is a physical-NAND charge partition with seed- and traversal-dependent introduction provenance, not the complete manuscript charge universe or its fixed global ownership function. Active requesting pairs are not assigned manuscript owners, and multiple physical charges may have the same requesting dependency. Materializer grouping and cross-support ownership transport remain open. The executable observer and profile model remain supplied data, while influence and semantic minima use exhaustive finite reference constructions. No full-minimum growth, quotient bound, global named route, input-derived terminal family, unconditional SaturatePositive, BCELReady or ZeroSlack, or complete polynomial PCCMin runtime and certificate bounds is established. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed saturated-support replacement context and physical slack | formalized-foundation-only | For every finite direct-wire candidate, executable terminal model and seed list, M237 computes a concrete replacement frame from production saturation and the actual physical gate complement. Gate-source closure proves that the selected support boundary contains only primary inputs. The unchanged extractor constructs both gate halves, and every complement boundary gate is bound to an actual selected-support interface port. The frame preserves the original ordered Boolean outputs when the extracted support is plugged back in. Every equivalent replacement on the exact extracted boundary and interface preserves the whole circuit, with an exact gate-count equation that counts every original physical NAND gate once and permits arbitrary replacement size. Transport through the existing constructive frame theorem bounds the computed support's physical Boolean residual slack by whole-circuit slack. All five general interfaces are root-built and audited using only propext and Quot.sound. No caller-supplied frame, fan-in-closure certificate, gate partition or whole-circuit correctness proof replaces the computation. | These are physical Boolean replacement and slack laws for actual production saturated supports, not arbitrary raw supports or full-profile compatibility. The complete manuscript profile/materializer charge universe, fixed global ownership map and full-profile replacement transport remain open. The executable observer and profile model remain supplied data, while influence and semantic minima use exhaustive finite reference constructions; no polynomial runtime is proved. No full-minimum growth, quotient bound, global named route, input-derived complete terminal family, unconditional SaturatePositive, BCELReady or ZeroSlack, or complete polynomial PCCMin runtime and certificate bounds is established. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root theorem remain absent; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed proper-support physical gain and exact slack descent | formalized-foundation-only | For every finite direct-wire candidate, executable terminal profile model and seed list, M238 computes the whole physical replacement using the existing reference-minimum witness and the actual production saturated-support context. The complete ordered Boolean output word is preserved. The replacement gate count plus the computed local gain equals the original gate count, and replacement residual slack plus that gain equals the original physical residual slack. These equations require no properness, positivity, supplied frame or supplied charge partition. The total optional gain step maps this construction over the existing candidate-derived proper-positive support search. Every returned result carries an actually selected canonical proper positive seed and strictly decreases whole physical gate count and residual slack. Failure is exactly absence of every canonical proper positive seed for that computed system, not global minimality. All five universal interfaces are root-built and audited using only propext and Quot.sound. The caller supplies no search result, replacement, seed, context or correctness certificate to the gain-step algorithm. | These are physical Boolean reference gain-realization and search-failure laws, not full-profile replacement compatibility or a complete named global route. The observer and profile model remain supplied data, while influence, canonical seed search and semantic minima use exhaustive finite reference constructions; no polynomial encoded-size or runtime bound is proved. Failure of the proper-support search does not imply global minimality or ZeroSlack. Complete profile/materializer charge coverage, fixed global ownership, obligation transport, full-minimum growth, quotient bounds, input-derived complete terminal families and global rank-decreasing route coverage remain open. No unconditional SaturatePositive, BCELReady or ZeroSlack, complete polynomial PCCMin, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed full-profile gain acceptance and exact slack descent | formalized-foundation-only | For every finite direct-wire candidate and supplied executable terminal profile model, M239 classifies the exact result of the existing production physical-gain search. No result is exactly the existing proper-support search failure. A result is scanned at every full-profile coordinate in canonical order. Rejection identifies the actual first mismatching coordinate and complete agreeing prefix; acceptance constructs the existing full-carrier realization without a caller-supplied result, seed, profile-equality proof or correctness certificate. Complete full-carrier realizations preserve the exhaustive full-profile minimum, and every accepted computed gain decreases full-profile slack by exactly the local physical gain of its actually selected proper-positive support. Existing obligation discharge transports through the full realization; an open obligation is not thereby discharged. The seven universal interfaces are explicit-root built with their reviewed standard axiom closures. The classifier reuses M238's proved Boolean equivalence rather than repeating an exhaustive equivalence check. | These are computed finite full-profile acceptance and accepted-result transport laws for the supplied executable profile system, not a derivation of manuscript carrier data or a theorem that every physical gain is full-profile compatible. A first profile mismatch is not a completed named global route. The observer and profile model remain supplied data; influence, canonical seed search and semantic minima remain exhaustive finite reference constructions, with no polynomial encoded-size or runtime theorem. The classifier does not search alternative physical gains after a mismatch or identify ambient observations with the differently typed global profile system. Complete materializer charges, fixed global ownership, full-minimum growth during saturation, quotient bounds, terminal-family derivation and global rank-decreasing route coverage remain open. No unconditional SaturatePositive, BCELReady or ZeroSlack, complete polynomial PCCMin, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Independent NAND materializer exact minimum-cost additivity | formalized-foundation-only | For every finite direct-wire circuit and every finite bank size, M240 physically appends one NAND for each disjoint pair of fresh Boolean inputs while retaining all original ordered outputs. The seven universal interfaces prove exact size and both output semantics, preservation of complete Boolean equivalence, a lower bound against every equivalent competing topology, exact reference-minimum additivity and unchanged physical residual slack. The original circuit need not be minimum, and its outputs may be constant, repeated or direct primary-input wires. The lower bound derives distinct actual bank-output gates from semantic output conditions, restricts fresh inputs to false and computes an erasure/rebinding that removes exactly those gates, including through retained consumers. The resulting implementation realizes the original function. Appending the bank to its attained minimum witness gives the matching upper bound. No correctness certificate or supplied minimum is an algorithm input. All seven interfaces are explicit-root built with their exact reviewed standard axiom closures. | This is an independent physical Boolean materializer forced-cost subcase, not transparency of every manuscript profile materializer or every actual saturation step. Fresh primary inputs and distinct appended NAND outputs are essential; repeating an existing output or reusing the original input pair does not justify additive cost. The result does not change or derive the full profile observer, global materializer ownership, quotient costs, obligation discharge, terminal families or complete rank-decreasing route coverage. Reference minima remain exhaustive finite constructions, not a polynomial-time algorithm. No unconditional SaturatePositive, BCELReady or ZeroSlack, complete polynomial PCCMin, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Support-independent physical ownership and exact materializer charges | formalized-foundation-only | For every finite candidate, arbitrary finite raw request family and terminal record list, M241 computes a first-requesting ambient owner before support selection, retaining unrequested physical gates in a fixed remainder bucket. Ten universal interfaces characterize the first requester and unrequested case, exact partition membership, disjointness and restriction stability, actual extracted gate counts, the complete integer charge identity, open semantics, induced-boundary reconnection and the whole-circuit charge total. Overlapping and duplicate requests require no supplied disjointness or coverage certificate. Each owned piece is an existing physical support extraction with its actual NAND size, not an arbitrary numerical weight. All ten interfaces are explicit-root built with their exact reviewed standard axiom closures. | This is a support-independent physical ownership and charge partition kernel, not the complete manuscript computational-record universe or proof of admissible materializer ownership. The raw finite request family is supplied data; a canonical assignment does not discharge unique active ownership, carrier, frontier or obligation conditions. The existing production nonunique-owner rejection is unchanged. Metadata records do not become free computational materializers or receive invented physical cost. The result does not establish HN assembly, forced full-profile minimum growth, quotient bounds, terminal-family derivation or complete rank-decreasing routes. Reference minima remain exhaustive finite constructions, not a polynomial-time algorithm. No unconditional SaturatePositive, BCELReady or ZeroSlack, complete polynomial PCCMin, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Constructive whole-program NAND sharing and checked normalization | formalized-foundation-only | For every finite direct-wire NAND program and complete ordered output word, M242 computes a full structural-sharing traversal, translating every source through earlier computed aliases and searching the actual retained program for equal or commuted NAND input pairs. Each successful lookup reuses one existing gate; each unsuccessful lookup appends one translated gate. Nine universal interfaces prove every alias value, exact retained-plus-fold gate accounting, complete multi-output equivalence, non-increasing physical size, invariant reference minimum, exact residual-slack savings, strict gain exactly when a fold occurs, strict residual descent and the computed normalizer outcome. The caller supplies only the implementation, not an optimizer or correctness certificate. The pass performs structural lookup, not semantic enumeration. All nine interfaces are explicit-root built with only propext and Quot.sound. | This is a computed physical R1/R4 structural-sharing stage, not complete manuscript N1-N10 normalization or a proof that every latent-sharing opportunity is found. A no-fold branch does not imply semantic minimality, absence of other gains or ZeroSlack; a checked regression gives a smaller equivalent circuit after the pass has no structural fold. The result does not construct a proper-support Package E ledger, arbitrary replacement pullback, full-profile carrier or obligation transport, terminal-derived families, global rank-decreasing route coverage or the complete PCCMin oracle. Reference minimum occurs in specification theorems, not in execution of this pass. No uniformly encoded-size polynomial execution theorem for the complete PCCMin construction is proved. No unconditional SaturatePositive, BCELReady or ZeroSlack, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Checked concrete SAT hardness in the active conditional final-report bridge | formalized-foundation-only | M243 connects the already checked all-input Cook-Levin NP-hardness and NP-completeness theorem to the exact report-facing SAT predicate. The existing accepted-generated-package consequence and final_report_bridge no longer take a CheckerTrustModel or supplied SAT-hardness argument. They consume the checked theorem while retaining the explicit PCCMinLoopCertificate, its concrete residual-band polynomial decider and the loop-existence antecedent. This strengthens the active conditional interface rather than adding an unused parallel route. Four reviewed interfaces have exact kernel types and the same standard axioms as the consumed M231 theorem: Classical.choice, Quot.sound and propext. | This is a conditional final-report bridge, not the eligible unconditional root theorem. No complete PCCMin loop certificate is constructed: the concrete residual-band decider remains an explicit certificate field. Canonical package acceptance does not supply missing algorithmic correctness, encoded-size bounds or loop existence. This does not establish unconditional SaturatePositive, BCELReady or ZeroSlack, a complete exact polynomial PCCMin construction, polynomial output and certificate bounds, deterministic CNFSAT in P or P = NP. M231 hardness was already credited; connecting it here earns no duplicate checkpoint points. No fixed weighted checkpoint or global gate closes. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Derived physical output cones and checked unused-gate pruning | formalized-foundation-only | For every finite direct-wire NAND candidate and complete ordered output word, M244 computes the least physical gate-predecessor closure of actual gate-valued outputs using the existing finite work-list saturation. It proves output inclusion, predecessor closure, leastness and absence of gate-valued external boundary wires. The existing checked support extractor is reused, its boundary is renamed to original primary inputs, and every ordered output position is reconnected, including constants and repeated wires. Eleven universal interfaces establish complete input/output equivalence, non-increasing physical gate count, exact retained-plus-deleted accounting, invariant semantic reference minimum, exact residual-slack savings, strict gain exactly when a gate is deleted and the concrete PCCMin normalizer outcome. The caller supplies only the implementation. All eleven interfaces are explicit-root built with only propext and Quot.sound. | This is a computed physical unused-gate pruning stage, not complete manuscript N1-N10 normalization or full-profile/metadata support derivation. The empty profile index describes only this physical closure; arbitrary full-profile observers may distinguish the pruned implementation. A no-deletion branch does not imply semantic minimality or ZeroSlack. The pass reuses executable closure and checked extraction; reference minimum occurs in specification theorems, not in execution. No proper-support Package E rewrite ledger, full-profile carrier/obligation transport, global terminal family, complete rank-decreasing route coverage or total PCCMin oracle is constructed. No uniformly encoded-size polynomial execution theorem for the complete construction is proved. No unconditional SaturatePositive, BCELReady or ZeroSlack, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed dead-support frames and proper physical deletion | formalized-foundation-only | For every finite direct-wire NAND implementation, M245 computes the physical complement of its output cone and proves that the actual extracted dead support has no outgoing interface. Its incoming wires are connected to original inputs or the computed live frontier; the live program also carries every original ordered output as bypass. The support is the actual extracted candidate, reindexed only by its proved empty interface. A concrete FramedContext inserts that support with the original physical gate count and plugs a zero-gate replacement with complete original output semantics. Twenty general interfaces establish exact boundary values, extraction identity, local and framed equivalence, retained-plus-dead gate accounting, exact residual-slack savings and a computed physical gain witness precisely when both dead support and live cone are nonempty. All twenty interfaces are explicit-root built with only propext and Quot.sound. No support, frame or correctness certificate is supplied by the caller. | This is a computed proper physical deletion component, not complete manuscript Package E admissibility or a complete N1-N10 rewrite ledger. Properness concerns the actual gate subset and concrete frame, not full-profile carrier and obligation compatibility. An all-unused circuit is not accepted as a proper-subset witness. Rejecting this route does not imply semantic minimality or ZeroSlack. The construction does not search reference minima; the reference minimum occurs only in specification theorems. No arbitrary full-profile preservation, complete Package E verifier, terminal-derived global family, complete rank-decreasing route coverage or total PCCMin oracle is constructed. No uniformly encoded-size polynomial execution theorem for the complete construction is proved. No unconditional SaturatePositive, BCELReady or ZeroSlack, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Full finite-profile and obligation acceptance for computed dead-support gains | formalized-foundation-only | For every finite direct-wire implementation and input finite profile observation system, M246 classifies the actual M245 proper dead-support replacement without an exhaustive physical support or reference-minimum search. It reuses the complete profile mismatch scan and adds a canonical obligation-role scan. No proper support, first full-profile mismatch, first open obligation and accepted full-mode result are distinct proof-bearing outcomes. Eight general interfaces prove exact obligation-scan completeness, first-failure prefixes, acceptance precisely when computed properness, complete profile equality and observed obligation discharge all hold, exact no-proper-support rejection, full-profile minimum invariance as a specification theorem, and branch soundness with physical equivalence, properness, exact size/slack savings and strict descent. Accepted results inhabit the existing full-carrier interface. All eight interfaces are explicit-root built with only propext and Quot.sound. No result, profile-invariance premise or correctness certificate is supplied by the caller. | This is computed acceptance against an input finite profile observation system, not derivation of the manuscript carrier or its semantic dependency graph. Checking observed obligation bits is not a constructed R5 creation or R6-R8 discharge ledger, and does not by itself establish complete Package E admissibility. Full-profile equality is not replaced by quotient equality; matching profiles with an open obligation are rejected. No-proper-support rejection excludes only this computed dead-support route, not every physical gain or a smaller semantic implementation. A mismatch or open obligation is explicit rejecting data, not a completed global routing theorem. The full-profile reference minimum appears only in a specification theorem; it is not computed by the classifier. No bound on the supplied observer computation or uniformly encoded-size polynomial runtime is established. No terminal-derived global family, complete N1-N10 normalization, full Package E verifier, total PCCMin oracle, unconditional SaturatePositive, BCELReady or ZeroSlack, deterministic CNFSAT in P or eligible root theorem is established. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed constant propagation through arbitrary NAND programs | formalized-foundation-only | For every finite direct-wire NAND program, M247 computes retained-source aliases for all original gates, substitutes them before inspecting each NAND operation, eliminates a gate exactly when literal constant NAND identities determine its value, and rewrites the complete ordered output tuple. Ten general interfaces prove primitive constant recognition sound for every valuation, alias semantics, exact retained-plus-eliminated gate accounting, complete multi-output equivalence, non-increasing size, invariant reference minimum, exact residual-slack savings, strict gain exactly when an elimination occurs, strict residual descent and the computed outcome of the existing total normalizer interface. Earlier constant eliminations propagate through arbitrarily later gates. The primitive recognition theorem is axiom-free; the other nine interfaces use only propext and Quot.sound. No optimizer, result, semantic equivalence certificate or caller correctness assertion is supplied. | This is the physical literal-constant structural-congruence component, not complete manuscript R1-R9 verification or N1-N10 normalization. No-elimination does not imply semantic minimality or ZeroSlack: the regression retains NAND(x, NAND(x,x)) even though a zero-gate constant-true word is equivalent. No full-profile preservation, derived manuscript carrier, arbitrary-support pullback/expansion, obligation lifecycle ledger, complete Package E or total PCCMin oracle is established. The reference minimum is used only in specification theorems, not executed by the pass. No uniformly encoded-size polynomial runtime, output-size or certificate-size theorem is established. Unconditional SaturatePositive, BCELReady and ZeroSlack, deterministic CNFSAT in P and the eligible root theorem remain open. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed closure of constant propagation, NAND sharing and output-cone pruning | formalized-foundation-only | For every finite direct-wire implementation, M248 computes a priority-respecting loop over the actual constant-propagation, NAND-sharing and output-cone-pruning constructions. It selects the first positive saving, recurses on strictly smaller physical gate count, and stops only when all three computed savings vanish on the same final implementation. Twelve general interfaces prove complete ordered-output equivalence, exact per-pass and telescoping trace savings, strict selected gains, priority, common quiescence, unchanged quiescent inputs, idempotent re-execution, invariant semantic reference minimum, exact residual-slack savings and a gain-iteration bound by the original slack. The existing normalizer interface exposes every positive total saving as a strict gain. All proof fields are derived by the construction, with no supplied normalizer, oracle, result or stopping certificate. The twelve interfaces use only propext and Quot.sound. | This is operational quiescence for exactly three physical passes, not semantic minimality, all structural congruences, complete manuscript N1-N10 normalization or ZeroSlack. The regression remains quiescent on NAND(x, NAND(x,x)) while a zero-gate constant-true word is strictly smaller and equivalent. The trace is not the arbitrary-support Pull/Expand materializer transport required by the complete Traceable normalization theorem. Full-profile preservation, the manuscript carrier and R5/R6-R8 obligation lifecycle ledger, complete Package E and the total PCCMin oracle remain open. Semantic reference minima occur only in specifications and are not executed by this loop. An iteration bound is not a runtime bound: no uniformly encoded-size polynomial construction, runtime, output-size or certificate-size theorem is established. Unconditional SaturatePositive, BCELReady and ZeroSlack, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed arbitrary-support literal NAND replacement and topological compilation | formalized-foundation-only | For all finite raw NAND graphs, M249 computes a topological order from the actual source fields or rejects the unresolved cyclic remainder. The ordered program has one position for every original node, exact size and complete output semantics for every solution of the original equations. For arbitrary finite terminal support records, it then constructs the actual literal splice from the canonical exterior gates, computed boundary and interface, replacement word and every original ordered output. Every accepted splice has exactly the exterior size plus replacement size; an equivalent replacement preserves all outputs, and a strict local physical saving yields a strict whole-program saving. Actual production saturation derives a primary-input-only boundary and hence unconditional compilation success, without a supplied frame, order, wiring map or acyclicity certificate. Its result agrees in size and semantics with the existing production physical constructor. Twenty-four interfaces have exactly reviewed dependencies contained in propext and Quot.sound. | Arbitrary raw port compatibility and equal open Boolean functions do not by themselves guarantee acyclic literal wiring. The regression gives an equivalent replacement with a raw graph solution but a genuine dependency cycle, which the compiler rejects. This is a physical substitution and well-formedness result, not a contradiction of the manuscript's unconstructed complete carrier premises. Full-profile preservation, the manuscript carrier and R5/R6-R8 obligation lifecycle, arbitrary-support Pull/Expand materializer identities, complete Package E and complete normalization remain open. No semantic minimum is computed, no unconditional global route is supplied, and a finite node-count termination argument is not a total encoded-size polynomial runtime theorem. Unconditional SaturatePositive, BCELReady and ZeroSlack, exact polynomial PCCMin, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global gate closes; P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed wire-backed carrier preservation under physical normalization and replacement | formalized-foundation-only | For all finite implementations with an ordered tuple of literal computational field sources, M250 exposes ordinary outputs followed by every field without adding NAND gates. Exact pack/unpack identities retain the program and ordered sources. The actual three-pass physical normalizer preserves every ordinary output and field value, has exact trace accounting, reaches common quiescence and is idempotent. Every selected field producer belongs to the computed support interface, even when no ordinary output uses it. The actual arbitrary-support splice compiler preserves both observation classes under equality of the complete extracted open function, counts every exterior and replacement gate, transfers strict local savings and rejects cyclic literal wiring. Production predecessor closure is derived from the combined physical program without a supplied observer, order, map or successful result. Sixteen interfaces have exactly reviewed dependencies contained in propext and Quot.sound. | This represents and transports wire-backed computational fields; the wire bindings are actual input data, not a derivation of the complete manuscript carrier or every noncomputational record, role and history. Proof-only records do not become Boolean sources. The empty additional abstract profile in the physical specialization does not assert that the full manuscript profile is empty. R5/R6-R8 obligation creation and discharge, arbitrary-support Pull/Expand materializer identities, complete Package E and N1-N10, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact PCCMin and total encoded-size polynomial bounds remain open. Equal local Boolean functions do not guarantee acyclic arbitrary literal splicing; the compiler can reject them. No semantic minimum is executed, no fixed weighted checkpoint or global gate closes, deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed lost-wire projection obligations and fully charged restoration | formalized-foundation-only | For every finite wire-backed carrier and finite keep mask, M251 computes the actual projected NAND word and one shared materializer for all forgotten computational fields. Literal common-input concatenation restores every ordinary output and every ordered field for all valuations, with exact projected-plus-materializer physical gate charge and an output gate-count bound of twice the original. The mask derives exactly the forgotten coordinates, each with an original-source-bound R5 creation and actual restored-source-bound full-value R8 discharge. The computed independent event ledger creates and discharges every lost coordinate once and closes under replay; whole-trace identity validation rejects reusing an already discharged identity. Gain detection pays the complete materializer before requiring a strict decrease. No observer, restoration program, discharge trace or correctness certificate is supplied. Twenty-two interfaces have exactly reviewed dependencies contained in propext and Quot.sound. | This is the computational lost-wire slice of the manuscript's projection and R5/full-R8 requirements, not the full carrier or complete finite-kernel rewrite and obligation calculus. The keep mask is an input, not a derived global route. Noncomputational records, R6/R7 cancellation, arbitrary obligation-dependency DAGs, complete Package E and N1-N10, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact PCCMin and total encoded-size polynomial runtime remain open. Quotient padding is not full-field agreement. Restoration need not be a net gain or a semantic minimum. The twice-original gate bound is only an output-size fact, not a total polynomial execution or certificate-size theorem. No fixed weighted checkpoint or global gate closes, deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed quotient replacement lift with matched materializer costs | formalized-foundation-only | For every finite computational wire carrier, finite keep mask and replacement satisfying local quotient agreement on all ordinary outputs and every kept field, M252 constructs the literal replacement-plus-shared-materializer word. All ordinary outputs and all ordered computational fields agree with the original for every valuation. The same derived materializer gives exact natural gate charges and matched integer cost differences for the reference lift and the replacement expansion. Cancellation transports a saving only against the lifted reference; an original-circuit gain separately requires the full replacement-plus-materializer cost to be strictly smaller than the original. Lost-field discharges bind the original R5 source to the actual expanded source and derive full-value witnesses without any agreement about the replacement's forgotten fields. The computed gain test retains the local quotient premise and returns a complete equivalent strict gain with every field preserved. Seventeen general interfaces have exact reviewed dependencies propext and Quot.sound. | This is a computational word-level quotient lift, not arbitrary-support Pull/Expand or an embedding of the reference lift into the original circuit. The keep mask, replacement and local quotient agreement are inputs; the construction does not discover replacements or derive that local agreement. The lifted reference can exceed the original gate count, and a relative saving can leave no original gain. It does not complete the manuscript carrier, R6/R7 and general obligation-dependency DAG calculus, N1-N10, Package E, global routing, unconditional SaturatePositive, BCELReady or ZeroSlack, exact PCCMin or total encoded-size polynomial construction, runtime, output and certificate bounds. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed original-accounted frontier replacement | formalized-foundation-only | For every finite computational wire carrier and keep mask, M253 computes the predecessor cone of ordinary and kept observations, then extracts its complete interface from the original fully exposed carrier. Selected forgotten field producers and exterior consumers participate in that frontier. Predecessor closure derives a primary-input-only boundary and actual replacement compiler success without a supplied support, order or success certificate. Every original exterior gate is retained exactly once. Original and expanded gate counts split into pulled or replacement gates plus the same actual exterior charge, giving matched integer differences and an exact original-saving equivalence. Equality with the complete extracted open function derives all ordinary and ordered computational field values and full discharges bound to the actual expanded sources. The computed proper-gain query separately requires proper support and strict local saving, then returns an actual original-circuit equivalent strict gain with every field preserved. Eighteen general interfaces have exact reviewed dependencies propext and Quot.sound. | This is a computational frontier lift for one input-derived visible predecessor cone, not arbitrary-support Pull/Expand across all manuscript traces. The keep mask, replacement and complete local open-function agreement are inputs. Ordinary-output or quotient-only agreement does not imply the stronger completed-frontier premise. No replacement search or automatic local agreement is proved. A whole-support saving is not a proper-support certificate. The construction does not complete the full manuscript carrier, noncomputational profile fields, R6/R7 and general obligation-dependency DAG calculus, N1-N10, Package E, global routing, unconditional SaturatePositive, BCELReady or ZeroSlack, exact PCCMin or total encoded-size polynomial construction, runtime, output and certificate bounds. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed full-mode wire cancellation | formalized-foundation-only | For every finite computational wire carrier and keep mask, M254 computes canonical retained representatives by literal source identity in the original ordinary outputs and kept fields. The precise local quotient-agreement premise derives each matched forgotten wire's full value in the actual replacement. A computed resolved mask selects one actual shared materializer for the remaining unresolved wires, with an explicit zero-gate case when all fields resolve. The expanded word preserves every ordinary output and ordered computational field, and has exactly replacement gates plus its actual materializer charge. Computed R6 cancellations and R8 restorations carry full-value witnesses bound to that expanded source. Their generated R5/discharge ledger covers exactly the forgotten coordinates, closes with no pending obligations and rejects creation-identity reuse across the whole transcript, including already discharged entries. The fully paid whole-word gain query compares the actual expansion with the original gate count. Twenty-six general interfaces have reviewed propext-only or propext-and-Quot.sound dependencies. | This is an original-source-identity computational R6 case, not every semantic cancellation or the complete manuscript obligation calculus. The keep mask, replacement and precise local quotient agreement are inputs; forgotten-field equality, supplied representatives, materializer weights and global correctness certificates are not. Syntactically different but semantically equal wires may remain unresolved. No universal comparison with independently normalized earlier materializers is proved. A whole-word strict gain is not a proper-support Package E certificate. The full manuscript carrier, noncomputational profile fields, R7 and arbitrary dependency DAGs, all-trace N1-N10 normalization, complete Package E, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact PCCMin and total encoded-size polynomial construction, runtime, output and certificate bounds remain open. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed unary full-word realization | formalized-foundation-only | For every unary computational wire carrier, with arbitrary original gate count, ordered ordinary outputs and computational fields, M255 derives both unary observation values from the actual source and constructs a complete zero-or-one-gate realization. Constants and the input cost zero gates; every negated observation shares the same actual NOT gate. The result preserves every ordinary output and computational field at every unary valuation. Its exact gate formula is minimal among all equivalent full computational carriers: a zero-gate source cannot negate the only boundary input. The constructor accepts no replacement, truth table, local agreement or minimizer. Source-exact R7 discharge witnesses for original R5 identities refer to the actual computed program and the original full field values. The whole-word strict-gain query succeeds whenever any equivalent full unary carrier is smaller. Eighteen general interfaces have reviewed axiom closures: five are axiom-free and thirteen use only propext and Quot.sound. | This is a complete unary computational word realization component for manuscript R7, not an arbitrary ambient-cut embedding, every R7 case or the complete manuscript obligation calculus. Minimum size is relative to all exposed computational observations, not ordinary outputs alone. A whole-word strict gain is not a proper-support Package E certificate. The keep mask and source-bound R5 identity are used only by the discharge interface; no supplied full-value agreement is a construction premise. There is no enumeration of implementations, but evaluating two unary inputs with recursive Program.eval does not prove polynomial encoded runtime. The full manuscript carrier, noncomputational profile fields, arbitrary obligation dependency DAGs, all-trace N1-N10 normalization, complete Package E, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and total encoded-size polynomial construction, runtime, output and certificate bounds remain open. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed constant and unary frontier replacement | formalized-foundation-only | For arbitrary computational wire carriers, original gate counts, ordered ordinary outputs, computational fields and keep masks, M256 computes the visible predecessor cone and its completed full frontier. When the actual incoming boundary has zero or one port, it derives a zero-gate constant word or the minimum complete unary word, preserving every local output at every open valuation. No replacement, truth table, semantic agreement, wire map, compilation order, compiler result or minimizer is supplied. The actual primary-boundary compiler substitutes this word into the original exterior, retained exactly once. Every ordinary output and computational field is preserved, including forgotten selected frontier fields and exterior fields depending on other ambient inputs. The result has exact replacement-plus-exterior gate count and cannot increase the original count. Source-bound R7 witnesses discharge original R5 identities with their full values in the actual expanded program. The proper-gain query computes boundary recognition, positive actual exterior and strict local saving separately; it succeeds whenever any complete local realization is smaller on such a proper recognized cone. Thirty general interfaces have reviewed axiom closures: two are axiom-free, one uses only Quot.sound and twenty-seven use only propext and Quot.sound. | This is source-derived constant/unary R7 realization for the computed visible predecessor cone, not every arbitrary ambient support or an external-gate boundary. Minimum size is relative to the complete fixed local frontier, not all ambient circuits with different exteriors. A whole-support saving is not a proper-support Package E certificate. More-than-unary boundaries are rejected rather than supplied with correctness certificates. Finite source evaluation and actual compiler termination are not encoded-size polynomial runtime theorems. The full manuscript carrier, noncomputational profile fields, arbitrary obligation dependency DAGs, every R5-R8 interaction, all-trace N1-N10 normalization, complete Package E, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and total encoded-size polynomial construction, runtime, output and certificate bounds remain open. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed constant and unary replacement on arbitrary completed supports | formalized-foundation-only | For arbitrary finite computational wire carriers, original gate counts, ordered ordinary outputs, computational fields and finite terminal-record supports, M257 derives the complete boundary and frontier from the actual exposed program. Every actual zero- or one-port boundary has a computed minimum complete local word with at most one shared NOT gate. General open-evaluation prefix causality proves that a selected frontier gate before the sole external boundary gate is represented by a literal constant. A rank derived from original gate indices then proves well-foundedness and successful compilation of this actual replacement, without a supplied rank, acyclicity certificate, order, agreement, replacement program or compiler result. Empty and primary-input boundaries are covered as well as a sole external-gate boundary. The actual exterior is retained exactly once. Every ordinary output and computational field is preserved at every ambient valuation, with exact replacement-plus-exterior charge, nonincrease and equivalent strict local-to-global saving. Source-exact full-value R7 witnesses discharge the original R5 identities. The proper-gain query computes boundary recognition, positive exterior and strict local saving, and succeeds whenever any complete local realization is smaller on a recognized proper support. Thirty-one general interfaces have reviewed axiom closures: one is axiom-free, one uses only Quot.sound and twenty-nine use only propext and Quot.sound. | Arbitrary support refers to the choice of finite physical support in a computational wire carrier, not arbitrary boundary width or the full manuscript carrier. Boundaries with more than one incoming port are rejected. The minimum is over complete local open realizations for the fixed extracted frontier, not over all ambient circuits or different exteriors. A whole-support saving is not a proper-support Package E certificate. Prefix causality and the source-derived rank prove termination of this compiler, not uniformly polynomial encoded-size execution. Noncomputational profile fields, arbitrary obligation dependency DAGs, every R5-R8 interaction, all-trace N1-N10 normalization, complete Package E, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and total encoded-size polynomial construction, runtime, output and certificate bounds remain open. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed complete search for proper constant and unary R7 gains | formalized-foundation-only | For arbitrary finite computational wire carriers, original gate counts, ordinary outputs and full computational fields, M258 computes a complete family of proper zero/unary support-gain candidates from the actual program. Optional boundary choices use only source wires actually consumed by gates, plus no boundary. For each choice and omitted gate, a topological maximal scan derives a support with at most one actual incoming wire and a nonempty exterior; canonical singleton supports handle one-gate savings. Any arbitrary proper zero/unary support is contained in the corresponding maximal candidate, and extraction is invariant under repeated or metadata records selecting the same gates. A support with at least two gates yields a strict saving because the complete zero/unary frontier word uses at most one shared NOT; a one-gate saving transfers to its canonical singleton. The carrier-only executable search is therefore complete whenever any proper zero/unary support admits a strictly smaller equivalent complete local word, without a supplied support family, replacement, rank, order, compiler result or completeness certificate. An accepted result constructs the actual expanded carrier, preserves every ordinary output and computational field at every valuation, charges the original exterior once, gives a strict fully paid saving and retains source-exact full-value R7 discharges for original R5 identities. A negative result excludes every such gain in this exact class. With g original gates, the | Completeness is for proper physical supports with zero or one actual incoming boundary in a computational wire carrier. It is not completeness for all boundary widths, every R7 case or all ambient circuit optimizations. A negative search result is not global minimality or unconditional ZeroSlack; a guarded two-boundary duplicate fixture has an explicit smaller global realization while this scoped search correctly returns no gain. A whole-support saving is not a proper-support Package E certificate. Boundary-choice enumeration excludes unused declared inputs, but inherited physical-port extraction and compilation retain their own execution costs. The physical candidate and record counts are not a theorem of total uniformly polynomial encoded-input-size execution, output size or certificate size. The full manuscript carrier, noncomputational profile fields, arbitrary obligation dependency DAGs, every R5-R8 interaction, all-trace N1-N10 normalization, complete Package E, global routing, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and its complete polynomial bounds remain open. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed full-field zero/unary descent and normalization closure | formalized-foundation-only | For every finite computational wire carrier, M259 computes a whole-span zero/unary saving branch and combines it with the complete proper-support search. The whole-span branch selects all physical gates, has zero exterior charge and is kept distinct from proper-support R7. The combined carrier-only search is complete whenever any support with zero or one actual incoming boundary admits a strictly smaller equivalent complete local word; neither a support family nor a replacement or completeness certificate is supplied by the caller. Every accepted branch builds the actual smaller carrier and preserves all ordinary outputs and every computational field at every valuation, with exact original-gate accounting. A well-founded executable closure repeatedly performs physical normalization, searches for a proper or whole-span gain, expands an accepted gain and restarts normalization. Its derived trace proves termination, full-field preservation, common physical quiescence and absence of every gain in this scoped zero/unary class. It proves exact gate savings, bounds normalization and gain iterations by the retired gates, bounds search calls including the final negative call, and proves output idempotence. Semantic reference minima occur only in specifications: their invariance yields exact residual-slack retirement and corresponding iteration and search-call bounds, without reference-minimum enumeration in the algorithm. | The stopping result is restricted to physical normalization and zero/one-actual-boundary computational gains. It is not global minimality: a guarded physically quiescent common fixed point has an explicit smaller equivalent carrier and strictly positive global residual slack. Whole-span descent is not relabelled as a proper-support Package E certificate. Preserving every computational field does not reconstruct the full manuscript carrier, noncomputational profile fields or arbitrary obligation dependency DAGs. Wider boundaries, every R5-R8 interaction, all-trace N1-N10 normalization, complete Package E, global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete encoded-size polynomial runtime, output and certificate bounds remain open. A gate-decrease or search-call bound is not a total polynomial execution theorem; inherited extraction, compilation and candidate-search costs still require their own bounds. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-bounded ordered physical boundary extraction | formalized-foundation-only | For arbitrary finite program dimensions and any list of terminal primitive records, M260 computes the incoming physical boundary from nonconstant sources of actual gates. It filters those occurrences with the existing physical crossing predicate, removes duplicates and sorts by the canonical primary-input-before-gate-output coordinate. Every boundary crossing is derived from an actual gate source. The resulting list is proved exactly equal, including order and multiplicity, to the former ambient-wire filtered reference. Source occurrences and the emitted boundary each have length at most twice the physical gate count, independent of unused declared input slots. The active terminalBoundaryPorts implementation uses this source-driven construction. Public reference, length, duplicate-free and order theorems preserve the existing physical extraction and downstream support interfaces. A generic finite-list canonicalizer proves exact equality with any ordered duplicate-free reference under an injective coordinate map, without enumerating the ambient type. Guarded execution exercises exact small reference lists and sparse programs with a billion declared inputs; those large fixtures do not execute the ambient reference. | Occurrence and output bounds are structural size bounds, not total encoded-input-size polynomial execution theorems. The reference equality preserves the existing physical semantics; it does not establish a new global minimization result. The old ambient enumeration remains only as a theorem-side specification, never the active boundary implementation. No caller-supplied boundary or completeness certificate is introduced. The full manuscript carrier, arbitrary obligation dependency DAGs, every normalization and gain interaction, complete Package E and global route coverage remain open. These results do not prove unconditional SaturatePositive, BCELReady or ZeroSlack, exact general PCCMin, or total polynomial runtime, output and certificate bounds for the complete construction. Runtime execution is regression evidence, not theorem authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Context-aware nested open-support and four-corner transport | formalized-foundation-only | For arbitrary finite candidate dimensions, terminal record lists and ordinary selected-gate inclusions, M261 computes a smaller open boundary valuation from the larger support evaluator. A wire internalized in the larger support receives its computed gate value, not an independent ambient bit. For every valuation on the larger open boundary, selected gates and retained interface producers have exactly the same Boolean values after this substitution. The complete valuation maps satisfy identity and three-support composition. Actual support-square inclusions derive the four leg maps, and both join-to-meet paths agree as valuation functions. Retained output values agree for extracted implementations, arbitrary valid full realizations and the actual canonical full and quotient local-minimum families. These semantic maps come from the original open carrier, not invented internal gates of a minimum realization. The existing unconstrained-ambient coherence classifier and its exact mismatch results are preserved as a different relation. | The theorem covers arbitrary larger open-boundary valuations, not only whole-circuit executions, and introduces no caller-supplied transport or correctness certificate. Retained Boolean-output transport does not prove equality of arbitrary implementation-dependent observers or profiles, physically glue minimum circuits, or establish coherent charge and obligation ownership. Canonical finite reference minima supply specification-level comparison objects, not an efficient executable minimizer. The full manuscript carrier, arbitrary obligation dependency DAGs, complete Package E and global route coverage remain open. This result does not prove unconditional SaturatePositive, BCELReady or ZeroSlack, exact general PCCMin, or total encoded-input-size polynomial runtime, output and certificate bounds for the complete construction. Runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Total source-ordered charged arbitrary-support expansion | formalized-foundation-only | For arbitrary finite candidate, support, replacement and computational-field dimensions, M262 derives an actual acyclic raw NAND graph and executable compiled expansion from source data. Each exterior gate is retained once and each distinct retained physical interface producer owns one complete replacement copy. Primary and earlier gate boundary ports retain actual values; only later gate-valued ports are deliberately masked, and a computed source rank decreases along every actual edge. Compilation requires no supplied rank, order, semantic or compiler-success certificate. Under complete local open-function equality, masking preserves the selected output for every open boundary valuation, the derived graph valuation satisfies every NAND equation, and every original ordered output and computational field retains its full value. Repeated literal references share the same actual compiled source. Exterior and copy ownership are injective and disjoint and cover every emitted gate. Existing R5 creations retain their coordinate and original full source value. Actual cost is E + K * R and strict original-size saving is equivalent to K * R < S, with properness S < G separately required. | This is a source-derived physical construction at arbitrary finite widths, not matched-kappa Pull/Expand or unrestricted CompatibleReplacement/SlackLaw from merely R < S. Full local open-function agreement is required for semantics; quotient-only or ordinary-output-only equality does not suffice for full computational fields. Distinct equal-valued physical producers are not merged, and repeated physical copies are not free. The inherited cyclic literal splice remains rejected; existing cone and unary constructions retain their own bounds. Literal R5 binding transport is not the complete R5–R8 event calculus or an arbitrary semantic obligation DAG. Arbitrary implementation-dependent observers, profile histories, the full manuscript carrier, complete Package E, global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and total encoded-input-size polynomial runtime, output and certificate bounds remain open. Runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed dependency-ordered physical obligation histories | formalized-foundation-only | For arbitrary finite input, output, computational-field and event dimensions, M263 computes a complete dependency order from raw event identities, explicit predecessors and intrinsic creation references. Duplicate identities, missing references and cyclic graphs reject. The scheduler succeeds exactly when the actual finite dependency relation is well-founded. The history constructor executes one evolving physical carrier from the original source, without caller-supplied order, rank, state, full-value witness or charge data. R5 captures the actual pre-drop carrier; physical normalization preserves open snapshots; source-identity R6 computes a visible representative; R8 appends and charges the complete materializer from the matching creation snapshot. Full reads require an available field, and final open obligations reject. Every successful history executes every event exactly once in dependency order, every creation discharges strictly later with its full source binding, and every ordinary output and computational field retains its original value for every valuation. Final physical gates plus actual normalization removals equal initial physical gates plus all appended materializer charges. | This is the computational R5/R6/R8 history component, not the complete manuscript obligation calculus or Package E. Acyclicity guarantees a computed order, not a valid lifecycle or confluence of underspecified event graphs. R5 projection alone removes no physical gates; repeated restorations receive no free cross-snapshot sharing, and restoration can increase physical size. Quotient-only agreement cannot discharge a full obligation. Full R7 and other R1-R9 semantics, all N1-N10 transports, arbitrary observers and profile histories, noncomputational carrier records, matched-kappa Pull/Expand, complete Package E, global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete encoded-input-size polynomial runtime, output and certificate bounds remain open. Runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-derived closed-history literal arbitrary-support replacement | formalized-foundation-only | For arbitrary finite candidate, support and raw-event dimensions, the constructor extracts actual computational fields with zero duplicate ordinary outputs, executes the existing closed R5/R6/R8 history, and derives literal-splice acyclicity from original gate labels and actual causal bounds. The current carrier and every pending snapshot preserve those bounds through actual normalization, source-identity cancellation and paid captured restoration. The existing literal graph compiler then succeeds for every accepted closed history without a caller-supplied replacement, rank, order or semantic certificate. All ordered original outputs are preserved; every exterior gate occurs once; final gates plus actual removals equal original gates plus all actual materializer charges. Strict gain follows when removals exceed charges. The splice stage introduces no rejection beyond the actual history compiler. | This covers the existing computational R5/R6/R8 language and three-pass physical normalizer, not all manuscript R1-R9 or N1-N10 rules, full R7, arbitrary observers, noncomputational carriers, full-profile compatibility, matched-kappa Pull/Expand or complete Package E. The support records and raw history remain input data. It does not derive terminal families, a globally successful rewrite strategy, global route coverage, unconditional SaturatePositive, BCELReady or ZeroSlack, exact general PCCMin or complete encoded-input polynomial runtime/output/certificate bounds. Boolean equivalence alone is not a causal certificate. Runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes; deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-derived R7 discharge in raw computational histories | formalized-foundation-only | For arbitrary finite computational carriers, support lists and raw-event dimensions, R7 decodes every gate, boundary and interface coordinate against the immutable carrier of the identified open creation. Whole lists round-trip without dropping or substituting coordinates. Recognition succeeds exactly when decoding succeeds and the actual completed boundary has at most one port. The replacement is computed from that source's complete zero/unary open function, never from a supplied implementation, truth table or correctness certificate. Labelled dependency bounds follow the actual topological compiler and the literal replacement, not Boolean equivalence alone. The resulting full-field materializer discharges the exact captured creation, charges every appended gate, preserves ordinary outputs and other pending snapshots, and preserves the current-and-snapshot causal invariant. The extended raw R5/R6/R7/R8 history retains full lifecycle closure and exact physical accounting. Every accepted closed history still admits the literal one-copy ambient splice without an additional supplied rank, order, agreement or successful-compilation premise; strict gain requires actual removals to exceed all charges. | This is the zero/unary R7 extension of the computational history language, not the complete manuscript R1-R9 or N1-N10 calculus, arbitrary observers, noncomputational carriers, full-profile compatibility, matched-kappa Pull/Expand or complete Package E. Support coordinates and raw events remain input data; the result does not derive terminal families or a globally successful rewrite strategy. Global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete encoded-input polynomial runtime, output and certificate bounds remain open. Boolean soundness alone does not bound syntactic dependencies. Materialization can increase physical size, and finite runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-derived physical ownership through computational histories | formalized-foundation-only | For arbitrary finite computational carriers, selected support lists and raw-event dimensions, physical origins follow the actual constant, sharing and cone compiler branches and compose through the existing normalization trace. Every initial gate retains its source coordinate; every R7/R8 appended gate is labelled by its executing event identity and local allocation coordinate. Creation, cancellation and reads allocate no gates. The complete closed history computes a duplicate-free partition of live and removed origins into original gates and all historical charges, including allocations later removed. The actual topological compiler's computed two-sided position inverse carries those labels into the literal ambient splice, with every exterior gate present once and extracted coordinates restored to ambient positions. A source-only ownership constructor returns exactly the existing constructor's result. Its disjoint raw-event requests and fixed original-gate remainder reuse the existing ownership kernel for support-stable membership, actual extracted piece sizes and charge identities, independent open semantics and induced-boundary reconnection. No owner family, provenance map, partition proof, charge amount, topological order or successful-splice certificate is supplied. | This is source-derived physical ownership for the existing computational history language, not the complete manuscript carrier/profile universe, all R1-R9 or N1-N10 routes, arbitrary observers, matched-kappa arbitrary-support Pull/Expand or complete Package E. Support coordinates and raw events remain input data; no terminal-derived family or globally successful history is constructed. Historical allocated size and surviving owned size are distinct, and physical identity or integer accounting does not prove a runtime bound. Global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete encoded-input polynomial runtime, output and certificate bounds remain open. Finite runtime fixtures are regression evidence, not proof authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Persistent physical ownership through descendant histories | formalized-foundation-only | For arbitrary finite source dimensions and raw stage/event lists, a source-only decoder and compiler traverse the complete list using each actual preceding descendant. Persistent physical origins relabel original positions through the existing literal ambient compiler and label allocations by computed stage position, executing raw event and local gate. The arbitrary-program theorem conserves original gates and every historical charge as a duplicate-free live/removed permutation with exact actual execution totals; later removal never erases an earlier charge. Accepted input-event keys are distinct across stages, even when local event numbers are reused, and every historical or surviving allocation traces to that raw family. Derived final owner requests are disjoint before support selection and reuse the existing extraction kernel for support-stable ownership, exact piece sizes and charges, open semantics and induced reconnection. Whole-program Boolean semantics and physical size accounting yield an optional final StrictEquivalentGain through a computed final-size comparison, permitting intermediate expansion and propagating every rejected stage. No intermediate implementation, owner family, provenance map, charge amount, rank, successful-history certificate or semantic oracle is supplied. | This covers arbitrary finite sequences of the existing closed computational-history compilations, not the full manuscript carrier/profile universe, cross-support transport of open obligations, all R1-R9 or N1-N10 rules, matched-kappa arbitrary-support Pull/Expand, proper-support VerifyDW, full ChargeSoundness or complete Package E. Raw supports and events remain inputs; no terminal-derived family or globally successful strategy is constructed. Final net gain does not require each stage to decrease, prove local minimality after rejection, bound temporary growth or establish encoded-input polynomial runtime. Global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin and complete polynomial runtime, output and certificate bounds remain open. Runtime fixtures are regression evidence, not theorem authority. No fixed weighted checkpoint or global gate closes. Deterministic CNFSAT in P and the eligible root remain absent, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Source-only proper-support certificates for descendant programs | formalized-foundation-only | For arbitrary finite computational wire-carrier dimensions, raw outer support records and raw descendant programs, source-only verification decodes every outer coordinate and executes the complete local program on the extracted source. Arbitrary-label dependency bounds follow actual execution and derive literal outer compilation without a supplied wiring order or successful-splice premise. Acceptance is exactly complete decoding, successful complete local execution, proper physical support and strict final local saving. The constructed one-copy ambient splice preserves all ordinary outputs and literal fields, retains the actual run and compiler receipts, and satisfies exact historical charge/removal accounting. Intermediate expansion is permitted; invalid coordinates, a failed later stage, whole support and final nondecrease reject. Every accepted certificate yields StrictEquivalentGain and strict residual descent without supplied intermediate circuits, correctness, owners, costs, ranks or semantic oracles. | This verifies offered finite programs in the existing closed computational wire-carrier language. It does not prove that every nonminimal input has an accepted certificate, construct a globally successful strategy, derive terminal families or establish local minimality after rejection. Full manuscript profiles, all R1-R9 and N1-N10 rule families, cross-support transport of open obligations, matched-kappa arbitrary-support Pull/Expand, full manuscript VerifyDW, ChargeSoundness and Package E remain open. No bound on temporary growth, encoded-input polynomial runtime, output size or certificate size is proved. Global route coverage, unconditional SaturatePositive, BCELReady and ZeroSlack, exact general PCCMin, deterministic CNFSAT in P and the eligible root remain open. Runtime fixtures are regression evidence, not theorem authority. No fixed weighted checkpoint or global gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Constructive wire-profile restoration cost and exact gain boundary | formalized-foundation-only | For arbitrary finite input, ordinary-output and computational-field widths, the existing concrete quotient agreement is equivalent to wire-profile quotient comparison. The actual shared hidden-wire materializer restores the computed quotient-minimum witness into a full-equivalent carrier, preserving every ordinary output and actual field at every input valuation. If F is the full minimum, Q the quotient minimum, C the actual materializer charge, D = F - Q and S the actual full-field normalization saving, the theorems prove F <= Q + C, D <= C, paid cost = F + (C - D), S <= C - D and normalized cost = F + (C - D - S). Strict improvement against the original carrier is equivalent to the remaining overhead being smaller than its full slack; the executable acceptance test checks the actual final size and yields a sound strict equivalent gain. The existing complete one-input constructor has gate count exactly F for arbitrary output and field widths, and improves the original exactly when full slack is positive. No full replacement, minimum, charge or correctness certificate is supplied to the constructed route. | This is a cost and compatibility interface for computational wire profiles, not complete manuscript profiles or a terminal-derived family. The materializer charge only upper-bounds projection defect; equality and forced-cost transparency are not established. Physical normalization is not a semantic minimizer. A kernel-checked positive-slack example is refused by restoration plus normalization, while the existing unary route recovers it; this limits that component, not every route or the manuscript's complete route family. Returning no gain does not establish ZeroSlack. Unary completeness is restricted to one input and is reused, not newly proved for arbitrary inputs. Reference minimization is exhaustive, not a polynomial PCCMin algorithm. Complete Package E, global route coverage and rank decrease, unconditional SaturatePositive, BCELReady and ZeroSlack, exact polynomial PCCMin, encoded runtime, output and certificate bounds, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global proof gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed wire-profile models and exact independent-field cost | formalized-foundation-only | For arbitrary finite input, ordinary-output and computational-field widths, target-relative availability computes one actual constant, input or gate source that realizes a field uniformly over every input valuation. Rebinding constructs a carrier without adding gates and identifies terminal full and quotient profile minima with the corresponding wire-profile minima. The model constructor proves coherence between its base and ambient observers under unused-input padding and output rewording. A source-derived field seed and physical dependency closure yield an actual extracted support preserving every field at all ambient input valuations. For any old implementation with semantic minimum F and any natural width k, an explicit extension appends k independent NAND fields on disjoint fresh input pairs. Semantic gate retraction proves that every equivalent implementation needs at least F + k gates; the matching construction attains that bound. Its actual wire profile therefore has full minimum F + k, all-forgotten quotient minimum F, projection defect k and unchanged full slack. Three general coupling theorems establish these same exact minima and their difference inside the constructed terminal profile model, without a supplied model, minimum, field support or correctness certificate. | This is a computed model for actual computational wire fields, not the complete manuscript profile grammar or a terminal-derived governed family. The extracted support is field-preserving but is not asserted to be proper, smaller or optimal. The exact additive cost theorem concerns the explicit independent fresh-input NAND family, not arbitrary correlated, duplicated or supplied manuscript fields. The semantic retraction helper has an explicit uniform constant-value hypothesis, discharged for that family; it is not a general polynomial semantic-constant detector. The existing shared materializer charge is only bounded below by the family's projection defect, not proved equal to it. Availability checks enumerate valuations and reference minima remain exhaustive; no complete polynomial minimizer follows. Complete Package E, global route coverage and rank decrease, unconditional SaturatePositive, BCELReady and ZeroSlack, exact polynomial PCCMin, encoded runtime, output and certificate bounds, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global proof gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed closed-support observations and profile-compatible squares | formalized-foundation-only | For arbitrary finite input, ordinary-output and computational-field widths, the actual computed dependency-closed support observes a field exactly when it retains an original constant, input or gate source whose value agrees with that field uniformly over every input valuation. The matching table and support are computed from the circuit and ordinary seed data. For arbitrary seed pairs, observations of the computed union are the Boolean union of the separate observations. This extends to arbitrary finite families, with the empty support's observation as the base, and to seed-inclusion monotonicity. Seeding the requested profile records derives availability and uniform field-value preservation without injecting their literal original gate bindings. The actual meet, left, right and join of the constructed support square all preserve those requested computational fields and agree pairwise on their values at every ambient input valuation. Existing source extraction, structural square laws and arbitrary-context profile locality are reused, not newly claimed. | This is computational-field compatibility, not ordinary-output equivalence, the complete manuscript profile grammar, a terminal-derived governed family, or a legitimate full projection square. Field preservation does not imply a proper, smaller, minimum or positive support. A kernel-checked duplicate-circuit example has positive full slack but no proper seed in the computed model; the existing sharing pass removes that duplicate, so the example is not a normalized terminal counterexample and does not refute a manuscript theorem with additional terminal or admissibility hypotheses. Source matching and influence remain finite exhaustive computations, not polynomial algorithms. Complete Package E, forced-cost transparency, global route coverage and rank decrease, unconditional SaturatePositive, BCELReady and ZeroSlack, exact polynomial PCCMin, encoded runtime, output and certificate bounds, deterministic CNFSAT in P and the eligible root remain open. No fixed weighted checkpoint or global proof gate closes, and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed closed-support full-profile physical gain | formalized-foundation-only | For arbitrary finite wire carriers, keep masks and seed record families under the computed computational-wire-profile model, the full-profile reference minimum of the actual closed support is specialized to primary inputs and reconnected to the derived physical complement. The resulting whole carrier preserves every ordered output and computational field for every input, with exact gate and whole-carrier full-slack balance. Positive computed local full slack yields a checked strict equivalent gain; zero local full slack returns no gain. The support, minimum, source bindings and reconnection are computed, not supplied correctness data. | The full-profile minimum, source matching and saturation influence tests remain exhaustive finite reference computations. This does not derive a proper positive support, make whole-span replacements locally VerifyDW-eligible, implement the manuscript's complete noncomputational profile grammar, establish global route coverage or unconditional SaturatePositive, BCELReady or ZeroSlack, or prove complete PCCMin polynomial runtime, output-size or certificate bounds. The eligible root theorem remains absent and P = NP is not proved. |
| Computed whole-support minimum bridge | formalized-foundation-only | For arbitrary finite wire carriers and keep masks, the physical whole-support seed is derived from all original gates. Its computed saturated interface contains exactly the original gate-valued ordinary outputs, while every computational field is available. Explicit size-preserving comparisons identify its full-profile reference minimum with the independent whole-carrier output-and-field minimum. The actual computed full-profile replacement attains that minimum, has zero remaining whole-carrier full slack, and returns no strict improvement exactly when the original whole-carrier full slack is zero. The seed, interface, observer, minimum and replacement are derived rather than supplied correctness data. | The whole-span reference branch remains exhaustive, including minimum search, source matching and saturation influence. Zero slack after exhaustive reference minimization is not the manuscript's unconditional ZeroSlack theorem or a polynomial PCCMin algorithm. This does not discover a proper positive support, compile arbitrary minima into proper-local VerifyDW histories, reconstruct the complete noncomputational profile grammar, derive global route coverage or unconditional SaturatePositive and BCELReady, or establish complete polynomial runtime, output-size or certificate bounds. The eligible root theorem remains absent and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Computed nested-support cost and positivity transport | formalized-foundation-only | For arbitrary finite wire carriers, keep masks and two raw seed lists whose computed completed supports are physically nested, derive the exact gate difference and crossing bindings. Extend the actual full or quotient reference minimum inside the common ambient input domain, preserving the larger padded ordinary interface and exact required computational-field availability, including false values. The derived comparisons prove that either minimum grows by at most the added physical gates, that full slack and support size minus quotient minimum are monotone, and that positive full slack or projection defect remains positive in the completed larger support. Raw-seed inclusion derives the physical inclusion. No cost inequality, field-equality certificate or optimizer is supplied to the final theorem. | This compares already completed dependency-closed supports in the computational-wire-profile model. It does not prove preservation of an arbitrary raw witness's initial positivity during completion, transparency of every intermediate event, monotonicity of projection defect alone, complete manuscript profile semantics, discovery of a proper positive support, global routing or unconditional SaturatePositive, BCELReady or ZeroSlack. Reference minimization, matching and influence computations remain exhaustive; complete polynomial PCCMin runtime, output-size and certificate bounds are not proved. The eligible root theorem remains absent and P = NP is not proved. |

| Milestone | Status | Exact scope | Boundary |
|---|---|---|---|
| Complete polynomial source-derived serialized normalizer | formalized-foundation-only | For every source bitstring, a concrete finite-machine refinement computes the complete source-derived serialized normalization packet and halts within a polynomial in the encoded input length. The packet has a polynomial encoded-size bound, strict decode roundtrip and malformed-input convention. Its encoded certificate language is equivalent to CNFSAT on that same input; satisfiable inputs have a certificate of at most 30 times the input length plus 12 bits. Every system packet retains all residual equations, and every model packet yields a satisfying source assignment. The packaged polynomial reduction uses the actual serialized machine output. No construction, completeness or correctness certificate is supplied by a caller. | This constructs and encodes an equivalent residual problem; a system packet is not an unsatisfiability verdict. Short certificate existence does not provide certificate discovery. The result does not implement the manuscript NormalizeOrGain minimizer, solve all residual systems, establish global route coverage or unconditional ZeroSlack, or prove exact polynomial PCCMin. Its polynomial bounds cover the complete normalizer, not a complete SAT solver. Deterministic CNFSAT membership in P and the eligible root theorem remain absent; P = NP is not proved. |

## 5 Compiled inventory summary

The inventory contains 68079 exported `PNP.*` kernel declarations from 1110 modules. It excludes 23202 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Bridge | 84 | 24 |
| PNP.Complexity | 25 | 6 |
| PNP.Concrete.BitRegisterCanonical | 43 | 35 |
| PNP.Concrete.BitRegisterClear | 41 | 29 |
| PNP.Concrete.BitRegisterCompilerControl | 65 | 37 |
| PNP.Concrete.BitRegisterCompilerExecution | 31 | 20 |
| PNP.Concrete.BitRegisterCompilerIO | 34 | 22 |
| PNP.Concrete.BitRegisterCompilerIOBounds | 19 | 17 |
| PNP.Concrete.BitRegisterCompilerIOPolynomial | 20 | 14 |
| PNP.Concrete.BitRegisterCompilerStep | 56 | 44 |
| PNP.Concrete.BitRegisterControl | 63 | 37 |
| PNP.Concrete.BitRegisterCopyProgram | 36 | 30 |
| PNP.Concrete.BitRegisterExecution | 40 | 27 |
| PNP.Concrete.BitRegisterInput | 47 | 37 |
| PNP.Concrete.BitRegisterInspection | 37 | 27 |
| PNP.Concrete.BitRegisterIOBounds | 23 | 21 |
| PNP.Concrete.BitRegisterIOPrimitives | 34 | 22 |
| PNP.Concrete.BitRegisterMutation | 67 | 28 |
| PNP.Concrete.BitRegisterOperations | 91 | 59 |
| PNP.Concrete.BitRegisterOutput | 32 | 24 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.BitRegisterProgram | 238 | 63 |
| PNP.Concrete.BitRegisterStep | 62 | 50 |
| PNP.Concrete.BitRegisterStoreTape | 50 | 32 |
| PNP.Concrete.BitRegisterTape | 38 | 11 |
| PNP.Concrete.BitString | 123 | 60 |
| PNP.Concrete.CNF | 247 | 60 |
| PNP.Concrete.CNFDecoderBits | 35 | 19 |
| PNP.Concrete.CNFDecoderExecution | 26 | 22 |
| PNP.Concrete.CNFDecoderGrammar | 57 | 50 |
| PNP.Concrete.CNFDecoderModel | 117 | 51 |
| PNP.Concrete.CNFDecoderPolynomial | 10 | 8 |
| PNP.Concrete.CNFDecoderProgram | 19 | 14 |
| PNP.Concrete.CNFSourceParserCompiled | 15 | 11 |
| PNP.Concrete.CNFSourceParserCorrectness | 198 | 133 |
| PNP.Concrete.CNFSourceParserMachine | 106 | 44 |
| PNP.Concrete.CNFSourceParserSpec | 18 | 15 |
| PNP.Concrete.CNFToNAND | 189 | 102 |
| PNP.Concrete.CNFToNANDCarrierEncoder | 571 | 264 |
| PNP.Concrete.CNFToNANDCarrierTokenReader | 70 | 27 |
| PNP.Concrete.CNFToNANDCompilerCompiled | 15 | 10 |
| PNP.Concrete.CNFToNANDCompilerMachine | 30 | 18 |
| PNP.Concrete.CNFToNANDCompilerPolynomialBound | 19 | 15 |
| PNP.Concrete.CNFToNANDCompilerTotalTrace | 7 | 7 |
| PNP.Concrete.CNFToNANDCompilerTrace | 16 | 15 |
| PNP.Concrete.CNFToNANDController | 479 | 109 |
| PNP.Concrete.CNFToNANDControllerBlocks | 79 | 29 |
| PNP.Concrete.CNFToNANDControllerCanonicalTrace | 151 | 110 |
| PNP.Concrete.CNFToNANDControllerCompletionTrace | 45 | 33 |
| PNP.Concrete.CNFToNANDControllerCountTrace | 239 | 203 |
| PNP.Concrete.CNFToNANDControllerPolynomialBound | 51 | 42 |
| PNP.Concrete.CNFToNANDControllerTotalBound | 2 | 1 |
| PNP.Concrete.CNFToNANDControllerTotalTrace | 4 | 4 |
| PNP.Concrete.CNFToNANDEmitterPlan | 224 | 125 |
| PNP.Concrete.CNFToNANDPolynomialReduction | 15 | 11 |
| PNP.Concrete.CNFToNANDWorkspace | 96 | 77 |
| PNP.Concrete.CNFVerifier | 10 | 7 |
| PNP.Concrete.CNFWorkCorrectness | 855 | 521 |
| PNP.Concrete.CNFWorkFrameCorrectness | 16 | 4 |
| PNP.Concrete.CNFWorkInput | 29 | 14 |
| PNP.Concrete.CNFWorkMachine | 85 | 3 |
| PNP.Concrete.CNFWorkTransitions | 64 | 63 |
| PNP.Concrete.CNFWorkUniversalCorrectness | 290 | 141 |
| PNP.Concrete.Complexity | 312 | 92 |
| PNP.Concrete.CookLevinBuilderArbitrarySlotHeaderRouter | 182 | 104 |
| PNP.Concrete.CookLevinBuilderArbitrarySlotPostHeader Decoder | 100 | 62 |
| PNP.Concrete.CookLevinBuilderBalancedCursor | 37 | 24 |
| PNP.Concrete.CookLevinBuilderBodyStartPrefix | 80 | 61 |
| PNP.Concrete.CookLevinBuilderBoundaryPayload | 107 | 56 |
| PNP.Concrete.CookLevinBuilderCanonicalOutput | 18 | 16 |
| PNP.Concrete.CookLevinBuilderClassifierRegisterRestore | 69 | 31 |
| PNP.Concrete.CookLevinBuilderClauseCoordinateRegisters | 42 | 32 |
| PNP.Concrete.CookLevinBuilderClauseDividerExecution | 61 | 44 |
| PNP.Concrete.CookLevinBuilderClauseDividerOperands | 85 | 61 |
| PNP.Concrete.CookLevinBuilderCompleteHeader | 141 | 92 |
| PNP.Concrete.CookLevinBuilderCompleteScheduleIteration | 27 | 15 |
| PNP.Concrete.CookLevinBuilderConstraintRegionAssembly | 71 | 51 |
| PNP.Concrete.CookLevinBuilderConstraintRegionDispatch | 220 | 134 |
| PNP.Concrete.CookLevinBuilderConstraintRegionRegisters | 129 | 61 |
| PNP.Concrete.CookLevinBuilderConstraintRegionSource | 57 | 41 |
| PNP.Concrete.CookLevinBuilderControlActionSource | 83 | 55 |
| PNP.Concrete.CookLevinBuilderControlCoordinates | 63 | 30 |
| PNP.Concrete.CookLevinBuilderControlHeadSource | 74 | 50 |
| PNP.Concrete.CookLevinBuilderControlImplicationPayload | 187 | 114 |
| PNP.Concrete.CookLevinBuilderControlLiteralSources | 180 | 118 |
| PNP.Concrete.CookLevinBuilderControlPayload | 91 | 61 |
| PNP.Concrete.CookLevinBuilderCursorLoop | 55 | 39 |
| PNP.Concrete.CookLevinBuilderCursorOutputAdvance | 40 | 26 |
| PNP.Concrete.CookLevinBuilderCursorRecovery | 41 | 32 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.CookLevinBuilderCursorRemainingOne | 36 | 26 |
| PNP.Concrete.CookLevinBuilderCursorSource | 25 | 17 |
| PNP.Concrete.CookLevinBuilderCursorTokenLookup | 32 | 24 |
| PNP.Concrete.CookLevinBuilderCursorTokenRecovery | 29 | 18 |
| PNP.Concrete.CookLevinBuilderDimensionRegisters | 22 | 16 |
| PNP.Concrete.CookLevinBuilderDividerCoordinateRegisters | 58 | 27 |
| PNP.Concrete.CookLevinBuilderDividerFootprint | 32 | 30 |
| PNP.Concrete.CookLevinBuilderDividerLayout | 24 | 15 |
| PNP.Concrete.CookLevinBuilderDividerOperands | 55 | 34 |
| PNP.Concrete.CookLevinBuilderDividerSourceExecution | 50 | 31 |
| PNP.Concrete.CookLevinBuilderDynamicTokenCursorStep | 68 | 54 |
| PNP.Concrete.CookLevinBuilderExactlyOneClauseOccupancy | 81 | 54 |
| PNP.Concrete.CookLevinBuilderExclusionClauseBoundary | 68 | 47 |
| PNP.Concrete.CookLevinBuilderExclusionClauseTokenSelector | 113 | 86 |
| PNP.Concrete.CookLevinBuilderExclusionPairFirstVariable | 81 | 57 |
| PNP.Concrete.CookLevinBuilderExclusionPairLiteralTokens | 65 | 48 |
| PNP.Concrete.CookLevinBuilderExclusionPairLookup | 71 | 51 |
| PNP.Concrete.CookLevinBuilderExclusionPairPreparation | 84 | 56 |
| PNP.Concrete.CookLevinBuilderExclusionPairRow | 46 | 36 |
| PNP.Concrete.CookLevinBuilderExclusionPairSecondVariable | 134 | 93 |
| PNP.Concrete.CookLevinBuilderExclusionPairSelection | 59 | 46 |
| PNP.Concrete.CookLevinBuilderExclusionTokenRecovery | 18 | 16 |
| Frame | | |
| PNP.Concrete.CookLevinBuilderFamilyPayload | 75 | 59 |
| PNP.Concrete.CookLevinBuilderFifthClausePaddingRun | 143 | 109 |
| PNP.Concrete.CookLevinBuilderFirstClausePaddingRun | 160 | 114 |
| PNP.Concrete.CookLevinBuilderFirstClausePrefix | 120 | 85 |
| PNP.Concrete.CookLevinBuilderFirstConstraintPaddingRun | 155 | 121 |
| PNP.Concrete.CookLevinBuilderFirstLiteralPrefix | 121 | 99 |
| PNP.Concrete.CookLevinBuilderFirstTokenPrefix | 48 | 36 |
| PNP.Concrete.CookLevinBuilderFourthClauseFirstLiteral | 138 | 102 |
| Prefix | | |
| PNP.Concrete.CookLevinBuilderFourthClausePaddingRun | 148 | 114 |
| PNP.Concrete.CookLevinBuilderFourthClausePrefix | 97 | 80 |
| PNP.Concrete.CookLevinBuilderFourthClauseSecondLiteral | 187 | 137 |
| Prefix | | |
| PNP.Concrete.CookLevinBuilderFourthClauseSeparatorStep | 85 | 70 |
| PNP.Concrete.CookLevinBuilderFullScheduleCursorController | 120 | 85 |
| PNP.Concrete.CookLevinBuilderIndexedInputRead | 55 | 41 |
| PNP.Concrete.CookLevinBuilderIndexedLiteralTokenSelector | 76 | 50 |
| PNP.Concrete.CookLevinBuilderInitialCellCoordinates | 149 | 107 |
| PNP.Concrete.CookLevinBuilderInitialCellDecoder | 146 | 81 |
| PNP.Concrete.CookLevinBuilderInitialCellHandoff | 205 | 105 |
| PNP.Concrete.CookLevinBuilderInitialCellSelection | 44 | 40 |
| PNP.Concrete.CookLevinBuilderInitialConstraintPayload | 68 | 53 |
| PNP.Concrete.CookLevinBuilderInitialInputOnlyPayload | 199 | 112 |
| PNP.Concrete.CookLevinBuilderInitialInputOnlySource | 59 | 40 |
| PNP.Concrete.CookLevinBuilderInitialization | 22 | 14 |
| PNP.Concrete.CookLevinBuilderInitialLengthPayload | 90 | 62 |
| PNP.Concrete.CookLevinBuilderInitialLengthSelection | 47 | 31 |
| PNP.Concrete.CookLevinBuilderInitialPairedCellPayload | 61 | 45 |
| PNP.Concrete.CookLevinBuilderInitialPairedCellResolution | 67 | 49 |
| PNP.Concrete.CookLevinBuilderInitialPairedCellSource | 182 | 113 |
| PNP.Concrete.CookLevinBuilderInitialPairedLiteralPayload | 215 | 108 |
| PNP.Concrete.CookLevinBuilderInitialPairedRequest | 279 | 132 |
| PNP.Concrete.CookLevinBuilderInitialPayload | 200 | 125 |
| PNP.Concrete.CookLevinBuilderInitialRequestResolution | 162 | 107 |
| PNP.Concrete.CookLevinBuilderInitialRowCarry | 128 | 93 |
| PNP.Concrete.CookLevinBuilderInitialRowLoop | 155 | 98 |
| PNP.Concrete.CookLevinBuilderInputLength | 47 | 25 |
| PNP.Concrete.CookLevinBuilderInputPrefix | 52 | 39 |
| PNP.Concrete.CookLevinBuilderLiteralArgumentSource | 180 | 91 |
| PNP.Concrete.CookLevinBuilderLiteralClauseTokenSelector | 118 | 85 |
| PNP.Concrete.CookLevinBuilderLiteralIndexExpression | 139 | 71 |
| PNP.Concrete.CookLevinBuilderLiteralListSearch | 78 | 44 |
| PNP.Concrete.CookLevinBuilderLiteralListSearchBounds | 18 | 12 |
| PNP.Concrete.CookLevinBuilderLiteralSearchAdvance | 66 | 46 |
| PNP.Concrete.CookLevinBuilderLiteralSearchComparison | 77 | 52 |
| PNP.Concrete.CookLevinBuilderLiteralSearchFrame | 61 | 46 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.CookLevinBuilderLiteralSearchGuard | 51 | 35 |
| PNP.Concrete.CookLevinBuilderLiteralSearchSelect | 74 | 51 |
| PNP.Concrete.CookLevinBuilderLiteralTokenSelector | 111 | 76 |
| PNP.Concrete.CookLevinBuilderLocalConstraintPayload | 114 | 78 |
| PNP.Concrete.CookLevinBuilderOperandRegisters | 88 | 50 |
| PNP.Concrete.CookLevinBuilderOutputFinalizer | 27 | 19 |
| PNP.Concrete.CookLevinBuilderPayloadBodyPreparation | 73 | 53 |
| PNP.Concrete.CookLevinBuilderPayloadBodyTokenLookup | 39 | 28 |
| PNP.Concrete.CookLevinBuilderPayloadClauseOccupancy | 117 | 76 |
| PNP.Concrete.CookLevinBuilderPayloadClauseTokenSelector | 84 | 61 |
| PNP.Concrete.CookLevinBuilderPayloadConclusionGuard | 51 | 35 |
| PNP.Concrete.CookLevinBuilderPayloadFieldCopy | 73 | 55 |
| PNP.Concrete.CookLevinBuilderPayloadLiteralTokenSelector | 248 | 130 |
| PNP.Concrete.CookLevinBuilderPayloadSearchAdvance | 60 | 44 |
| PNP.Concrete.CookLevinBuilderPayloadSearchComparison | 93 | 63 |
| PNP.Concrete.CookLevinBuilderPayloadSearchHit | 39 | 27 |
| PNP.Concrete.CookLevinBuilderPayloadSearchSource | 100 | 50 |
| PNP.Concrete.CookLevinBuilderPayloadSourceSearch | 49 | 16 |
| PNP.Concrete.CookLevinBuilderPayloadSourceSearchBlank | 15 | 14 |
| PNP.Concrete.CookLevinBuilderPayloadSourceSearchBounds | 26 | 26 |
| PNP.Concrete.CookLevinBuilderPayloadSourceSearchControl | 56 | 40 |
| PNP.Concrete.CookLevinBuilderPayloadSourceSearchEnvelope | 40 | 33 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierAllBody StagedRequestMirroredDispatch | 123 | 80 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierAllRoute BodyRemainderSplit | 117 | 71 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierAllRoute DerivedFinishSplit | 134 | 85 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierAllRoute StagedRequestMirroredDispatch | 99 | 65 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierFinish MirroredDispatch | 69 | 50 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierFinish Request | 46 | 32 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierFinish WorkspaceOrientation | 86 | 56 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierFirstBody SeparatorMirroredDispatch | 184 | 126 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierPipeline | 107 | 63 |
| PNP.Concrete.CookLevinBuilderPhysicalClassifierTerminal Join | 47 | 32 |
| PNP.Concrete.CookLevinBuilderPhysicalDispatchSchedule | 39 | 26 |
| PNP.Concrete.CookLevinBuilderPhysicalFinishRequest | 73 | 45 |
| PNP.Concrete.CookLevinBuilderPhysicalOptionalToken Dispatch | 60 | 36 |
| PNP.Concrete.CookLevinBuilderPolynomialRegisterCopies | 170 | 87 |
| PNP.Concrete.CookLevinBuilderPostDividerRawRoute Classifier | 163 | 108 |
| PNP.Concrete.CookLevinBuilderPostDividerSelectedToken Launch | 42 | 28 |
| PNP.Concrete.CookLevinBuilderPostHeaderRawDivider | 123 | 83 |
| PNP.Concrete.CookLevinBuilderPostHeaderRawLaunch | 33 | 21 |
| PNP.Concrete.CookLevinBuilderPostHeaderRawTapeBridge | 143 | 91 |
| PNP.Concrete.CookLevinBuilderPreservationCoordinates | 56 | 28 |
| PNP.Concrete.CookLevinBuilderPreservationImplication Payload | 97 | 65 |
| PNP.Concrete.CookLevinBuilderPreservationLiteralSources | 77 | 56 |
| PNP.Concrete.CookLevinBuilderPreservationPayload | 88 | 59 |
| PNP.Concrete.CookLevinBuilderRawInputLoop | 21 | 16 |
| PNP.Concrete.CookLevinBuilderRegionComparisonOperands | 55 | 41 |
| PNP.Concrete.CookLevinBuilderRegionCoordinateDivision | 59 | 41 |
| PNP.Concrete.CookLevinBuilderRegionPairComparison | 36 | 23 |
| PNP.Concrete.CookLevinBuilderRegionRadixDecoder | 119 | 67 |
| PNP.Concrete.CookLevinBuilderRegionRadixDispatch | 84 | 64 |
| PNP.Concrete.CookLevinBuilderRegionRadixSource | 119 | 71 |
| PNP.Concrete.CookLevinBuilderRegionResidualOperands | 54 | 41 |
| PNP.Concrete.CookLevinBuilderRegionResidualRegisters | 80 | 39 |
| PNP.Concrete.CookLevinBuilderRegionResidualSelection | 72 | 53 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.CookLevinBuilderRegisterAccess | 32 | 21 |
| PNP.Concrete.CookLevinBuilderRegisterCompareResidual | 91 | 59 |
| PNP.Concrete.CookLevinBuilderRegisterCountdownControl | 55 | 43 |
| PNP.Concrete.CookLevinBuilderRegisterDescendingRange | 111 | 81 |
| PNP.Concrete.CookLevinBuilderRegisterEquality | 45 | 30 |
| PNP.Concrete.CookLevinBuilderRegisterErase | 41 | 30 |
| PNP.Concrete.CookLevinBuilderRegisterExactlyOnePayload | 83 | 60 |
| PNP.Concrete.CookLevinBuilderRegisterExpression | 140 | 67 |
| PNP.Concrete.CookLevinBuilderRegisterHalve | 75 | 54 |
| PNP.Concrete.CookLevinBuilderRegisterHeadMove | 128 | 88 |
| PNP.Concrete.CookLevinBuilderRegisterIndexedCopy | 123 | 84 |
| PNP.Concrete.CookLevinBuilderRegisterLessThan | 64 | 42 |
| PNP.Concrete.CookLevinBuilderRegisterPack | 78 | 33 |
| PNP.Concrete.CookLevinBuilderRegisterPairExterior | 10 | 8 |
| PNP.Concrete.CookLevinBuilderRegisterRootCopy | 88 | 53 |
| PNP.Concrete.CookLevinBuilderRegisterRootErase | 37 | 29 |
| PNP.Concrete.CookLevinBuilderRegisterTable | 100 | 59 |
| PNP.Concrete.CookLevinBuilderRequestDispatch | 106 | 53 |
| PNP.Concrete.CookLevinBuilderRequestedExclusionInput | 115 | 84 |
| PNP.Concrete.CookLevinBuilderRequestedExclusionToken Lookup | 60 | 46 |
| PNP.Concrete.CookLevinBuilderRequestedPairLookup | 54 | 40 |
| PNP.Concrete.CookLevinBuilderRequestedPairVariables | 214 | 151 |
| PNP.Concrete.CookLevinBuilderRequestRegisterMatch | 45 | 29 |
| PNP.Concrete.CookLevinBuilderRequestTokenLookup | 54 | 39 |
| PNP.Concrete.CookLevinBuilderSecondClauseFirstLiteral Prefix | 135 | 106 |
| PNP.Concrete.CookLevinBuilderSecondClausePaddingRun | 126 | 92 |
| PNP.Concrete.CookLevinBuilderSecondClausePrefix | 86 | 69 |
| PNP.Concrete.CookLevinBuilderSecondClauseSecondLiteral Prefix | 158 | 118 |
| PNP.Concrete.CookLevinBuilderSecondClauseSeparatorStep | 84 | 67 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFifth PaddingOrTerminatorOpportunityStep | 114 | 82 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral FirstUnaryUnitStep | 94 | 79 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral SecondUnaryUnitStep | 94 | 79 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral SignStep | 94 | 79 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral SuccessorTokenStep | 111 | 79 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral TerminatorStep | 310 | 295 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFirstLiteral ThirdUnaryUnitStep | 94 | 79 |
| PNP.Concrete.CookLevinBuilderSecondConstraintFourth PaddingOrUnaryOpportunityStep | 96 | 67 |
| PNP.Concrete.CookLevinBuilderSecondConstraintPaddingOr UnaryOpportunityStep | 114 | 82 |
| PNP.Concrete.CookLevinBuilderSecondConstraintSecond PaddingOrUnaryOpportunityStep | 96 | 67 |
| PNP.Concrete.CookLevinBuilderSecondConstraintSeparator Step | 93 | 78 |
| PNP.Concrete.CookLevinBuilderSecondConstraintSeventh PaddingOrUnaryOpportunityStep | 96 | 67 |
| PNP.Concrete.CookLevinBuilderSecondConstraintSixth PaddingOrOpeningUnaryOpportunityStep | 96 | 67 |
| PNP.Concrete.CookLevinBuilderSecondConstraintThird PaddingOrUnaryOpportunityStep | 96 | 67 |
| PNP.Concrete.CookLevinBuilderShapeBranchPayload | 92 | 65 |
| PNP.Concrete.CookLevinBuilderShapeCoordinates | 119 | 69 |
| PNP.Concrete.CookLevinBuilderShapePayload | 101 | 67 |
| PNP.Concrete.CookLevinBuilderSourceClassifier | 45 | 26 |
| PNP.Concrete.CookLevinBuilderSourceClauseCoordinate | 51 | 38 |
| PNP.Concrete.CookLevinBuilderSourceClauseOccupancy | 40 | 28 |
| PNP.Concrete.CookLevinBuilderSourcePayload | 67 | 47 |
| PNP.Concrete.CookLevinBuilderSourceRegisterRestore | 56 | 42 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.CookLevinBuilderSourceTokenRequest | 231 | 143 |
| PNP.Concrete.CookLevinBuilderThirdClauseFirstLiteralPrefix | 131 | 105 |
| PNP.Concrete.CookLevinBuilderThirdClausePaddingRun | 134 | 100 |
| PNP.Concrete.CookLevinBuilderThirdClausePrefix | 93 | 76 |
| PNP.Concrete.CookLevinBuilderThirdClauseSecondLiteral Prefix | 213 | 158 |
| PNP.Concrete.CookLevinBuilderThirdClauseSeparatorStep | 81 | 66 |
| PNP.Concrete.CookLevinBuilderTokenAppender | 99 | 64 |
| PNP.Concrete.CookLevinBuilderUnaryPolynomial | 328 | 196 |
| PNP.Concrete.CookLevinBuilderUnaryTagMatch | 27 | 22 |
| PNP.Concrete.CookLevinClauseOccupancy | 12 | 6 |
| PNP.Concrete.CookLevinClauseOccupancyDivision | 23 | 17 |
| PNP.Concrete.CookLevinCompleteBuilder | 15 | 11 |
| PNP.Concrete.CookLevinFormulaCursor | 259 | 121 |
| PNP.Concrete.CookLevinFormulaSchedule | 105 | 74 |
| PNP.Concrete.CookLevinFormulaSize | 168 | 149 |
| PNP.Concrete.CookLevinLayout | 204 | 71 |
| PNP.Concrete.CookLevinLocalCNF | 172 | 53 |
| PNP.Concrete.CookLevinNPCompleteness | 2 | 2 |
| PNP.Concrete.CookLevinRawTapeBridge | 107 | 95 |
| PNP.Concrete.CookLevinTableau | 71 | 23 |
| PNP.Concrete.CookLevinTableauCNF | 179 | 60 |
| PNP.Concrete.CookLevinTableauCNFSemantics | 190 | 136 |
| PNP.Concrete.CookLevinVerifierTableau | 58 | 25 |
| PNP.Concrete.LockedNANDEncoding | 611 | 215 |
| PNP.Concrete.LockedNANDPolynomialReduction | 8 | 6 |
| PNP.Concrete.LockedNANDRawBuilder | 292 | 169 |
| PNP.Concrete.LockedNANDReduction | 24 | 15 |
| PNP.Concrete.LockedNANDSourceParserCompiled | 15 | 11 |
| PNP.Concrete.LockedNANDSourceParserCorrectness | 23 | 20 |
| PNP.Concrete.LockedNANDSourceParserFailureShapes | 124 | 44 |
| PNP.Concrete.LockedNANDSourceParserMachine | 207 | 32 |
| PNP.Concrete.LockedNANDSourceParserSemantics | 33 | 33 |
| PNP.Concrete.LockedNANDSourceParserSpec | 17 | 14 |
| PNP.Concrete.LockedNANDSourceParserTotalTrace | 54 | 28 |
| PNP.Concrete.LockedNANDSourceParserValidTrace | 353 | 232 |
| PNP.Concrete.LockedNANDTargetEmitterBlockCompiler | 66 | 38 |
| PNP.Concrete.LockedNANDTargetEmitterCapacity | 160 | 130 |
| PNP.Concrete.LockedNANDTargetEmitterCheckStack | 341 | 149 |
| PNP.Concrete.LockedNANDTargetEmitterController | 214 | 25 |
| PNP.Concrete.LockedNANDTargetEmitterControllerCheck Trace | 21 | 18 |
| PNP.Concrete.LockedNANDTargetEmitterControllerCompiled | 19 | 11 |
| PNP.Concrete.LockedNANDTargetEmitterController CompletionTrace | 33 | 17 |
| PNP.Concrete.LockedNANDTargetEmitterControllerGate Bound | 65 | 41 |
| PNP.Concrete.LockedNANDTargetEmitterControllerGateList Trace | 34 | 18 |
| PNP.Concrete.LockedNANDTargetEmitterControllerGate Trace | 82 | 47 |
| PNP.Concrete.LockedNANDTargetEmitterControllerHeader Bound | 29 | 21 |
| PNP.Concrete.LockedNANDTargetEmitterControllerHeader Trace | 13 | 10 |
| PNP.Concrete.LockedNANDTargetEmitterControllerInitial Trace | 8 | 7 |
| PNP.Concrete.LockedNANDTargetEmitterController NormalizationBound | 45 | 33 |
| PNP.Concrete.LockedNANDTargetEmitterController NormalizationTrace | 27 | 19 |
| PNP.Concrete.LockedNANDTargetEmitterControllerOutput Bound | 47 | 28 |
| PNP.Concrete.LockedNANDTargetEmitterControllerOutput Trace | 51 | 38 |
| PNP.Concrete.LockedNANDTargetEmitterController PolynomialBound | 33 | 22 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.LockedNANDTargetEmitterControllerPrefixBound | 75 | 43 |
| PNP.Concrete.LockedNANDTargetEmitterControllerPrefixTrace | 49 | 34 |
| PNP.Concrete.LockedNANDTargetEmitterControllerSourceTrace | 41 | 32 |
| PNP.Concrete.LockedNANDTargetEmitterControllerTotalTrace | 15 | 15 |
| PNP.Concrete.LockedNANDTargetEmitterControllerTrace | 69 | 59 |
| PNP.Concrete.LockedNANDTargetEmitterCursorAppender | 118 | 71 |
| PNP.Concrete.LockedNANDTargetEmitterCursorControl | 39 | 22 |
| PNP.Concrete.LockedNANDTargetEmitterCursorFinalizer | 32 | 15 |
| PNP.Concrete.LockedNANDTargetEmitterCursorNatLoop | 202 | 95 |
| PNP.Concrete.LockedNANDTargetEmitterFinalizer | 29 | 14 |
| PNP.Concrete.LockedNANDTargetEmitterGrammarScanner | 307 | 130 |
| PNP.Concrete.LockedNANDTargetEmitterLedger | 854 | 359 |
| PNP.Concrete.LockedNANDTargetEmitterMachine | 178 | 67 |
| PNP.Concrete.LockedNANDTargetEmitterMarkedSourceReload | 229 | 110 |
| PNP.Concrete.LockedNANDTargetEmitterNatLoop | 187 | 82 |
| PNP.Concrete.LockedNANDTargetEmitterNavigator | 127 | 39 |
| PNP.Concrete.LockedNANDTargetEmitterPlan | 538 | 186 |
| PNP.Concrete.LockedNANDTargetEmitterPrimitiveCompiler | 56 | 44 |
| PNP.Concrete.LockedNANDTargetEmitterProgramSemantics | 367 | 222 |
| PNP.Concrete.LockedNANDTargetEmitterRuntime | 15 | 10 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeCheckStack | 18 | 14 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeLayout | 37 | 22 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimePrimitives | 96 | 86 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeProgram | 132 | 45 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeProgramBound | 64 | 58 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeProgramSafety | 141 | 93 |
| PNP.Concrete.LockedNANDTargetEmitterRuntimeSourceControl | 20 | 15 |
| PNP.Concrete.LockedNANDTargetEmitterSchedule | 28 | 10 |
| PNP.Concrete.LockedNANDTargetEmitterScratchAddSlot | 177 | 79 |
| PNP.Concrete.LockedNANDTargetEmitterScratchCompareSlot | 110 | 31 |
| PNP.Concrete.LockedNANDTargetEmitterScratchCompareSlotExact | 38 | 34 |
| PNP.Concrete.LockedNANDTargetEmitterScratchIncrement | 66 | 21 |
| PNP.Concrete.LockedNANDTargetEmitterScratchReset | 77 | 28 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticCompletion | 16 | 7 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticFinal | 7 | 6 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticNormalization | 57 | 41 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticOutput | 15 | 12 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticPrefix | 65 | 32 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticPrefixBridge | 2 | 1 |
| PNP.Concrete.LockedNANDTargetEmitterSemanticSchedule | 72 | 46 |
| PNP.Concrete.LockedNANDTargetEmitterSlotIncrement | 202 | 88 |
| PNP.Concrete.LockedNANDTargetEmitterSourceCapture | 210 | 73 |
| PNP.Concrete.LockedNANDTargetEmitterSpec | 57 | 51 |
| PNP.Concrete.LockedNANDThresholdPublication | 1 | 1 |
| PNP.Concrete.Machine | 284 | 67 |
| PNP.Concrete.NANDFrontendBounds | 19 | 13 |
| PNP.Concrete.NANDFrontendCanonical | 11 | 11 |
| PNP.Concrete.NANDFrontendClause | 14 | 11 |
| PNP.Concrete.NANDFrontendClosed | 19 | 11 |
| PNP.Concrete.NANDFrontendConstants | 17 | 13 |
| PNP.Concrete.NANDFrontendData | 44 | 30 |
| PNP.Concrete.NANDFrontendGate | 15 | 13 |
| PNP.Concrete.NANDFrontendGateLoop | 31 | 18 |
| PNP.Concrete.NANDFrontendGateRead | 18 | 14 |
| PNP.Concrete.NANDFrontendGateResult | 42 | 16 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NANDFrontendHeader | 46 | 31 |
| PNP.Concrete.NANDFrontendLiteral | 41 | 41 |
| PNP.Concrete.NANDFrontendLoopResult | 32 | 24 |
| PNP.Concrete.NANDFrontendMarkers | 63 | 33 |
| PNP.Concrete.NANDFrontendOriginal | 15 | 10 |
| PNP.Concrete.NANDFrontendOutput | 36 | 25 |
| PNP.Concrete.NANDFrontendOutputTail | 17 | 12 |
| PNP.Concrete.NANDFrontendPack | 55 | 28 |
| PNP.Concrete.NANDFrontendPolynomial | 28 | 21 |
| PNP.Concrete.NANDFrontendProgram | 82 | 25 |
| PNP.Concrete.NANDFrontendRead | 102 | 38 |
| PNP.Concrete.NANDFrontendSource | 139 | 76 |
| PNP.Concrete.NANDFrontendUnary | 84 | 48 |
| PNP.Concrete.NativeNormalization.CNFCompact | 31 | 24 |
| PNP.Concrete.NativeNormalization.CNFCore | 63 | 50 |
| PNP.Concrete.NativeNormalization.CNFTransport | 42 | 32 |
| PNP.Concrete.NativeNormalization.CoefficientBridge | 29 | 23 |
| PNP.Concrete.NativeNormalization.FrontendBridge | 9 | 8 |
| PNP.Concrete.NativeNormalization.MatchingCache | 40 | 18 |
| PNP.Concrete.NativeNormalization.MatchingReach | 101 | 40 |
| PNP.Concrete.NativeNormalization.MatchingSearch | 68 | 33 |
| PNP.Concrete.NativeNormalization.MatchingUpdate | 90 | 43 |
| PNP.Concrete.NativeNormalization.MeasuredAccess | 58 | 42 |
| PNP.Concrete.NativeNormalization.MeasuredAssembly | 44 | 31 |
| PNP.Concrete.NativeNormalization.MeasuredAugment | 62 | 37 |
| PNP.Concrete.NativeNormalization.MeasuredBits | 30 | 25 |
| PNP.Concrete.NativeNormalization.MeasuredCache | 73 | 46 |
| PNP.Concrete.NativeNormalization.MeasuredCandidate | 42 | 32 |
| PNP.Concrete.NativeNormalization.MeasuredChoice | 43 | 33 |
| PNP.Concrete.NativeNormalization.MeasuredCollections | 38 | 22 |
| PNP.Concrete.NativeNormalization.MeasuredCompanion | 28 | 19 |
| PNP.Concrete.NativeNormalization.MeasuredCompilation Execution | 39 | 27 |
| PNP.Concrete.NativeNormalization.MeasuredCompilationPlan | 48 | 32 |
| PNP.Concrete.NativeNormalization.MeasuredCut | 28 | 24 |
| PNP.Concrete.NativeNormalization.MeasuredCutRecovery | 33 | 25 |
| PNP.Concrete.NativeNormalization.MeasuredDecoder | 29 | 21 |
| PNP.Concrete.NativeNormalization.MeasuredDecoderTokens | 58 | 34 |
| PNP.Concrete.NativeNormalization.MeasuredEncoded Producer | 42 | 20 |
| PNP.Concrete.NativeNormalization.MeasuredExplore | 19 | 12 |
| PNP.Concrete.NativeNormalization.MeasuredFrontendCost | 13 | 11 |
| PNP.Concrete.NativeNormalization.MeasuredMatching | 53 | 38 |
| PNP.Concrete.NativeNormalization.MeasuredMatchingLoop | 32 | 23 |
| PNP.Concrete.NativeNormalization.MeasuredMatching Polynomial | 3 | 2 |
| PNP.Concrete.NativeNormalization.MeasuredOperations | 33 | 14 |
| PNP.Concrete.NativeNormalization.MeasuredParity | 13 | 8 |
| PNP.Concrete.NativeNormalization.MeasuredPrepared | 36 | 30 |
| PNP.Concrete.NativeNormalization.MeasuredPrepared Polynomial | 3 | 2 |
| PNP.Concrete.NativeNormalization.MeasuredProducer | 48 | 23 |
| PNP.Concrete.NativeNormalization.MeasuredProducer Polynomial | 11 | 8 |
| PNP.Concrete.NativeNormalization.MeasuredRawCompiler | 37 | 26 |
| PNP.Concrete.NativeNormalization.MeasuredRawFrontend | 62 | 43 |
| PNP.Concrete.NativeNormalization.MeasuredReach | 69 | 36 |
| PNP.Concrete.NativeNormalization.MeasuredRecovery Polynomial | 9 | 5 |
| PNP.Concrete.NativeNormalization.MeasuredRowEncoding | 18 | 16 |
| PNP.Concrete.NativeNormalization.MeasuredRows | 13 | 10 |
| PNP.Concrete.NativeNormalization.MeasuredRun | 62 | 25 |
| PNP.Concrete.NativeNormalization.MeasuredRunCorrectness | 15 | 11 |
| PNP.Concrete.NativeNormalization.MeasuredRunPolynomial | 3 | 2 |
| PNP.Concrete.NativeNormalization.MeasuredSearch | 58 | 42 |
| PNP.Concrete.NativeNormalization.MeasuredSelection | 50 | 31 |
| PNP.Concrete.NativeNormalization.MeasuredShared Normalizer | 23 | 16 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NativeNormalization.MeasuredSharedProducer | 32 | 25 |
| PNP.Concrete.NativeNormalization.MeasuredSourceCache | 46 | 25 |
| PNP.Concrete.NativeNormalization.MeasuredSourceCache Polynomial | 5 | 4 |
| PNP.Concrete.NativeNormalization.MeasuredSourceRecovery | 34 | 27 |
| PNP.Concrete.NativeNormalization.MeasuredTerms | 34 | 26 |
| PNP.Concrete.NativeNormalization.MeasuredWitness | 21 | 16 |
| PNP.Concrete.NativeNormalization.NativeBasis | 146 | 49 |
| PNP.Concrete.NativeNormalization.NativeBitCodec | 82 | 41 |
| PNP.Concrete.NativeNormalization.NativeCircuit | 41 | 33 |
| PNP.Concrete.NativeNormalization.NativeCoefficients | 66 | 44 |
| PNP.Concrete.NativeNormalization.NativeContext | 117 | 68 |
| PNP.Concrete.NativeNormalization.NativeDimensionCache | 57 | 21 |
| PNP.Concrete.NativeNormalization.NativeEquations | 37 | 19 |
| PNP.Concrete.NativeNormalization.NativeFrontend | 66 | 20 |
| PNP.Concrete.NativeNormalization.NativeFrontendBounds | 6 | 6 |
| PNP.Concrete.NativeNormalization.NativeOutputPacket | 66 | 34 |
| PNP.Concrete.NativeNormalization.NativePreparedProducer | 9 | 6 |
| PNP.Concrete.NativeNormalization.NativeSerialized Normalizer | 39 | 28 |
| PNP.Concrete.NativeNormalization.NativeSource | 64 | 30 |
| PNP.Concrete.NativeNormalization.NativeSourceCache | 25 | 20 |
| PNP.Concrete.NativeNormalization.NativeSourceEquations | 49 | 23 |
| PNP.Concrete.NativeNormalization.NativeSparseChecker | 10 | 9 |
| PNP.Concrete.NativeNormalization.NativeSparseSystem | 78 | 32 |
| PNP.Concrete.NativeNormalization.NativeSystemCodec | 30 | 25 |
| PNP.Concrete.NativeNormalization.NativeTseitin | 44 | 33 |
| PNP.Concrete.NativeNormalization.RecoveryBridge | 19 | 17 |
| PNP.Concrete.NativeNormalization.SerializedMachineBridge | 20 | 18 |
| PNP.Concrete.NativeNormalization.SourceAssemblyBridge | 19 | 16 |
| PNP.Concrete.NativeNormalization.SourceCheckBridge | 298 | 250 |
| PNP.Concrete.NativeNormalization.SourceRecoveryBridge | 9 | 8 |
| PNP.Concrete.NativeNormalization.SystemAssemblyBridge | 16 | 14 |
| PNP.Concrete.NormalizerAugmentationBounds | 16 | 12 |
| PNP.Concrete.NormalizerAugmentationPhase | 56 | 16 |
| PNP.Concrete.NormalizerAugmentationPhaseRegression | 1 | 0 |
| PNP.Concrete.NormalizerAugmentationPolynomial | 32 | 20 |
| PNP.Concrete.NormalizerBatchBody | 61 | 25 |
| PNP.Concrete.NormalizerBatchCanonical | 15 | 9 |
| PNP.Concrete.NormalizerBatchCanonicalBody | 13 | 11 |
| PNP.Concrete.NormalizerBatchHeaders | 5 | 5 |
| PNP.Concrete.NormalizerBatchLoop | 50 | 24 |
| PNP.Concrete.NormalizerBatchPacketCanonical | 10 | 9 |
| PNP.Concrete.NormalizerBatchPacketLoader | 26 | 16 |
| PNP.Concrete.NormalizerBatchPacketSplit | 19 | 11 |
| PNP.Concrete.NormalizerBatchPolynomial | 34 | 25 |
| PNP.Concrete.NormalizerBitLookup | 26 | 18 |
| PNP.Concrete.NormalizerBlockExecution | 10 | 9 |
| PNP.Concrete.NormalizerBlocks | 153 | 57 |
| PNP.Concrete.NormalizerBooleanHead | 19 | 11 |
| PNP.Concrete.NormalizerCacheCell | 11 | 7 |
| PNP.Concrete.NormalizerCacheLiteralQuery | 27 | 18 |
| PNP.Concrete.NormalizerCacheLiteralScan | 33 | 18 |
| PNP.Concrete.NormalizerCacheModeBody | 41 | 24 |
| PNP.Concrete.NormalizerCacheModeCall | 38 | 22 |
| PNP.Concrete.NormalizerCacheModeLoop | 71 | 39 |
| PNP.Concrete.NormalizerCacheModeQuery | 29 | 15 |
| PNP.Concrete.NormalizerCacheModeRead | 54 | 17 |
| PNP.Concrete.NormalizerCacheOwnerFold | 38 | 21 |
| PNP.Concrete.NormalizerCacheOwnerQuery | 56 | 36 |
| PNP.Concrete.NormalizerCacheSigmaCall | 34 | 18 |
| PNP.Concrete.NormalizerCacheSigmaRead | 26 | 14 |
| PNP.Concrete.NormalizerCacheSlotCall | 19 | 12 |
| PNP.Concrete.NormalizerCacheSlotChoose | 46 | 29 |
| PNP.Concrete.NormalizerCacheSlotProbe | 26 | 15 |
| PNP.Concrete.NormalizerCacheSlots | 24 | 12 |
| PNP.Concrete.NormalizerCandidateBitsBody | 49 | 15 |
| PNP.Concrete.NormalizerCandidateBitsCall | 42 | 22 |
| PNP.Concrete.NormalizerCandidateBitsLoop | 43 | 21 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NormalizerCandidateBody | 39 | 27 |
| PNP.Concrete.NormalizerCandidateCanonical | 15 | 7 |
| PNP.Concrete.NormalizerCandidateCanonicalBody | 7 | 5 |
| PNP.Concrete.NormalizerCandidateClause | 55 | 29 |
| PNP.Concrete.NormalizerCandidateLookup | 81 | 27 |
| PNP.Concrete.NormalizerCandidateLoop | 24 | 16 |
| PNP.Concrete.NormalizerCandidateOrder | 9 | 8 |
| PNP.Concrete.NormalizerCandidatePacketLoader | 44 | 28 |
| PNP.Concrete.NormalizerCandidatePacketOperations | 32 | 16 |
| PNP.Concrete.NormalizerCandidatePolynomial | 25 | 19 |
| PNP.Concrete.NormalizerCandidateTruth | 28 | 13 |
| PNP.Concrete.NormalizerClauseCanonical | 8 | 7 |
| PNP.Concrete.NormalizerClauseCount | 66 | 36 |
| PNP.Concrete.NormalizerClauseCountCanonical | 7 | 7 |
| PNP.Concrete.NormalizerClauseOperations | 17 | 7 |
| PNP.Concrete.NormalizerClausePack | 16 | 11 |
| PNP.Concrete.NormalizerClausePolynomial | 37 | 20 |
| PNP.Concrete.NormalizerClauseScan | 33 | 21 |
| PNP.Concrete.NormalizerClauseSemantics | 16 | 14 |
| PNP.Concrete.NormalizerClauseTokens | 30 | 11 |
| PNP.Concrete.NormalizerCoefficientAnchor | 27 | 18 |
| PNP.Concrete.NormalizerCoefficientAuxiliary | 15 | 10 |
| PNP.Concrete.NormalizerCoefficientBasicRows | 25 | 20 |
| PNP.Concrete.NormalizerCoefficientCache | 56 | 21 |
| PNP.Concrete.NormalizerCoefficientCompanion | 41 | 17 |
| PNP.Concrete.NormalizerCoefficientEligibility | 23 | 16 |
| PNP.Concrete.NormalizerCoefficientEll | 39 | 26 |
| PNP.Concrete.NormalizerCoefficientEntry | 11 | 8 |
| PNP.Concrete.NormalizerCoefficientLiteral | 68 | 23 |
| PNP.Concrete.NormalizerCoefficientPacket | 24 | 16 |
| PNP.Concrete.NormalizerCoefficientRoutine | 50 | 28 |
| PNP.Concrete.NormalizerCoefficientRow | 52 | 29 |
| PNP.Concrete.NormalizerCoefficientScratch | 31 | 16 |
| PNP.Concrete.NormalizerCoefficientState | 101 | 23 |
| PNP.Concrete.NormalizerCoefficientTruth | 24 | 18 |
| PNP.Concrete.NormalizerCompactEmission | 43 | 26 |
| PNP.Concrete.NormalizerCompactLiteral | 9 | 6 |
| PNP.Concrete.NormalizerCompactLookup | 28 | 18 |
| PNP.Concrete.NormalizerCompactLoop | 47 | 24 |
| PNP.Concrete.NormalizerCompactRead | 59 | 22 |
| PNP.Concrete.NormalizerCompactSemantics | 15 | 13 |
| PNP.Concrete.NormalizerCompactStep | 32 | 19 |
| PNP.Concrete.NormalizerConstructiveLists | 4 | 4 |
| PNP.Concrete.NormalizerCutBounds | 20 | 14 |
| PNP.Concrete.NormalizerCutCalls | 42 | 21 |
| PNP.Concrete.NormalizerCutPack | 44 | 20 |
| PNP.Concrete.NormalizerCutPolynomial | 30 | 23 |
| PNP.Concrete.NormalizerCutPrepared | 20 | 10 |
| PNP.Concrete.NormalizerCutSelectionCall | 21 | 13 |
| PNP.Concrete.NormalizerCutSelectionLoader | 29 | 17 |
| PNP.Concrete.NormalizerCutSelectionPolynomial | 35 | 24 |
| PNP.Concrete.NormalizerCutSelectionRange | 41 | 20 |
| PNP.Concrete.NormalizerCutSelections | 19 | 11 |
| PNP.Concrete.NormalizerCutSelectionSemantics | 10 | 8 |
| PNP.Concrete.NormalizerCutSelectionState | 59 | 21 |
| PNP.Concrete.NormalizerCutSelectionStep | 19 | 13 |
| PNP.Concrete.NormalizerCutState | 52 | 17 |
| PNP.Concrete.NormalizerCutTrace | 45 | 29 |
| PNP.Concrete.NormalizerCutTracePolynomial | 25 | 21 |
| PNP.Concrete.NormalizerDenseAppend | 25 | 14 |
| PNP.Concrete.NormalizerDenseBody | 13 | 9 |
| PNP.Concrete.NormalizerDenseCall | 31 | 16 |
| PNP.Concrete.NormalizerDenseLoop | 64 | 33 |
| PNP.Concrete.NormalizerDenseQuery | 25 | 14 |
| PNP.Concrete.NormalizerDenseRead | 48 | 16 |
| PNP.Concrete.NormalizerDenseSemantics | 15 | 13 |
| PNP.Concrete.NormalizerEdgeBranches | 28 | 17 |
| PNP.Concrete.NormalizerEdgeDispatch | 66 | 31 |
| PNP.Concrete.NormalizerEdgePolynomial | 25 | 18 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NormalizerEligibility | 57 | 23 |
| PNP.Concrete.NormalizerEligibilityPolynomial | 30 | 21 |
| PNP.Concrete.NormalizerExplorationAccept | 45 | 24 |
| PNP.Concrete.NormalizerExplorationAcceptRegression | 1 | 0 |
| PNP.Concrete.NormalizerExplorationBounds | 36 | 26 |
| PNP.Concrete.NormalizerExplorationBudget | 27 | 13 |
| PNP.Concrete.NormalizerExplorationData | 33 | 19 |
| PNP.Concrete.NormalizerExplorationInitialize | 33 | 23 |
| PNP.Concrete.NormalizerExplorationLoader | 69 | 38 |
| PNP.Concrete.NormalizerExplorationPolynomial | 36 | 23 |
| PNP.Concrete.NormalizerExplorationRuntime | 6 | 6 |
| PNP.Concrete.NormalizerExplorationScan | 42 | 20 |
| PNP.Concrete.NormalizerExplorationScanRegression | 1 | 0 |
| PNP.Concrete.NormalizerExplorationSemantics | 12 | 6 |
| PNP.Concrete.NormalizerExplorationState | 48 | 16 |
| PNP.Concrete.NormalizerExplorationStateRegression | 1 | 0 |
| PNP.Concrete.NormalizerExplorationStep | 23 | 14 |
| PNP.Concrete.NormalizerExplorationStepRegression | 1 | 0 |
| PNP.Concrete.NormalizerExplorationVertices | 35 | 16 |
| PNP.Concrete.NormalizerExtractFinish | 42 | 22 |
| PNP.Concrete.NormalizerExtractFinishRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractLoader | 59 | 38 |
| PNP.Concrete.NormalizerExtractObservation | 38 | 24 |
| PNP.Concrete.NormalizerExtractObservationRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractPack | 69 | 31 |
| PNP.Concrete.NormalizerExtractPackRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractPolynomial | 39 | 29 |
| PNP.Concrete.NormalizerExtractQuery | 36 | 20 |
| PNP.Concrete.NormalizerExtractQueryRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractRange | 49 | 29 |
| PNP.Concrete.NormalizerExtractRead | 74 | 23 |
| PNP.Concrete.NormalizerExtractReadRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractScan | 43 | 21 |
| PNP.Concrete.NormalizerExtractScanRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractSemantics | 12 | 11 |
| PNP.Concrete.NormalizerExtractStep | 44 | 29 |
| PNP.Concrete.NormalizerExtractStepRegression | 1 | 0 |
| PNP.Concrete.NormalizerExtractView | 12 | 8 |
| PNP.Concrete.NormalizerFirstFailureBody | 33 | 17 |
| PNP.Concrete.NormalizerFirstFailureCall | 37 | 20 |
| PNP.Concrete.NormalizerFirstFailureLoop | 42 | 20 |
| PNP.Concrete.NormalizerFormulaOccurrence | 15 | 11 |
| PNP.Concrete.NormalizerForwardLoader | 34 | 23 |
| PNP.Concrete.NormalizerForwardOperations | 74 | 23 |
| PNP.Concrete.NormalizerForwardPolynomial | 38 | 22 |
| PNP.Concrete.NormalizerGraphQuery | 29 | 20 |
| PNP.Concrete.NormalizerMatchingAdvance | 47 | 20 |
| PNP.Concrete.NormalizerMatchingCalls | 33 | 21 |
| PNP.Concrete.NormalizerMatchingExecution | 15 | 9 |
| PNP.Concrete.NormalizerMatchingInitialize | 50 | 24 |
| PNP.Concrete.NormalizerMatchingInvariant | 10 | 9 |
| PNP.Concrete.NormalizerMatchingLoop | 43 | 21 |
| PNP.Concrete.NormalizerMatchingPackets | 35 | 19 |
| PNP.Concrete.NormalizerMatchingPolynomial | 21 | 18 |
| PNP.Concrete.NormalizerMatchingRange | 7 | 7 |
| PNP.Concrete.NormalizerMatchingRawInvariant | 21 | 15 |
| PNP.Concrete.NormalizerMatchingRawRange | 12 | 10 |
| PNP.Concrete.NormalizerMatchingRead | 13 | 9 |
| PNP.Concrete.NormalizerMatchingRelation | 12 | 9 |
| PNP.Concrete.NormalizerMatchingRoot | 10 | 8 |
| PNP.Concrete.NormalizerMatchingRoundBounds | 32 | 25 |
| PNP.Concrete.NormalizerMatchingSemantics | 15 | 8 |
| PNP.Concrete.NormalizerMatchingSize | 17 | 16 |
| PNP.Concrete.NormalizerMatchingState | 75 | 23 |
| PNP.Concrete.NormalizerMatchingStep | 48 | 30 |
| PNP.Concrete.NormalizerMatchingTable | 27 | 11 |
| PNP.Concrete.NormalizerMatchingValueCanonical | 3 | 3 |
| PNP.Concrete.NormalizerMatchingValueOperations | 7 | 6 |
| PNP.Concrete.NormalizerMatchingValuePolynomial | 30 | 18 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NormalizerMatchingValueScan | 27 | 18 |
| PNP.Concrete.NormalizerMemberCanonical | 14 | 10 |
| PNP.Concrete.NormalizerMemberCompare | 38 | 21 |
| PNP.Concrete.NormalizerMemberScan | 43 | 21 |
| PNP.Concrete.NormalizerMemberStep | 28 | 19 |
| PNP.Concrete.NormalizerModeCacheDerive | 24 | 14 |
| PNP.Concrete.NormalizerModeCachePolynomial | 38 | 29 |
| PNP.Concrete.NormalizerModeCacheState | 16 | 9 |
| PNP.Concrete.NormalizerOccurrenceCanonical | 10 | 10 |
| PNP.Concrete.NormalizerOccurrenceHeader | 28 | 20 |
| PNP.Concrete.NormalizerOccurrenceOperations | 74 | 19 |
| PNP.Concrete.NormalizerOccurrencePolynomial | 26 | 14 |
| PNP.Concrete.NormalizerOccurrenceScan | 37 | 25 |
| PNP.Concrete.NormalizerOccurrenceSemantics | 10 | 7 |
| PNP.Concrete.NormalizerOccurrenceTokens | 17 | 10 |
| PNP.Concrete.NormalizerOrdinalTest | 36 | 18 |
| PNP.Concrete.NormalizerOriginalPolynomial | 19 | 12 |
| PNP.Concrete.NormalizerOutputDispatch | 18 | 13 |
| PNP.Concrete.NormalizerOutputLoader | 62 | 23 |
| PNP.Concrete.NormalizerOutputLoaderCanonical | 21 | 16 |
| PNP.Concrete.NormalizerOutputPackets | 32 | 21 |
| PNP.Concrete.NormalizerOutputPostprocess | 33 | 19 |
| PNP.Concrete.NormalizerOutputPostprocessPolynomial | 35 | 27 |
| PNP.Concrete.NormalizerOutputState | 23 | 13 |
| PNP.Concrete.NormalizerOwnerCacheDerive | 28 | 16 |
| PNP.Concrete.NormalizerOwnerCachePolynomial | 36 | 27 |
| PNP.Concrete.NormalizerOwnerCanonical | 6 | 6 |
| PNP.Concrete.NormalizerOwnerFrame | 5 | 4 |
| PNP.Concrete.NormalizerOwnerOperations | 23 | 12 |
| PNP.Concrete.NormalizerOwnerOrder | 2 | 2 |
| PNP.Concrete.NormalizerOwnerPolynomial | 21 | 14 |
| PNP.Concrete.NormalizerOwnerScan | 33 | 21 |
| PNP.Concrete.NormalizerOwnerSemantics | 19 | 11 |
| PNP.Concrete.NormalizerOwnerTokens | 15 | 9 |
| PNP.Concrete.NormalizerOwnerVertices | 11 | 8 |
| PNP.Concrete.NormalizerPacketFrame | 70 | 34 |
| PNP.Concrete.NormalizerPacketFrameRegression | 1 | 0 |
| PNP.Concrete.NormalizerPairCanonical | 16 | 10 |
| PNP.Concrete.NormalizerPairEmission | 26 | 12 |
| PNP.Concrete.NormalizerPairEmissionRegression | 1 | 0 |
| PNP.Concrete.NormalizerPairPhase | 29 | 14 |
| PNP.Concrete.NormalizerPairPhaseRegression | 1 | 0 |
| PNP.Concrete.NormalizerPairPolynomial | 28 | 19 |
| PNP.Concrete.NormalizerPairReader | 62 | 17 |
| PNP.Concrete.NormalizerPairScan | 42 | 20 |
| PNP.Concrete.NormalizerPairScanRegression | 1 | 0 |
| PNP.Concrete.NormalizerPairStep | 44 | 29 |
| PNP.Concrete.NormalizerPairStepRegression | 1 | 0 |
| PNP.Concrete.NormalizerParentLoader | 40 | 22 |
| PNP.Concrete.NormalizerParentNode | 36 | 22 |
| PNP.Concrete.NormalizerParentPhase | 37 | 24 |
| PNP.Concrete.NormalizerParentPolynomial | 24 | 18 |
| PNP.Concrete.NormalizerParentRange | 14 | 7 |
| PNP.Concrete.NormalizerParentRoot | 26 | 15 |
| PNP.Concrete.NormalizerParentScan | 43 | 21 |
| PNP.Concrete.NormalizerParentSemantics | 17 | 11 |
| PNP.Concrete.NormalizerParentState | 51 | 13 |
| PNP.Concrete.NormalizerParentStep | 41 | 25 |
| PNP.Concrete.NormalizerPathAppend | 32 | 17 |
| PNP.Concrete.NormalizerPathAppendRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathCanonical | 11 | 11 |
| PNP.Concrete.NormalizerPathCanonicalRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathCompare | 14 | 9 |
| PNP.Concrete.NormalizerPathCompareRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathData | 23 | 10 |
| PNP.Concrete.NormalizerPathFollow | 31 | 17 |
| PNP.Concrete.NormalizerPathFollowRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathLoader | 17 | 8 |
| PNP.Concrete.NormalizerPathNode | 60 | 23 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NormalizerPathNodeRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathPack | 29 | 16 |
| PNP.Concrete.NormalizerPathPackRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathPhase | 21 | 13 |
| PNP.Concrete.NormalizerPathPhaseRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathPolynomial | 25 | 19 |
| PNP.Concrete.NormalizerPathScan | 45 | 23 |
| PNP.Concrete.NormalizerPathScanRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathSemantics | 7 | 6 |
| PNP.Concrete.NormalizerPathSemanticsRegression | 1 | 0 |
| PNP.Concrete.NormalizerPathStep | 25 | 15 |
| PNP.Concrete.NormalizerPathStepRegression | 1 | 0 |
| PNP.Concrete.NormalizerPreparedDerive | 16 | 10 |
| PNP.Concrete.NormalizerPreparedPolynomial | 34 | 25 |
| PNP.Concrete.NormalizerPreparedState | 17 | 11 |
| PNP.Concrete.NormalizerRecoveryBit | 32 | 18 |
| PNP.Concrete.NormalizerRecoveryCall | 28 | 14 |
| PNP.Concrete.NormalizerRecoveryLoop | 58 | 28 |
| PNP.Concrete.NormalizerRecoveryOwner | 89 | 21 |
| PNP.Concrete.NormalizerRecoveryPreferred | 51 | 23 |
| PNP.Concrete.NormalizerRegisterFrame | 42 | 28 |
| PNP.Concrete.NormalizerRegisterReset | 25 | 12 |
| PNP.Concrete.NormalizerRootKernel | 10 | 6 |
| PNP.Concrete.NormalizerRootKinds | 37 | 24 |
| PNP.Concrete.NormalizerSearchBounds | 27 | 23 |
| PNP.Concrete.NormalizerSearchCandidate | 47 | 23 |
| PNP.Concrete.NormalizerSearchCandidateRegression | 1 | 0 |
| PNP.Concrete.NormalizerSearchExecution | 10 | 5 |
| PNP.Concrete.NormalizerSearchOutput | 79 | 33 |
| PNP.Concrete.NormalizerSearchPath | 17 | 7 |
| PNP.Concrete.NormalizerSearchPathRegression | 1 | 0 |
| PNP.Concrete.NormalizerSearchPolynomial | 24 | 19 |
| PNP.Concrete.NormalizerSearchState | 67 | 36 |
| PNP.Concrete.NormalizerSearchStateRegression | 1 | 0 |
| PNP.Concrete.NormalizerSearchTrace | 32 | 17 |
| PNP.Concrete.NormalizerSearchTraceRegression | 1 | 0 |
| PNP.Concrete.NormalizerSeenData | 12 | 8 |
| PNP.Concrete.NormalizerSeenEmission | 51 | 32 |
| PNP.Concrete.NormalizerSeenEmissionRegression | 1 | 0 |
| PNP.Concrete.NormalizerSeenPhase | 31 | 18 |
| PNP.Concrete.NormalizerSeenPhaseRegression | 1 | 0 |
| PNP.Concrete.NormalizerSeenScan | 43 | 21 |
| PNP.Concrete.NormalizerSeenScanRegression | 1 | 0 |
| PNP.Concrete.NormalizerSelectedFold | 88 | 40 |
| PNP.Concrete.NormalizerSelectedFoldCall | 27 | 17 |
| PNP.Concrete.NormalizerSourceAssemblyDispatch | 24 | 15 |
| PNP.Concrete.NormalizerSourceAssemblyMigration | 26 | 17 |
| PNP.Concrete.NormalizerSourceAssemblyOverlay | 15 | 7 |
| PNP.Concrete.NormalizerSourceAssemblyPolynomial | 36 | 25 |
| PNP.Concrete.NormalizerSourceCacheDimensions | 30 | 14 |
| PNP.Concrete.NormalizerSourceCachePolynomial | 47 | 35 |
| PNP.Concrete.NormalizerSourceCacheSlots | 32 | 14 |
| PNP.Concrete.NormalizerSourceCandidatePolynomial | 33 | 25 |
| PNP.Concrete.NormalizerSourceCandidateState | 31 | 19 |
| PNP.Concrete.NormalizerSourceCheckPolynomial | 32 | 24 |
| PNP.Concrete.NormalizerSourceCheckState | 29 | 16 |
| PNP.Concrete.NormalizerSourceRecoveryPolynomial | 30 | 22 |
| PNP.Concrete.NormalizerSourceRecoveryState | 38 | 19 |
| PNP.Concrete.NormalizerSystemAssembly | 30 | 20 |
| PNP.Concrete.NormalizerSystemAssemblyPrepend | 44 | 18 |
| PNP.Concrete.NormalizerSystemAssemblyState | 74 | 31 |
| PNP.Concrete.NormalizerSystemRowsA | 23 | 15 |
| PNP.Concrete.NormalizerSystemRowsAppend | 24 | 14 |
| PNP.Concrete.NormalizerSystemRowsE | 29 | 20 |
| PNP.Concrete.NormalizerSystemRowsList | 33 | 20 |
| PNP.Concrete.NormalizerSystemRowsLoop | 63 | 26 |
| PNP.Concrete.NormalizerSystemRowsPackets | 27 | 15 |
| PNP.Concrete.NormalizerSystemRowsSemantics | 9 | 6 |
| PNP.Concrete.NormalizerSystemRowsState | 49 | 20 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.Concrete.NormalizerSystemRowsWholeSemantics | 8 | 8 |
| PNP.Concrete.NormalizerTableCanonical | 8 | 7 |
| PNP.Concrete.NormalizerTableData | 12 | 11 |
| PNP.Concrete.NormalizerTableEmission | 17 | 11 |
| PNP.Concrete.NormalizerTableGrowth | 23 | 20 |
| PNP.Concrete.NormalizerTableInitialization | 22 | 14 |
| PNP.Concrete.NormalizerTableInitializationCanonical | 9 | 7 |
| PNP.Concrete.NormalizerTableInitializationPolynomial | 37 | 23 |
| PNP.Concrete.NormalizerTableOperations | 28 | 16 |
| PNP.Concrete.NormalizerTablePolynomial | 53 | 31 |
| PNP.Concrete.NormalizerTableScan | 36 | 24 |
| PNP.Concrete.NormalizerTableSemantics | 5 | 5 |
| PNP.Concrete.NormalizerTableTokens | 12 | 6 |
| PNP.Concrete.NormalizerTraceCount | 26 | 15 |
| PNP.Concrete.NormalizerTraceData | 40 | 23 |
| PNP.Concrete.NormalizerUnseenData | 47 | 26 |
| PNP.Concrete.NormalizerVertexCanonical | 15 | 12 |
| PNP.Concrete.NormalizerVertexEmission | 31 | 17 |
| PNP.Concrete.NormalizerVertexEquality | 3 | 3 |
| PNP.Concrete.NormalizerVertexHeader | 18 | 9 |
| PNP.Concrete.NormalizerVertexPhase | 20 | 15 |
| PNP.Concrete.NormalizerVertexPolynomial | 28 | 18 |
| PNP.Concrete.NormalizerVertexRange | 25 | 18 |
| PNP.Concrete.PipelineCompiler | 38 | 27 |
| PNP.Concrete.PipelineInputFramer | 106 | 18 |
| PNP.Concrete.PipelineMachineSimulation | 175 | 79 |
| PNP.Concrete.PipelineOutputHandoff | 23 | 4 |
| PNP.Concrete.PipelinePairedCompiler | 29 | 25 |
| PNP.Concrete.PipelineRefinement | 68 | 24 |
| PNP.Concrete.PipelineSequentialCompiler | 35 | 27 |
| PNP.Concrete.PipelineSequentialStateNamespace | 30 | 21 |
| PNP.Concrete.PipelineStageBridges | 77 | 59 |
| PNP.Concrete.PipelineStateNamespace | 77 | 34 |
| PNP.Concrete.PipelineTapeGeometry | 20 | 12 |
| PNP.Concrete.PipelineTerminalBridge | 73 | 62 |
| PNP.Concrete.SerializedNormalizer | 19 | 14 |
| PNP.Concrete.TapeBlankEquivalence | 25 | 17 |
| PNP.Concrete.TapeHandoff | 18 | 11 |
| PNP.Concrete.Target | 2 | 1 |
| PNP.Concrete.TerminalOutputPacker | 111 | 36 |
| PNP.Concrete.WorkInput | 23 | 14 |
| PNP.Concrete.WorkMachine | 308 | 108 |
| PNP.Concrete.WorkMachineBlankEquivalence | 51 | 33 |
| PNP.Concrete.WorkMachineChain | 37 | 29 |
| PNP.Concrete.WorkMachineLeftBoundary | 9 | 4 |
| PNP.Concrete.WorkMachineProgramGraph | 225 | 118 |
| PNP.Concrete.WorkMachineProgramPath | 23 | 8 |
| PNP.Concrete.WorkMachineProgramPathBlankEquivalence | 1 | 1 |
| PNP.Concrete.WorkMachineTerminalHandoff | 41 | 31 |
| PNP.ConcreteLegacyLockedNANDCompatibility | 4 | 4 |
| PNP.ConcreteResidualBandCompatibility | 3 | 3 |
| PNP.DirectWireBaseline | 48 | 25 |
| PNP.FiniteDependencyScheduler | 149 | 68 |
| PNP.LockedNAND | 5 | 4 |
| PNP.LockedNANDBaseline | 57 | 22 |
| PNP.LockedNANDCarrierTrace | 327 | 201 |
| PNP.LockedNANDDirect | 84 | 57 |
| PNP.LockedNANDGlobalCandidates | 180 | 107 |
| PNP.LockedNANDGlobalSemanticThreshold | 8 | 7 |
| PNP.LockedNANDGlobalUnsatisfiableFinalZero | 2 | 2 |
| PNP.LockedNANDLocalBaseline | 30 | 22 |
| PNP.LockedNANDMacros | 288 | 98 |
| PNP.LockedNANDPrefix | 84 | 40 |
| PNP.LockedNANDResidualGainBound | 4 | 3 |
| PNP.LockedNANDThresholdBoundary | 60 | 35 |
| PNP.Main | 36 | 12 |
| PNP.NANDArbitrarySupportSplice | 76 | 51 |
| PNP.NANDBoundedPhysicalWindowSearch | 72 | 26 |
| PNP.NANDBoundedWindowSchedule | 18 | 12 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.NANDCausalBounds | 39 | 26 |
| PNP.NANDCausalGuard | 11 | 7 |
| PNP.NANDClosedSupportFullGain | 35 | 24 |
| PNP.NANDClosedSupportFullGainPrefix | 21 | 13 |
| PNP.NANDClosedSupportNestedCandidate | 9 | 6 |
| PNP.NANDClosedSupportNestedCost | 19 | 19 |
| PNP.NANDClosedSupportNestedGain | 6 | 6 |
| PNP.NANDClosedSupportNestedOrigin | 3 | 3 |
| PNP.NANDClosedSupportNestedProfile | 6 | 4 |
| PNP.NANDClosedSupportNestedProgram | 7 | 3 |
| PNP.NANDClosedSupportNestedSelection | 14 | 9 |
| PNP.NANDClosedSupportObservation | 15 | 9 |
| PNP.NANDClosedSupportProfile | 6 | 4 |
| PNP.NANDClosedSupportSquare | 6 | 4 |
| PNP.NANDClosedSupportUnion | 7 | 7 |
| PNP.NANDClosedWholeMinimum | 16 | 12 |
| PNP.NANDClosedWholeMinimumPrefix | 15 | 11 |
| PNP.NANDCompatibleSupportSlackComparison | 19 | 15 |
| PNP.NANDCompatibleSupportSlackMinimum | 37 | 25 |
| PNP.NANDCompatibleSupportSlackObstruction | 4 | 4 |
| PNP.NANDCompiledGateProvenance | 11 | 9 |
| PNP.NANDCompleteBoundedWindowSearch | 63 | 20 |
| PNP.NANDComposition | 77 | 38 |
| PNP.NANDComputedWireProfileCost | 3 | 3 |
| PNP.NANDConstantSourceIncidence | 14 | 9 |
| PNP.NANDDemandDeficit | 92 | 56 |
| PNP.NANDDemandForest | 35 | 13 |
| PNP.NANDEnumerator | 120 | 46 |
| PNP.NANDEssentialInputBound | 24 | 13 |
| PNP.NANDFreshFieldCost | 25 | 13 |
| PNP.NANDFreshFieldExtension | 20 | 12 |
| PNP.NANDFreshFieldProfile | 13 | 11 |
| PNP.NANDGateRenaming | 46 | 18 |
| PNP.NANDGateRenamingEncoding | 5 | 4 |
| PNP.NANDGrowingWindowUniverseBound | 17 | 15 |
| PNP.NANDGuardedSpineBoundedQuiet | 10 | 10 |
| PNP.NANDGuardedSpineCounts | 10 | 8 |
| PNP.NANDGuardedSpineCuts | 27 | 27 |
| PNP.NANDGuardedSpineFamily | 34 | 13 |
| PNP.NANDGuardedSpineJointCoverage | 8 | 8 |
| PNP.NANDGuardedSpineMinimum | 14 | 13 |
| PNP.NANDGuardedSpineOpenPrefix | 14 | 12 |
| PNP.NANDGuardedSpineOpenSegments | 59 | 51 |
| PNP.NANDGuardedSpineSupportMinimum | 7 | 7 |
| PNP.NANDJointAsymmetricBound | 13 | 13 |
| PNP.NANDMinimum | 56 | 23 |
| PNP.NANDNoReuseForest | 21 | 20 |
| PNP.NANDNormalizationAbstractValue | 78 | 41 |
| PNP.NANDNormalizationCausalBounds | 24 | 24 |
| PNP.NANDNormalizationTerm | 103 | 51 |
| PNP.NANDNormalizationTermTransport | 31 | 23 |
| PNP.NANDPhysicalGateProvenance | 69 | 35 |
| PNP.NANDPhysicalWindowBudget | 15 | 13 |
| PNP.NANDReadOnceSymmetry | 8 | 8 |
| PNP.NANDReadOnceTerm | 23 | 18 |
| PNP.NANDReindexedCompiledSplice | 15 | 12 |
| PNP.NANDReindexedOpenSemantics | 8 | 8 |
| PNP.NANDReindexedPorts | 53 | 33 |
| PNP.NANDReindexedReplacement | 14 | 13 |
| PNP.NANDReindexedSplice | 20 | 18 |
| PNP.NANDReindexedSupport | 49 | 39 |
| PNP.NANDReindexingCausalBounds | 4 | 4 |
| PNP.NANDSemanticGateRetraction | 19 | 9 |
| PNP.NANDSemantics | 198 | 80 |
| PNP.NANDSharedOutputOverlap | 26 | 26 |
| PNP.NANDSharingSourceIncidence | 8 | 4 |
| PNP.NANDSingleReuseForest | 58 | 40 |
| PNP.NANDSlack | 15 | 12 |
| PNP.NANDSourceListOrder | 18 | 12 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.NANDStructuralReindexing | 24 | 17 |
| PNP.NANDSupportRecordTransport | 17 | 11 |
| PNP.NANDTightDemandSymmetry | 16 | 15 |
| PNP.NANDTopologicalCausalBounds | 15 | 13 |
| PNP.NANDTopologicalCompiler | 162 | 74 |
| PNP.NANDTopologicalWireStructure | 8 | 7 |
| PNP.NANDTruthTable | 72 | 28 |
| PNP.NANDWireCarrier | 49 | 27 |
| PNP.NANDWireCarrierRecoding | 41 | 21 |
| PNP.NANDWireCarrierZeroCostExposure | 3 | 3 |
| PNP.NANDWireCausalBounds | 17 | 15 |
| PNP.NANDWireCausalExpansion | 70 | 47 |
| PNP.NANDWireDescendantCausalBounds | 5 | 5 |
| PNP.NANDWireDescendantCertificate | 65 | 28 |
| PNP.NANDWireDescendantCharges | 17 | 13 |
| PNP.NANDWireDescendantEvents | 20 | 15 |
| PNP.NANDWireDescendantGain | 27 | 12 |
| PNP.NANDWireDescendantInput | 246 | 133 |
| PNP.NANDWireDescendantLedger | 87 | 36 |
| PNP.NANDWireDescendantOwnership | 22 | 16 |
| PNP.NANDWireDescendantProperSupport | 44 | 25 |
| PNP.NANDWireDescendantRun | 63 | 23 |
| PNP.NANDWireDescendantStage | 41 | 18 |
| PNP.NANDWireFrontierLift | 64 | 31 |
| PNP.NANDWireHistoryAmbientOwnership | 48 | 23 |
| PNP.NANDWireHistoryArbitrarySupport | 19 | 15 |
| PNP.NANDWireHistoryCausalBounds | 16 | 12 |
| PNP.NANDWireHistoryOwnershipCharges | 19 | 15 |
| PNP.NANDWireHistoryPhysicalOwnership | 98 | 42 |
| PNP.NANDWireMatchedCancellation | 203 | 79 |
| PNP.NANDWireObligationHistory | 160 | 81 |
| PNP.NANDWireObligationHistoryExecution | 222 | 52 |
| PNP.NANDWireObligationHistoryR7 | 96 | 49 |
| PNP.NANDWireObligationHistoryState | 83 | 43 |
| PNP.NANDWireObligationRestoration | 141 | 59 |
| PNP.NANDWireOpenCertificate | 69 | 31 |
| PNP.NANDWireOpenPrimitiveOwnership | 19 | 12 |
| PNP.NANDWireOpenProgram | 148 | 44 |
| PNP.NANDWireOpenProgramInput | 131 | 56 |
| PNP.NANDWireOpenProgramOwnership | 112 | 53 |
| PNP.NANDWireOpenProperOwnership | 17 | 13 |
| PNP.NANDWireOpenProperSupport | 43 | 24 |
| PNP.NANDWireOpenSupportOwnership | 19 | 14 |
| PNP.NANDWireOpenSupportSplice | 44 | 25 |
| PNP.NANDWireProfileAmbient | 21 | 12 |
| PNP.NANDWireProfileAvailability | 28 | 21 |
| PNP.NANDWireProfileExposure | 51 | 41 |
| PNP.NANDWireProfileFieldClosed | 9 | 5 |
| PNP.NANDWireProfileRestoration | 55 | 36 |
| PNP.NANDWireQuotientLift | 70 | 32 |
| PNP.NANDWireRecodingInput | 43 | 19 |
| PNP.NANDWireRecodingOwnership | 20 | 13 |
| PNP.NANDWireRecodingState | 35 | 18 |
| PNP.NANDWireStructuralOwnership | 10 | 9 |
| PNP.NANDWireStructuralProgram | 3 | 3 |
| PNP.NANDWireStructuralReindexing | 14 | 11 |
| PNP.NANDWireStructuralState | 31 | 14 |
| PNP.NANDWireUnaryArbitrarySupport | 79 | 45 |
| PNP.NANDWireUnaryCausalBounds | 17 | 17 |
| PNP.NANDWireUnaryFrontier | 64 | 41 |
| PNP.NANDWireUnaryRealization | 66 | 28 |
| PNP.NANDWireUnarySupportSearch | 79 | 47 |
| PNP.NANDWireZeroUnaryClosure | 154 | 61 |
| PNP.NANDZeroCostExposure | 115 | 54 |
| PNP.PCCMin | 45 | 8 |
| PNP.PCCMinCheckedPacketBN6BCELActivationRoute | 120 | 35 |
| PNP.PCCMinCheckedPacketBN6BCELCanonicalConstantCut Basis | 32 | 11 |
| PNP.PCCMinCheckedPacketBN6BCELCanonicalCutLedger | 2 | 2 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.PCCMinCheckedPacketBN6BCELCanonicalGrouping | 49 | 17 |
| PNP.PCCMinCheckedPacketBN6BCELDerivedFamily | 94 | 28 |
| PNP.PCCMinCheckedPacketBN6BCELSparseActivationRoute | 55 | 22 |
| PNP.PCCMinCheckedPacketBN6HBZeroSlackBridge | 71 | 18 |
| PNP.PCCMinCheckedPacketHBZeroSlackBridge | 74 | 20 |
| PNP.PCCMinCheckedPacketPkgCAmbientBN4Extraction Route | 144 | 42 |
| PNP.PCCMinCheckedPacketPkgCBN6BCELSourceRoute | 78 | 21 |
| PNP.PCCMinCheckedPacketPkgCRestorationCoverageBN6 BCELRoute | 68 | 22 |
| PNP.PCCMinCheckedPacketPkgCRestorationCoverageCharge Descent | 92 | 33 |
| PNP.PCCMinCheckedPacketRankedSelector | 76 | 20 |
| PNP.PCCMinConstantPropagation | 54 | 26 |
| PNP.PCCMinConstructiveNANDSharing | 55 | 27 |
| PNP.PCCMinDeadSupportContext | 59 | 36 |
| PNP.PCCMinDeadSupportFullMode | 108 | 40 |
| PNP.PCCMinNormalizeOracleComposition | 69 | 19 |
| PNP.PCCMinOutputConePruning | 60 | 46 |
| PNP.PCCMinPhysicalNormalizationClosure | 178 | 69 |
| PNP.PCCMinRankOrderedOracle | 204 | 47 |
| PNP.PCCMinTotalOracleLoop | 87 | 36 |
| PNP.ResidualBand | 4 | 3 |
| PNP.ResidualGainChain | 22 | 9 |
| PNP.ResidualGainStopping | 14 | 12 |
| PNP.ResidualIndependentMaterializerCost | 11 | 10 |
| PNP.ResidualRoutes | 141 | 46 |
| PNP.ResidualTerminalBCELAnchorNucleus | 408 | 138 |
| PNP.ResidualTerminalBCELPacketNoLowerZeroSlackSidecar | 11 | 5 |
| PNP.ResidualTerminalBN2SquareLegitimacy | 40 | 22 |
| PNP.ResidualTerminalBN3RequestEnvelope | 84 | 44 |
| PNP.ResidualTerminalBN4ActivationCancellation | 227 | 97 |
| PNP.ResidualTerminalBN5FullShadowLocalization | 278 | 100 |
| PNP.ResidualTerminalBN6CanonicalCutLedger | 7 | 3 |
| PNP.ResidualTerminalBN6CanonicalPositiveCellGrouping | 69 | 41 |
| PNP.ResidualTerminalBN6HypergraphPacket | 94 | 37 |
| PNP.ResidualTerminalBudgetEnvelopeResolver | 95 | 36 |
| PNP.ResidualTerminalBudgetNoLowerLedger | 42 | 15 |
| PNP.ResidualTerminalBudgetZeroSlackSidecar | 28 | 9 |
| PNP.ResidualTerminalCandidateSaturation | 56 | 11 |
| PNP.ResidualTerminalConstantCutHypergraphRigidity | 155 | 109 |
| PNP.ResidualTerminalConsumerAntichainNormalForm | 64 | 38 |
| PNP.ResidualTerminalExecutableSaturation | 95 | 36 |
| PNP.ResidualTerminalFiniteBCELPacketActivation Obstruction | 39 | 16 |
| PNP.ResidualTerminalFiniteBCELPacketCarrierCoherence | 26 | 11 |
| PNP.ResidualTerminalFiniteBCELReady | 34 | 17 |
| PNP.ResidualTerminalFiniteSaturatePositive | 68 | 23 |
| PNP.ResidualTerminalFourCornerCarrier | 93 | 43 |
| PNP.ResidualTerminalFourCornerOpenTransport | 11 | 10 |
| PNP.ResidualTerminalFourCornerOptimumCoherence | 179 | 56 |
| PNP.ResidualTerminalFourCornerOptimumCompatibility | 83 | 38 |
| PNP.ResidualTerminalFourCornerSideTightCompletion | 65 | 24 |
| PNP.ResidualTerminalFourCornerTightBasisMaximum | 57 | 23 |
| PNP.ResidualTerminalFrontierPushout | 52 | 25 |
| PNP.ResidualTerminalFullBridge | 56 | 23 |
| PNP.ResidualTerminalGainProfileFirewall | 95 | 32 |
| PNP.ResidualTerminalGovernedSupportCompletion | 68 | 30 |
| PNP.ResidualTerminalHBActiveDependencyClosure | 16 | 9 |
| PNP.ResidualTerminalHBBlockerGraphAcyclicity | 93 | 32 |
| PNP.ResidualTerminalHBDependencyTableClosure | 47 | 25 |
| PNP.ResidualTerminalHBExecutableSelectorSilenceInduction | 11 | 8 |
| PNP.ResidualTerminalHBSelectorSilenceClosure | 4 | 4 |
| PNP.ResidualTerminalHNBWLCertifiedPathMinimum | 137 | 50 |
| PNP.ResidualTerminalHResolveCertifiedPathFamily | 43 | 18 |
| PNP.ResidualTerminalHResolveCoverageLedger | 65 | 23 |
| PNP.ResidualTerminalHResolveHDisjointFamily | 115 | 47 |
| PNP.ResidualTerminalHResolveSupportResolver | 15 | 10 |
| PNP.ResidualTerminalHResolveZeroSlackSidecar | 37 | 13 |

| Module | Declarations | Theorems |
|---|---|---|
| PNP.ResidualTerminalInterfaceExposureRouting | 224 | 75 |
| PNP.ResidualTerminalModeFirewall | 148 | 41 |
| PNP.ResidualTerminalOriginKernelObligationRouting | 330 | 97 |
| PNP.ResidualTerminalPacketBN4ActivationRouteReflection | 40 | 33 |
| PNP.ResidualTerminalPacketBN5ObligationRouteReflection | 62 | 37 |
| PNP.ResidualTerminalPacketBudgetHBActivityBinding | 10 | 8 |
| PNP.ResidualTerminalPacketBudgetNoLowerComposition | 10 | 8 |
| PNP.ResidualTerminalPacketBudgetNoLowerZeroSlackSidecar | 51 | 9 |
| PNP.ResidualTerminalPacketBudgetRouteReflection | 60 | 38 |
| PNP.ResidualTerminalPacketChargeRouteReflection | 31 | 24 |
| PNP.ResidualTerminalPacketChargeSurplus | 48 | 21 |
| PNP.ResidualTerminalPacketColourRouteReflection | 36 | 28 |
| PNP.ResidualTerminalPacketDescentNoLowerBinding | 6 | 4 |
| PNP.ResidualTerminalPacketDescentRouteReflection | 21 | 14 |
| PNP.ResidualTerminalPacketDirectionRouteReflection | 58 | 36 |
| PNP.ResidualTerminalPacketExactRouteReflection | 28 | 21 |
| PNP.ResidualTerminalPacketFrontierRouteReflection | 60 | 35 |
| PNP.ResidualTerminalPacketNoLowerLedger | 10 | 8 |
| PNP.ResidualTerminalPacketRankRouteReflection | 24 | 17 |
| PNP.ResidualTerminalPacketSelectorCodec | 25 | 11 |
| PNP.ResidualTerminalPacketSelectorFaithfulnessRouting | 114 | 42 |
| PNP.ResidualTerminalPacketSelectorFaithfulnessTable | 9 | 8 |
| PNP.ResidualTerminalPacketSelectorFirstRouteOutcome | 9 | 7 |
| PNP.ResidualTerminalPacketSelectorFirstRouteSemantics | 19 | 16 |
| PNP.ResidualTerminalPacketSelectorGainCoverage | 58 | 17 |
| PNP.ResidualTerminalPacketSelectorGainScan | 73 | 24 |
| PNP.ResidualTerminalPacketSelectorHandles | 22 | 12 |
| PNP.ResidualTerminalPacketSelectorPayloadRealization | 53 | 25 |
| PNP.ResidualTerminalPacketSelectorSeeds | 17 | 9 |
| PNP.ResidualTerminalPacketSelectorUniverse | 15 | 6 |
| PNP.ResidualTerminalPacketSelectorUniverseGainScan | 89 | 29 |
| PNP.ResidualTerminalPacketSemanticHNActivityBinding | 9 | 6 |
| PNP.ResidualTerminalPacketTypedRealizerContract | 166 | 64 |
| PNP.ResidualTerminalPacketUnitChargeBlueprintRealizer | 173 | 62 |
| PNP.ResidualTerminalPhysicalChargeLedger | 64 | 21 |
| PNP.ResidualTerminalPhysicalGain | 8 | 6 |
| PNP.ResidualTerminalPhysicalOwnership | 18 | 13 |
| PNP.ResidualTerminalPhysicalSaturationAccounting | 4 | 4 |
| PNP.ResidualTerminalPhysicalSupportCompletion | 114 | 50 |
| PNP.ResidualTerminalPkgCAmbientBN4Ledger | 60 | 22 |
| PNP.ResidualTerminalPkgCAmbientBN4ResidualReduction | 43 | 16 |
| PNP.ResidualTerminalPkgCBN6PositiveCellization | 79 | 35 |
| PNP.ResidualTerminalPkgCRestorationCoverageAmbient Route | 122 | 46 |
| PNP.ResidualTerminalPkgCRestorationCoverageBN6Ledger | 72 | 23 |
| PNP.ResidualTerminalPkgCSameKeyCancellation | 75 | 37 |
| PNP.ResidualTerminalPkgCSeparatingConsumers | 131 | 51 |
| PNP.ResidualTerminalPkgCTypedRestoration | 86 | 35 |
| PNP.ResidualTerminalProfileLocality | 8 | 7 |
| PNP.ResidualTerminalProjectionMinimum | 36 | 23 |
| PNP.ResidualTerminalProjectionSquare | 21 | 17 |
| PNP.ResidualTerminalProjectionTransfer | 29 | 8 |
| PNP.ResidualTerminalProperSupport | 56 | 27 |
| PNP.ResidualTerminalRankWF | 54 | 23 |
| PNP.ResidualTerminalSaturatedSupportContext | 15 | 13 |
| PNP.ResidualTerminalSaturation | 170 | 60 |
| PNP.ResidualTerminalSaturationCostBalance | 162 | 48 |
| PNP.ResidualTerminalSaturationPositivityFirewall | 56 | 23 |
| PNP.ResidualTerminalSaturationTraceFidelity | 7 | 7 |
| PNP.ResidualTerminalSelectorHBZeroSlackSidecar | 34 | 12 |
| PNP.ResidualTerminalSideTightMinimum | 98 | 35 |
| PNP.ResidualTerminalSupportExtraction | 110 | 67 |
| PNP.ResidualTerminalSupportSquareClosure | 77 | 30 |
| PNP.ResidualTerminalV53CanonicalConstantCutBasis | 94 | 46 |
| PNP.ResidualTerminalV53SparseProperCutBasis | 108 | 65 |
| PNP.ResidualTerminalZeroSlackPacketSelectorHBCoherence | 8 | 6 |
| PNP.SAT | 8 | 4 |
| PNP.ZeroSlack | 50 | 9 |

# 6 Remaining formal blockers

Five formal blockers remain active:

1. `Formal.ConcreteSAT`
2. `Formal.ResidualBandMinimizer`
3. `Formal.ZeroSlack`
4. `Formal.PolynomialRuntimeAndCertificateBounds`
5. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. The gain-chain theorem bounds every supplied verified sequence, including locked-family sequences by four, but does not generate or complete that sequence. None of these results proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

# 7 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

| | |
|---|---|
| Source tag | `final-pnp-proof-report-hardened-7072f8d` |
| Source commit | `7072f8d0bda6d44d240f9bb3fad624fd357e1278` |
| Archive manifest | `archive/legacy-v0/ARCHIVE.json` |

# 8 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is `PNP-LEAN-THEOREM-INVENTORY-2026-09-10-230`. Its exact byte digest is `49cfcf6de14f560842766adef7fe9e54437ce57ac2a6c4ace650308164be890a`. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is `74e468b5d0b56a3eeb98794e81848300677a8412a2fc6d58733f856e452702ed`; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.